به نام خدا



درس هوش مصنوعی- ترم ۴۰۵۱

توضیح بلاک کد ها:

بخش DT_Library :

```python
class Node:
    def __init__(
            self,
            feature_index=None,
            threshold=None,
            left=None,
            right=None,
            value=None,
            samples=0,
            impurity=0.0,
            class_counts=None,
        ):
            self.feature_index = feature_index
            self.threshold = threshold
            self.left = left
            self.right = right
            self.value = value
            self.samples = samples
            self.impurity = impurity
            self.class_counts = class_counts

    def is_leaf(self):
        if self.left == None and self.right == None:
            return True

        return False
```

کلاس Node

کلاس Node برای ساخت گره‌های درخت تصمیم استفاده می‌شود. هر گره می‌تواند یک گره داخلی برای تقسیم داده‌ها یا یک گره برگ برای مشخص کردن کلاس پیش‌بینی‌شده باشد.

ویژگی feature_index شماره‌ی ویژگی انتخاب‌شده برای تقسیم را مشخص می‌کند و threshold آستانه‌ی تقسیم را نگه می‌دارد. ویژگی‌های left و right نیز برای نگهداری گره‌های فرزند استفاده می‌شوند. ویژگی value کلاس پیش‌بینی‌شده را در گره برگ ذخیره می‌کند.

همچنین samples تعداد نمونه‌های موجود در گره، impurity میزان ناخالصی کلاس‌ها و class_counts تعداد نمونه‌های هر کلاس را نگه می‌دارند.

در ادامه، متد is_leaf() بررسی می‌کند که آیا گره فرزندی دارد یا خیر. اگر هر دو فرزند چپ و راست برابر None باشند، مقدار True برگردانده می‌شود و گره به عنوان برگ شناخته می‌شود؛ در غیر این صورت، مقدار False برگردانده می‌شود.

کلاس Decision Tree :

```python
def __init__(
    self,
    criterion="gini",
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features=None,
    random_state=None,
):
    # Hyperparameters
    # criterion_type ="gini or entropy"
    self.criterion = criterion
    self.max_depth = max_depth
    self.min_samples_split = min_samples_split
    self.min_samples_leaf = min_samples_leaf
    self.max_features = max_features
    self.random_state = random_state
    #
    # Learned attributes
    self.root_ = None
    self.n_features_ = None
    self.n_classes_ = None
    self.classes_ = None
    self.n_nodes_ = None
    self.depth_ = None
    self.feature_importances_ = None
```

پارامتر criterionمعیار محاسبه‌ی ناخالصی را تعیین می‌کند که می‌تواند giniیا entropyباشد. پارامتر max_depthحداکثر عمق درخت را مشخص می‌کند .min_samples_split حداقل تعداد نمونه‌های لازم برای تقسیم یک گره و min_samples_leafحداقل تعداد نمونه‌های لازم برای تشکیل یک گره برگ را تعیین می‌کنند.

پارامتر max_featuresتعداد ویژگی‌های مورد بررسی برای تقسیم و random_stateمقدار مربوط به کنترل تصادفی بودن عملیات را مشخص می‌کند.

در ادامه، چند ویژگی برای نگهداری اطلاعاتی که هنگام آموزش مدل به دست می‌آیند تعریف شده‌اند root_ . ریشه‌ی درخت را نگه می‌دارد، n_features_تعداد ویژگی‌های ورودی، n_classes_تعداد کلاس‌ها و classes_فهرست کلاس‌های موجود را ذخیره می‌کنند. همچنین n_nodes_تعداد گره‌های درخت، depth_عمق نهایی درخت و feature_importances_میزان اهمیت هر ویژگی در پیش‌بینی را نگه می‌دارند.

مقدار اولیه‌ی این ویژگی‌ها Noneاست، زیرا اطلاعات آن‌ها پس از آموزش مدل مشخص می‌شود.

```python
def _weighted_entropy(self, y_left, y_right):
    # TODO
    Total_samples = len(y_left) + len(y_right)

    if Total_samples == 0:
        return 0.0

    result = 0.0

    for y_child in [y_left, y_right]:
        n = len(y_child)
        if n == 0:
            continue

        entropy = 0.0



        for value in np.unique(y_child):
            p = np.sum(y_child == value) / n
            entropy -= p * np.log2(p)

        result += (n / Total_samples) * entropy

    return result
```

محاسبه‌ی Weighted Entropy

تابع weighted_entropy_ میزان آنتروپی وزنی را پس از تقسیم یک گره به دو فرزند محاسبه می‌کند. آنتروپی نشان می‌دهد که کلاس‌های مختلف تا چه اندازه در یک گره با هم ترکیب شده‌اند؛ هرچه این مقدار کمتر باشد، داده‌های گره از نظر کلاس خالص‌تر هستند.

ابتدا تعداد کل نمونه‌های دو فرزند محاسبه می‌شود. اگر تعداد کل نمونه‌ها صفر باشد، مقدار صفر برگردانده می‌شود. سپس برای هر فرزند، تعداد نمونه‌ها و آنتروپی آن محاسبه می‌شود. احتمال هر کلاس از تقسیم تعداد نمونه‌های آن کلاس بر تعداد کل نمونه‌های همان فرزند به دست می‌آید و آنتروپی با استفاده از فرمول

زیرمحاسبه می‌شود:

$$\left[H(Y) = -\sum_{i=1}^{k} p_i \log_2(p_i)\right]$$

در این فرمول، (p_i)احتمال تعلق نمونه‌ها به کلاس (i) و (k) تعداد کلاس‌های موجود است.

در پایان، آنتروپی هر فرزند در نسبت تعداد نمونه‌های آن به کل نمونه‌های دو فرزند ضرب می‌شود و نتایج با هم جمع می‌شوند:

$$\left[H_{\text{weighted}} = \frac{N_L}{N} H_L + \frac{N_R}{N} H_R\right]$$

در این رابطه، (N_L)و (N_R) تعداد نمونه‌های فرزند چپ و راست و (N) تعداد کل نمونه‌ها هستند. نتیجه‌ی نهایی، آنتروپی وزنی تقسیم‌بندی است که برای ارزیابی کیفیت تقسیم گره در درخت تصمیم استفاده می‌شود.

```python
def _weighted_gini(self, y_left, y_right):

    # TODO
    Total_samples = len(y_right) + len(y_left)

    if Total_samples == 0:
        return 0.0

    result = 0.0

    for y_child in [y_left , y_right]:
        n = len(y_child)
        if n == 0:
            continue


        sumOfPowers = 0.0
        for value in np.unique(y_child):
            p = np.sum(y_child == value) / n
            sumOfPowers += (p * p)


        gini = 1 - sumOfPowers
        result += (n / Total_samples) * gini


    return result
```

محاسبه‌ی Weighted Gini

تابع weighted_gini_برای محاسبه‌ی ناخالصی وزنی پس از تقسیم یک گره به دو فرزند استفاده می‌شود. معیار Gini میزان ترکیب شدن کلاس‌های مختلف در یک گره را نشان می‌دهد. هرچه مقدار آن کمتر باشد، نمونه‌های گره بیشتر متعلق به یک کلاس هستند.

ابتدا تعداد کل نمونه‌های دو فرزند محاسبه می‌شود. اگر تعداد کل نمونه‌ها صفر باشد، تابع مقدار صفر را برمی‌گرداند. سپس هر فرزند به‌صورت جداگانه بررسی می‌شود و احتمال هر کلاس با تقسیم تعداد نمونه‌های آن کلاس بر تعداد کل نمونه‌های همان فرزند به دست می‌آید.

ناخالصی Gini برای هر فرزند با فرمول زیر محاسبه می‌شود:

$$\left[Gini = 1 - \sum_{i=1}^{k} p_i^2 \right]$$

در این فرمول، (p_i)احتمال تعلق نمونه‌ها به کلاس (i) و (k) تعداد کلاس‌های موجود در گره است.

در مرحله‌ی بعد، مقدار Gini هر فرزند در نسبت تعداد نمونه‌های آن به تعداد کل نمونه‌های دو فرزند ضرب می‌شود. سپس مقادیر به دست آمده با یکدیگر جمع می‌شوند:

$$\left[Gini_{\text{weighted}} = \frac{N_L}{N} Gini_L + \frac{N_R}{N} Gini_R\right]$$

در این رابطه، (N_L)و (N_R) تعداد نمونه‌های فرزند چپ و راست و (N) تعداد کل نمونه‌ها هستند. در نهایت، تابع مقدار Gini وزنی را برمی‌گرداند تا از آن برای ارزیابی کیفیت تقسیم‌بندی درخت تصمیم استفاده شود. تقسیم‌بندی‌ای که Gini وزنی کمتری داشته باشد، معمولاً باعث خالص‌تر شدن گره‌های فرزند می‌شود.

```python
def _information_gain(self, y, y_left, y_right):

    # TODO
    if self.criterion == "gini":
        parent_impurity = self._gini(y)
        weighted_impurity = self._weighted_gini(y_left, y_right)

    elif self.criterion == "entropy":
        parent_impurity = self._entropy(y)
        weighted_impurity = self._weighted_entropy(y_left, y_right)

    return parent_impurity - weighted_impurity
```

محاسبه‌ی Information Gain

تابع information_gain_میزان بهبود حاصل از تقسیم یک گره را محاسبه می‌کند. این معیار نشان می‌دهد که تقسیم داده‌ها تا چه اندازه باعث کاهش ناخالصی گره‌ها می‌شود.

ابتدا با توجه به مقدار پارامتر criterion، ناخالصی گره اصلی محاسبه می‌شود. اگر معیار انتخاب‌شده giniباشد، تابع gini_و اگر entropyباشد، تابع entropy_برای محاسبه‌ی ناخالصی گره اصلی استفاده می‌شود.

در مرحله‌ی بعد، ناخالصی وزنی دو گره فرزند با استفاده از تابع متناسب با معیار انتخاب‌شده محاسبه می‌شود. این مقدار با در نظر گرفتن تعداد نمونه‌های هر فرزند به دست می‌آید.

در نهایت، مقدار ناخالصی وزنی فرزندان از ناخالصی گره اصلی کم می‌شود:

$$\left[IG = I_{\text{parent}} - I_{\text{children}}\right]$$

در این فرمول، (I_{parent}) ناخالصی گره اصلی و(I_{children}) ناخالصی وزنی دو گره فرزند است.

هرچه مقدار Information Gain بیشتر باشد، تقسیم انجام‌شده کاهش بیشتری در ناخالصی ایجاد کرده است. بنابراین این معیار برای ارزیابی کیفیت تقسیم‌بندی‌ها و انتخاب تقسیم مناسب در درخت تصمیم استفاده می‌شود.

```
def _get_candidate_thresholds(self, feature_values):

    # TODO

    values = np.unique(feature_values)

    thresholds = []


    for i in range(len(values) - 1):
        threshold = (values[i] + values[i + 1]) / 2
        thresholds.append(threshold)

    return thresholds
```

تولید آستانه‌ها(_get_candidate_thresholds)

تابع get_candidate_thresholds_ برای تولید آستانه‌های احتمالی تقسیم داده‌ها بر اساس یک ویژگی استفاده می‌شود. این آستانه‌ها در مرحله‌ی بعد توسط تابع find_best_split_ بررسی می‌شوند تا بهترین تقسیم انتخاب شود.

ابتدا مقادیر تکراری حذف می‌شوند و مقادیر یکتای ویژگی به دست می‌آیند. سپس با یک حلقه، هر دو مقدار یکتای متوالی بررسی می‌شوند و میانگین آن‌ها به عنوان یک آستانه‌ی احتمالی در نظر گرفته می‌شود:

$$\left[threshold = \frac{x_i + x_{i+1}}{2}\right]$$

استفاده از میانگین دو مقدار متوالی باعث می‌شود آستانه‌ای بین آن‌ها قرار بگیرد و بتوان نمونه‌ها را به دو گروه تقسیم کرد. همچنین نیازی نیست تمام اعداد ممکن به عنوان آستانه بررسی شوند؛ زیرا مقادیر آستانه‌ای که بین دو مقدار یکتای متوالی قرار دارند، معمولاً تقسیم‌بندی یکسانی ایجاد می‌کنند.

در پایان، تمام آستانه‌های تولیدشده در یک لیست قرار می‌گیرند و به تابع فراخواننده برگردانده می‌شوند. این روش امکان بررسی تقسیم‌های مختلف را فراهم می‌کند، اما با افزایش تعداد مقادیر یکتای ویژگی، تعداد آستانه‌های مورد بررسی نیز بیشتر می‌شود و می‌تواند زمان آموزش درخت را افزایش دهد.

```python
def _gini(self, y):

    # TODO
    n = len(y)
    if n == 0:
        return 0.0


    sumOfPowers = 0.0
    for value in np.unique(y):
        p = np.sum(y == value) / n
        sumOfPowers += (p * p)


    gini = 1 - sumOfPowers



    return gini
```

محاسبه‌ی ناخالصی(_gini)

این تابع همان منطق قسمتی از تابع Weighted_gini_ را دارد . از این تابع در مراحل بعد استفاده میشود.

```python
def _entropy(self, y):

    # TODO
    n = len(y)
    if n == 0:
        return 0.0


    entropy = 0.0



    for value in np.unique(y):
        p = np.sum(y == value) / n
        entropy -= p * np.log2(p)



    return entropy
```

محاسبه‌ی آنتروپی(_entropy)

این تابع همان منطق قسمتی از تابع Weighted_entropy_ را دارد . از این تابع در مراحل بعد استفاده میشود.

```python
def _calculate_impurity(self, y):

    # TODO

    if self.criterion == "gini":
        return self._gini(y)
    elif self.criterion == "entropy":
        return self._entropy(y)
```

محاسبه‌ی ناخالصی گره(_calculate_impurity)

تابع calculate_impurity_برای محاسبه‌ی ناخالصی یک گره بر اساس معیار انتخاب‌شده در مدل استفاده می‌شود. هدف این تابع، یکپارچه کردن روش محاسبه‌ی ناخالصی است تا با توجه به مقدار پارامتر criterion، معیار مناسب انتخاب شود.

اگر معیار giniباشد، تابع gini_برای محاسبه‌ی ناخالصی فراخوانی می‌شود. اگر معیار entropyباشد، تابع entropy_مقدار آنتروپی گره را محاسبه می‌کند.

در نهایت، مقدار ناخالصی محاسبه‌شده برگردانده می‌شود.

```python
def _split(self, X, y, feature_index, threshold):

    # TODO
    X = np.asarray(X, dtype=float)
    y = np.asarray(y)

    left_mask = X[:, feature_index] <= threshold


    X_left = X[left_mask]
    y_left = y[left_mask]

    X_right = X[~ left_mask]
    y_right = y[~ left_mask]

    return X_left, y_left, X_right, y_right
```

تابع split_

تابع split_وظیفه‌ی تقسیم داده‌ها به دو گروه چپ و راست را بر اساس یک ویژگی و آستانه‌ی مشخص (threshold)بر عهده دارد. ابتدا با استفاده از feature_index، ویژگی موردنظر انتخاب شده و مقدار آن برای هر نمونه با آستانه مقایسه می‌شود.

نمونه‌هایی که مقدار ویژگی آن‌ها کمتر یا مساوی آستانه باشد، در گره چپ قرار می‌گیرند و سایر نمونه‌ها به گره راست منتقل می‌شوند. این تقسیم‌بندی هم‌زمان روی ماتریس ویژگی‌ها (X) و برچسب‌ها (y) اعمال می‌شود تا ارتباط هر نمونه با برچسب مربوط به آن حفظ شود.

در نهایت، تابع چهار آرایه‌ی X_left، y_left، X_right و y_rightرا برمی‌گرداند تا در ادامه‌ی فرایند ساخت درخت تصمیم استفاده شوند.

```python
def _find_best_split(self, X, y):
    X = np.asarray(X, dtype=float)
    y = np.asarray(y)
    if self.n_features_ is None:
        self.n_features_ = X.shape[1]
    best_gain = -1.0
    best_feature = None
    best_threshold = None

    for feature_index in range(self.n_features_):

        feature_values = np.unique(X[:, feature_index])

        if len(feature_values) < 2:
            continue


        thresholds = []
        thresholds = self._get_candidate_thresholds(feature_values)

        for threshold in thresholds:

            left_mask = X[:, feature_index] <= threshold
            right_mask = X[:, feature_index] > threshold

            y_left = y[left_mask]
            y_right = y[right_mask]


            if (
                len(y_left) < self.min_samples_leaf
                or len(y_right) < self.min_samples_leaf
            ):
                continue

            gain = self._information_gain(
                y, y_left, y_right
            )

            if gain > best_gain:
                best_gain = gain
                best_feature = feature_index
                best_threshold = threshold


    if best_feature is None:
     return None

    return best_feature, best_threshold
```

پیدا کردن بهترین تقسیم درخت(_find_best_split)

تابع find_best_split_وظیفه دارد بهترین ویژگی و آستانه را برای تقسیم داده‌های موجود در یک گره پیدا کند. هدف این است که تقسیم انتخاب‌شده بیشترین کاهش ناخالصی را ایجاد کند.

در ابتدا، ویژگی‌ها یکی‌یکی بررسی می‌شوند. برای هر ویژگی، مقادیر یکتای آن استخراج شده و با استفاده از تابع get_candidate_thresholds_آستانه‌های احتمالی تقسیم به دست می‌آیند. اگر یک ویژگی کمتر از دو مقدار متفاوت داشته باشد، امکان تقسیم بر اساس آن وجود ندارد و از بررسی آن صرف‌نظر می‌شود.

برای هر آستانه، نمونه‌ها به دو گروه تقسیم می‌شوند. نمونه‌هایی که مقدار ویژگی آن‌ها کمتر یا مساوی آستانه باشد، در گروه چپ و نمونه‌هایی که مقدارشان بیشتر از آستانه باشد، در گروه راست قرار می‌گیرند.

سپس تعداد نمونه‌های هر گروه بررسی می‌شود. اگر تعداد نمونه‌های یکی از گروه‌ها از مقدار min_samples_leafکمتر باشد، آن تقسیم معتبر در نظر گرفته نمی‌شود. در غیر این صورت، مقدار Information Gainبا توجه به معیار انتخاب‌شده، یعنی Gini یاEntropy ، محاسبه می‌شود.

در طول بررسی ویژگی‌ها و آستانه‌ها، بهترین مقدار Information Gain و ویژگی و آستانه‌ی مربوط به آن ذخیره می‌شوند. در پایان، اگر هیچ تقسیم معتبری پیدا نشده باشد، مقدار Noneبرگردانده می‌شود؛ در غیر این صورت، شماره‌ی بهترین ویژگی و آستانه‌ی آن به‌صورت یک زوج برگردانده می‌شوند.

```python
def _is_stopping_condition(self, X, y, depth):

    # TODO
    n_samples = len(y)

    if self.criterion == "gini":
        impurity = self._gini(y)
    elif self.criterion == "entropy":
        impurity = self._entropy(y)


    if (
    impurity == 0
    or (self.max_depth is not None and depth >= self.max_depth)
    or n_samples < self.min_samples_split
    ):
        return True

    return False
```

تابع _is_stopping_condition

تابع is_stopping_condition_بررسی می‌کند که آیا تقسیم گره فعلی باید متوقف شود یا خیر. ابتدا تعداد نمونه‌های گره محاسبه می‌شود و سپس ناخالصی آن بر اساس معیار انتخاب‌شده، یعنی Gini یاEntropy ، به دست می‌آید.

اگر یکی از سه شرط زیر برقرار باشد، تابع مقدار Trueرا برمی‌گرداند و گره به یک برگ تبدیل می‌شود:

- ناخالصی گره صفر باشد؛ یعنی تمام نمونه‌ها به یک کلاس تعلق داشته باشند.
- عمق درخت به حداکثر مقدار تعیین‌شده (max_depth) رسیده باشد.
- تعداد نمونه‌ها کمتر از min_samples_splitباشد.

در غیر این صورت، مقدار Falseبرگردانده می‌شود و فرایند تقسیم گره ادامه پیدا می‌کند. این شرایط از تقسیم‌های غیرضروری جلوگیری کرده و به کنترل پیچیدگی درخت تصمیم کمک می‌کنند.

```python
def _create_leaf(self, y):

    # TODO
    n_samples = len(y)

    if self.criterion == "gini":
        impurity = self._gini(y)
    elif self.criterion == "entropy":
        impurity = self._entropy(y)

    values, counts = np.unique(y, return_counts=True)

    majority_class = values[0]
    max_count = counts[0]

    for i in range(1, len(counts)):
        if counts[i] > max_count:
            max_count = counts[i]
            majority_class = values[i]

    class_counts = {}

    for i in range(len(values)):
        class_counts[values[i]] = counts[i]

    return Node(
        value=majority_class,
        samples=n_samples,
        impurity=impurity,
        class_counts=class_counts
    )
```

تابع _create_leaf

تابع create_leaf_برای ایجاد یک گره برگ در درخت تصمیم استفاده می‌شود. ابتدا تعداد نمونه‌ها و میزان ناخالصی گره بر اساس معیار انتخاب‌شده، یعنی Gini یاEntropy ، محاسبه می‌شود.

سپس کلاس‌های موجود در گره و تعداد نمونه‌های هر کلاس مشخص می‌شوند. کلاسی که بیشترین تعداد نمونه را دارد، به‌عنوان کلاس پیش‌بینی‌شده‌ی برگ انتخاب می‌شود .(majority_class) همچنین، تعداد نمونه‌های هر کلاس در دیکشنری class_countsذخیره می‌شود.

در پایان، یک شیء از کلاس Nodeساخته می‌شود که شامل کلاس پیش‌بینی‌شده، تعداد نمونه‌ها، میزان ناخالصی و توزیع کلاس‌ها است. این اطلاعات برای پیش‌بینی و بررسی وضعیت گره در درخت تصمیم کاربرد دارند.

```python
def _predict_one(self, x):

    # TODO
    node = self.root_
    while not node.is_leaf():
            if x[node.feature_index] <= node.threshold:
                node = node.left
            else:
                node = node.right

    return node.value
```

تابع _predict_one

تابع predict_one_برای پیش‌بینی کلاس یک نمونه‌ی جدید استفاده می‌شود. ابتدا حرکت از ریشه‌ی درخت (root_) آغاز می‌شود و تا زمانی که به یک گره برگ نرسیده باشد، مسیر مناسب را انتخاب می‌کند.

در هر گره، مقدار ویژگی مشخص‌شده با feature_indexبا آستانه‌ی thresholdمقایسه می‌شود. اگر مقدار ویژگی کمتر یا مساوی آستانه باشد، حرکت به گره چپ ادامه پیدا می‌کند؛ در غیر این صورت، گره راست انتخاب می‌شود.

این فرایند تا رسیدن به یک برگ ادامه می‌یابد و در نهایت مقدار valueآن برگ به‌عنوان کلاس پیش‌بینی‌شده برگردانده می‌شود.

```python
def predict(self, X):

    # TODO
    if not hasattr(self, "root_") or self.root_ is None:
        raise RuntimeError("DecisionTree must be fitted before predict.")

    X = np.asarray(X, dtype=float)
    predictions = []
    for value in X:
        predictions.append(self._predict_one(value))

    return predictions
```

تابع predict

تابع predictبرای پیش‌بینی کلاس چندین نمونه به‌صورت هم‌زمان استفاده می‌شود. ابتدا بررسی می‌کند که درخت تصمیم آموزش داده شده باشد؛ در غیر این صورت، خطای RuntimeErrorایجاد می‌کند. سپس ماتریس ویژگی‌ها به آرایه‌ی NumPy تبدیل می‌شود.

در ادامه، با پیمایش نمونه‌ها، تابع predict_oneبرای هر نمونه اجرا می‌شود و کلاس پیش‌بینی‌شده در لیست predictionsقرار می‌گیرد. در نهایت، لیست پیش‌بینی‌ها برگردانده می‌شود که شامل یک مقدار پیش‌بینی‌شده برای هر نمونه‌ی ورودی است.

```python
def _compute_tree_stats(self):
    self.n_nodes_ = 0
    self.depth_ = 0

    # Count nodes and calculate tree depth
    stack = [(self.root_, 0)]

    while stack:
        node, depth = stack.pop()

        if node is None:
            continue

        self.n_nodes_ += 1

        if depth > self.depth_:
            self.depth_ = depth

        if not node.is_leaf():
            stack.append((node.left, depth + 1))
            stack.append((node.right, depth + 1))
```

```python
    self.feature_importances_ = np.zeros(self.n_features_)

    def calculate_importance(node):
        if node is None or node.is_leaf():
            return

        left = node.left
        right = node.right

        n = node.samples
        n_left = left.samples
        n_right = right.samples
```

```
        if n > 0:
            decrease = (
                node.impurity
                - (n_left / n) * left.impurity
                - (n_right / n) * right.impurity
            )

            self.feature_importances_[node.feature_index] += (
                (n / self.root_.samples) * decrease
            )

        calculate_importance(left)
        calculate_importance(right)

    calculate_importance(self.root_)

    total_importance = np.sum(self.feature_importances_)

    if total_importance > 0:
        self.feature_importances_ /= total_importance
```

تابع _compute_tree_stats

تابع compute_tree_stats_برای محاسبه‌ی اطلاعات کلی درخت تصمیم و اهمیت ویژگی‌ها استفاده می‌شود. ابتدا تعداد گره‌ها (n_nodes_) و عمق درخت (depth_) صفر قرار می‌گیرند. سپس با استفاده از یک پشته(stack) ، گره‌های درخت پیمایش می‌شوند تا تعداد کل گره‌ها و بیشترین عمق درخت محاسبه شود.

در ادامه، آرایه‌ی feature_importances_برای ذخیره‌ی اهمیت هر ویژگی ایجاد می‌شود. تابع داخلی calculate_importanceبه‌صورت بازگشتی گره‌های درخت را بررسی می‌کند و کاهش ناخالصی حاصل از هر تقسیم را محاسبه می‌کند. سهم هر تقسیم با توجه به تعداد نمونه‌های گره نسبت به تعداد نمونه‌های ریشه وزن‌دهی شده و به اهمیت ویژگی استفاده‌شده در آن تقسیم اضافه می‌شود.

در پایان، اگر مجموع اهمیت ویژگی‌ها بیشتر از صفر باشد، مقادیر اهمیت نرمال‌سازی می‌شوند تا مجموع آن‌ها برابر با یک شود. این مقادیر نشان می‌دهند هر ویژگی چه سهمی در کاهش ناخالصی درخت داشته است.

```
def _build_tree(self, X, y, depth):

    if self._is_stopping_condition(X, y, depth):
        return self._create_leaf(y)

    best_split = self._find_best_split(X, y)

    if best_split is None:
        return self._create_leaf(y)

    feature_index, threshold = best_split

    X_left, y_left, X_right, y_right = self._split(
        X, y, feature_index, threshold
    )

    left = self._build_tree(X_left, y_left, depth + 1)
    right = self._build_tree(X_right, y_right, depth + 1)

    values, counts = np.unique(y, return_counts=True)
    majority_class = values[np.argmax(counts)]
```

```
node = Node(
    feature_index=feature_index,
    threshold=threshold,
    left=left,
    right=right,
    value=majority_class,
    samples=len(y),
    impurity=self._gini(y) if self.criterion == "gini" else self._entropy(y)
)

return node
```

تابع _build_tree

تابع _build_tree هسته‌ی اصلی ساخت درخت تصمیم است و با استفاده از روش بازگشتی، گره‌ها و شاخه‌های درخت را ایجاد می‌کند. ابتدا با تابع _is_stopping_condition بررسی می‌شود که آیا تقسیم گره باید متوقف شود؛ در صورت برقرار بودن شرایط توقف، تابع _create_leaf یک گره برگ ایجاد می‌کند.

اگر امکان ادامه‌ی تقسیم وجود داشته باشد، تابع _find_best_split بهترین ویژگی و آستانه را برای تقسیم داده‌ها مشخص می‌کند. در صورت پیدا نشدن تقسیم معتبر نیز گره به‌صورت برگ ساخته می‌شود. سپس تابع _split داده‌ها را به دو گروه چپ و راست تقسیم می‌کند و با فراخوانی بازگشتی _build_tree، زیردرخت‌های مربوط به هر گروه ساخته می‌شوند.

در پایان، یک گره داخلی از نوع Node ایجاد می‌شود که شامل ویژگی انتخاب‌شده، آستانه‌ی تقسیم، گره‌های فرزند، کلاس اکثریت، تعداد نمونه‌ها و میزان ناخالصی است. این گره به‌عنوان ریشه‌ی زیردرخت ساخته‌شده برگردانده می‌شود. استفاده از پارامتر depth و شرایط توقف، امکان کنترل عمق و پیچیدگی درخت را فراهم می‌کند.

```python
def fit(self, X, y):

    # TODO
    X = np.asarray(X, dtype=float)
    y = np.asarray(y)

    self.n_features_ = X.shape[1]
    self.classes_ = np.unique(y)
    self.n_classes_ = len(self.classes_)
    self.root_ = self._build_tree(X, y, depth=0)
    self._compute_tree_stats()

    return self
```

تابع fit

تابع fitوظیفه‌ی آموزش مدل درخت تصمیم را بر عهده دارد. ابتدا داده‌های ورودی Xو برچسب‌های هدف yبه آرایه‌های NumPy تبدیل می‌شوند. سپس تعداد ویژگی‌ها در n_features_ذخیره شده و کلاس‌های منحصربه‌فرد داده‌های هدف و تعداد آن‌ها به‌ترتیب در classes_و n_classes_قرار می‌گیرند.

در مرحله‌ی بعد، تابع build_tree_به‌صورت بازگشتی درخت تصمیم را از عمق صفر می‌سازد و ریشه‌ی درخت در root_ذخیره می‌شود. سپس compute_tree_stats_برای محاسبه‌ی تعداد گره‌ها، عمق درخت و اهمیت ویژگی‌ها اجرا می‌شود.

در نهایت، خود شیء مدل (self) برگردانده می‌شود تا امکان استفاده‌ی زنجیره‌ای از توابع، مانند tree.fit(X, y).predict(X)، فراهم باشد.

در این بخش ، کلاس های Node و Decision_tree تکمیل شد. حالا به سراغ یونیت تست ها و آزمایش درخت می رویم.

بخش اول

تست شماره ۱ : The Node Class

```
All Node tests passed successfully.
```

برای بررسی درستی پیاده‌سازی کلاس Node، مجموعه‌ای از تست‌ها با استفاده از دستور assertنوشته شده است. این تست‌ها مقداردهی اولیه‌ی ویژگی‌ها، عملکرد متد ()is_leafو ارتباط بین گره‌های درخت را بررسی می‌کنند.

```
All hyperparameter tests passed successfully.
```

تست شماره ۲ : Decision Tree Hyperparameters

این تست‌ها برای بررسی درستی مقداردهی ابرپارامترهای کلاس DecisionTreeطراحی شده‌اند. با استفاده از دستور assert، مقادیر تنظیم‌شده با مقادیر مورد انتظار مقایسه می‌شوند.

تست شماره ۳ : Weighted Impurity

در این بخش، توابع weighted_entropy_و weighted_gini_با چند ورودی مختلف بررسی می‌شوند تا از صحت محاسبات آن‌ها اطمینان حاصل شود. برای این کار، از جمله آرایه‌ی خالی، به توابع داده می‌شوند و نتیجه‌ی محاسبه‌شده با مقدار مورد انتظار مقایسه می‌شود. همچنین بررسی می‌شود که در صورت خالی بودن یکی از آرایه‌ها، خروجی همچنان معتبر باشد و مقدار NaNتولید نشود.

تست شماره ۴ : _entropy, _gini and _information_gain

در این بخش، صحت توابع entropy_، gini_و information_gain_با استفاده از ورودی‌های مختلف بررسی می‌شود. داده‌های آزمایشی شامل حالت‌هایی مانند وجود کلاس‌های غیرمتوالی، تعداد زیاد کلاس‌های متفاوت، گره خالص و توزیع نامساوی نمونه‌ها بین چند کلاس هستند.

همچنین عملکرد تابع information_gain_با معیار Gini بررسی می‌شود تا اطمینان حاصل شود که کاهش ناخالصی حاصل از تقسیم یک گره به دو فرزند به‌درستی محاسبه می‌شود.

تست شماره ۵ : ( ) fit

در این بخش، عملکرد درخت تصمیم در تقسیم داده‌های جدا پذیر و همچنین آموزش مجدد مدل بررسی می‌شود. ابتدا داده‌هایی که دو کلاس آن‌ها به‌خوبی از یکدیگر جدا شده‌اند، به مدل داده می‌شوند تا مشخص شود در ریشه تقسیم مناسبی انجام می‌شود و با تنظیم max_depth=1، گره‌های فرزند به برگ تبدیل می‌شوند.سپس مدل با یک مجموعه داده آموزش داده شده و پس از آن با مجموعه‌ای دیگر که برچسب‌های متفاوتی دارد، دوباره آموزش داده می‌شود. در این مرحله بررسی می‌شود که کلاس‌های مدل به‌روزرسانی شوند و درخت جدید با توجه به داده‌های جدید ساخته شود، نه اینکه اطلاعات یا گره‌های درخت قبلی باقی بمانند.

```
Recursion test passed.
```

تست شماره ۶ : _build_tree() recursion (mock-based)

در این بخش، عملکرد بازگشتی تابع build_tree_و انتقال صحیح عمق درخت بررسی می‌شود. برای این کار، از MagicMockاستفاده شده تا رفتار توابع کمکی کنترل شود و بتوان فراخوانی‌های آن‌ها را بررسی کرد.

در تست، ابتدا گره ریشه در عمق صفر قرار دارد و سپس دو فرزند آن در عمق یک ساخته می‌شوند. همچنین بررسی می‌شود که داده‌های هر شاخه به‌درستی به فرزند مربوطه منتقل شوند، گره ریشه ویژگی و آستانه‌ی تقسیم مورد انتظار را داشته باشد و هر برگ با برچسب‌های همان شاخه ساخته شود.

در نهایت، تعداد فراخوانی تابع split_و ترتیب ایجاد برگ‌ها نیز بررسی می‌شود.

```
test passed.
```

تست شماره ۷: _build_tree() on a real (tiny) dataset

در این تست، عملکرد تابع build_tree_با مجموعه‌داده‌ای بررسی می‌شود که کلاس‌های آن با استفاده از ویژگی اول به‌طور کامل از یکدیگر جدا می‌شوند. هدف این است که مشخص شود درخت، ویژگی و آستانه‌ی مناسبی برای تقسیم انتخاب می‌کند و دو گره فرزند را به‌درستی می‌سازد.

همچنین بررسی می‌شود که گره ریشه برگ نباشد، هر دو فرزند به برگ تبدیل شوند و کلاس پیش‌بینی‌شده‌ی هر برگ با برچسب نمونه‌های مربوط به آن مطابقت داشته باشد. در پایان، آستانه‌ی انتخاب‌شده بررسی می‌شود تا اطمینان حاصل شود که نمونه‌های دارای مقادیر 1.5و 3.0در دو شاخه‌ی متفاوت قرار می‌گیرند.

```
test passed.
```

تست شماره ۸: _find_best_split()

در این بخش، عملکرد تابع find_best_split_بررسی می‌شود. تست‌ها شرایط مختلفی را در نظر می‌گیرند تا مشخص شود تابع می‌تواند محدودیت‌های تقسیم را رعایت کند و بهترین تقسیم ممکن را پیدا کند.

در حالت اول، مقدار min_samples_leafبه‌گونه‌ای تنظیم شده است که هیچ تقسیم معتبری امکان‌پذیر نباشد؛ بنابراین انتظار می‌رود تابع Noneبرگرداند. در حالت دوم، داده‌ها امکان جداسازی کامل دو کلاس را دارند و باید ویژگی اول و آستانه‌ی 2.5انتخاب شوند. در حالت سوم نیز ثابت بودن مقادیر ویژگی بررسی می‌شود تا اطمینان حاصل شود که در صورت نبود امکان تقسیم، تابع نتیجه‌ی نامعتبر تولید نمی‌کند.

تست شماره ۹: _calculate_impurity()

```
test passed.
```

در این بخش، عملکرد تابع calculate_impurity_در انتخاب معیار مناسب برای محاسبه‌ی ناخالصی بررسی می‌شود. ابتدا معیار مدل روی entropyقرار می‌گیرد تا مشخص شود تابع، متد entropy_را فراخوانی می‌کند و به‌اشتباه از gini_استفاده نمی‌کند. برای بررسی این موضوع، از MagicMockاستفاده شده است تا فراخوانی متدها و مقدار خروجی آن‌ها قابل کنترل باشد.

در ادامه، معیار giniبا داده‌هایی شامل چند کلاس مختلف آزمایش می‌شود. در این حالت بررسی می‌شود که تابع متد gini_را دقیقاً یک بار فراخوانی کند و آرایه‌ی لیبل ها را بدون تغییر به آن انتقال دهد.

تست شماره ۱۰: _calculate_information_gain()

```
test passed.
```

در این بخش، درستی عملکرد تابع calculate_information_gain_در محاسبه‌ی کاهش ناخالصی بررسی می‌شود. برای این کار، چند مجموعه‌داده با تعداد نمونه‌ها و کلاس‌های متفاوت در نظر گرفته شده‌اند. با استفاده از MagicMock، مقادیر ناخالصی به‌صورت کنترل‌شده برگردانده می‌شوند تا بتوان نتیجه‌ی نهایی را با مقدار مورد انتظار مقایسه کرد.

همچنین بررسی می‌شود که تابع، ناخالصی گره اصلی و دو گره فرزند را به‌درستی محاسبه کند و برای هر مجموعه‌داده، متد calculate_impurity_را با آرایه‌ی صحیح فراخوانی کند.

تست شماره ۱۱: _predict_one()

```
test passed.
```

در این بخش، عملکرد تابع predict_one_در پیش‌بینی کلاس یک نمونه بررسی می‌شود. ابتدا یک درخت ساده با دو گره برگ ساخته می‌شود که هرکدام نماینده‌ی یکی از کلاس‌های صفر و یک هستند. سپس نمونه‌هایی با مقادیر متفاوت به تابع داده می‌شوند تا مشخص شود حرکت در درخت بر اساس آستانه‌ی تعیین‌شده به‌درستی انجام می‌شود.

همچنین بررسی می‌شود که مقدار برابر با آستانه به شاخه‌ی چپ هدایت شود، پیش‌بینی‌های متوالی بدون تأثیر بر یکدیگر انجام شوند و ریشه‌ی درخت در طول پیش‌بینی تغییر نکند.

تست شماره ۱۲ : Helpers, fit and predict end-to-end

```
Extra tests passed.
```

در این بخش، عملکرد کلی درخت تصمیم با مجموعه‌ای از تست‌های تکمیلی بررسی می‌شود. ابتدا نحوه‌ی تولید آستانه‌های تقسیم، جداسازی داده‌ها و تشخیص شرایط توقف ارزیابی می‌شود. سپس فرایند آموزش و پیش‌بینی

مدل روی داده‌های مختلف آزمایش می‌شود تا مشخص شود مدل می‌تواند کلاس‌ها را به‌درستی تشخیص دهد، عمق تعیین‌شده را رعایت کند و اهمیت ویژگی‌ها را محاسبه کند.

همچنین حالت‌های خاصی مانند مسئله‌ی XOR، برچسب‌های غیرعددی، داده‌های دارای سه کلاس و فراخوانی predict پیش از آموزش مدل بررسی می‌شوند. در حالت آخر هم، انتظار می‌رود خطای RuntimeError ایجاد شود.

در بخش اول تمامی تست ها همانطور که مشاهده شد ، پاس شدند. حال به سراغ بخش دوم یعنی آزمایش عملی رو داده های واقعی و دیتا ست پروژه می رویم.

بخش دوم:

پارت A شامل توابع چاپ نمودار ها و تصاویر تحلیل و آزمایش های عملی است که به بررسی این متد ها نمی پردازیم.

```python
def accuracy_score_manual(y_true, y_pred):

    # TODO
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)
    pred_stat = (y_true == y_pred)
    true_prediction = np.sum(pred_stat)
    true_prediction = true_prediction / len(pred_stat)

    return true_prediction


def count_leaves(tree):

    # TODO
    count = 0
    stack = [tree.root_]
    while(stack):
        node = stack.pop()
        if node.is_leaf():
            count = count + 1
            continue

        stack.append(node.left)
        stack.append(node.right)

    return count
```

محاسبه دقت و تعداد برگ‌های درخت تصمیم

در این بخش، تابع accuracy_score_manual با مقایسه مقادیر واقعی و پیش‌بینی‌شده، تعداد پیش‌بینی‌های صحیح را محاسبه کرده و با تقسیم آن بر تعداد کل نمونه‌ها، دقت مدل را به دست می‌آورد. برای این کار، ورودی‌ها ابتدا به آرایه‌های نامپای تبدیل می‌شوند تا مقایسه به‌درستی انجام شود.

تابع count_leavesنیز تعداد برگ‌های درخت تصـمیم را محاسـبه می‌کند. در این تابع، گره ریشـه در یک پشـته قرار می‌گیرد و با اسـتفاده از پیمایش درخت، گره‌ها یکی‌یکی بررسـی می‌شـوند. اگر گره‌ای برگ باشـد، شـمارنده افزایش می‌یابد؛ در غیر این صـورت، فرزندان آن به پشـته اضـافه می‌شـوند تا بررسـی ادامه پیدا کند. در پایان، تعداد کل برگ‌های درخت برگردانده می‌شود.

```python
def post_prune(tree, X_val, y_val):
    # TODO
    X_val = np.asarray(X_val)
    y_val = np.asarray(y_val)

    def prune(node , X , y):
        if node.is_leaf():
            return

        X_left, y_left, X_right, y_right = tree._split(
        X, y, node.feature_index, node.threshold
        )

        prune(node.left , X_left , y_left)
        prune(node.right , X_right , y_right)

        if node.left.is_leaf() and node.right.is_leaf():

            errors_left = np.sum(y_left != node.left.value)
            errors_right = np.sum(y_right != node.right.value)
            errors_subtree = errors_left + errors_right

            errors_leaf = np.sum(y != node.value)

            if errors_leaf <= errors_subtree:
                node.left = node.right = None
                node.feature_index = node.threshold = None
    prune(tree.root_ , X_val , y_val)
    tree._compute_tree_stats()
    return tree
```

پس هرس درخت تصمیم(Post-Pruning)

در این بخش، تابع post_pruneبرای هرس درخت تصمیم به روش کاهش خطا پیاده‌سازی شده است. در این روش، گره‌های درخت به‌صورت بازگشتی و از پایین به بالا بررسی می‌شوند تا مشخص شود آیا جایگزین کردن یک زیردرخت با یک برگ می‌تواند عملکرد مدل را حفظ کند یا بهتر کند.

برای این کار، ابتدا داده‌های اعتبارسنجی بر اساس شرط تقسیم هر گره به دو بخش تقسیم می‌شوند و هر بخش برای بررسـی فرزندان مربوطه اسـتفاده می‌شـود. پس از بررسـی فرزندان، اگر هر دو فرزند برگ باشـند، تعداد خطاهای پیش‌بینی زیردرخت با تعداد خطاهای حالتی مقایسه می‌شود که در آن گره به‌تنهایی به‌عنوان یک برگ عمل کند. اگر تعداد خطاهای برگ کمتر یا مساوی خطاهای زیردرخت باشد، گره هرس شده و فرزندان آن حذف می‌شوند.

در پایان، آمار درخت دوباره محاسبه می‌شود تا تعداد گره‌ها، عمق درخت و اهمیت ویژگی‌ها با ساختار جدید هماهنگ باشند. این روش باعث ساده‌تر شدن درخت و کاهش پیچیدگی آن می‌شود، بدون اینکه خطای مدل روی داده‌های اعتبارسنجی افزایش پیدا کند.

```
def print_tree(tree, names=None, max_depth=None):

    # TODO
    names = _names(tree , names)
    def rec(node , depth):
        pad = "|   " * depth
        if node.is_leaf() or (max_depth is not None and depth >= max_depth):
            hidden = "" if node.is_leaf() else " [subtree hidden]"
            print(pad + f"--> class = {node.value}"
                        f"(samples = {node.samples} , counts = {node.class_counts})"
                        f"{hidden}")
        else:
            feature = names[node.feature_index]
            print(pad + f"if {feature} <= {node.threshold:.3f}:")
            rec(node.left , depth + 1)
            print(f"{pad}else:   # {feature} > {node.threshold:.3f}")
            rec(node.right, depth + 1)
        return

    rec(tree.root_ , 0)
```

نمایش ساختار درخت تصمیم

در این بخش، تابع print_treeبرای نمایش ساختار درخت تصمیم به‌صورت مجموعه‌ای از شرط‌های if/else پیاده‌سازی شده است.. برای هر گره داخلی، شرط تقسیم بر اساس نام ویژگی و مقدار آستانه نمایش داده می‌شود و فرزندان آن نیز چاپ می‌شوند. در گره‌های برگ، کلاس پیش‌بینی‌شده، تعداد نمونه‌ها و توزیع کلاس‌ها نمایش داده می‌شود.

همچنین، پارامتر max_depthامکان محدود کردن عمق نمایش درخت را فراهم می‌کند. اگر این محدودیت برقرار شود، ادامه‌ی زیردرخت نمایش داده نمی‌شود و عبارت [subtree hidden]مشخص می‌کند که بخش دیگری از درخت وجود دارد.

```python
def trace_path(tree, x, names=None):

    # TODO
    names = _names(tree , names)
    x = np.asarray(x, dtype=float).ravel()
    node = tree.root_
    step = 1
    while not node.is_leaf():
        go_left = x[node.feature_index] <= node.threshold
        feature = names[node.feature_index]
        value = x[node.feature_index]
        direction = "left" if go_left else "right"
        operator = "<=" if go_left else ">"
        print(
        f"Step {step}: {feature} = {value:g} "
        f"{operator} {node.threshold:.3f} -> go {direction} "
        f"(node samples = {node.samples})"
        )
        node = node.left if go_left else node.right
        step += 1

    print(
    f"Leaf reached -> predict {node.value} "
    f"(samples = {node.samples}, counts = {node.class_counts})"
    )

    return node.value
```

نمایش مسیر پیش‌بینی درخت تصمیم

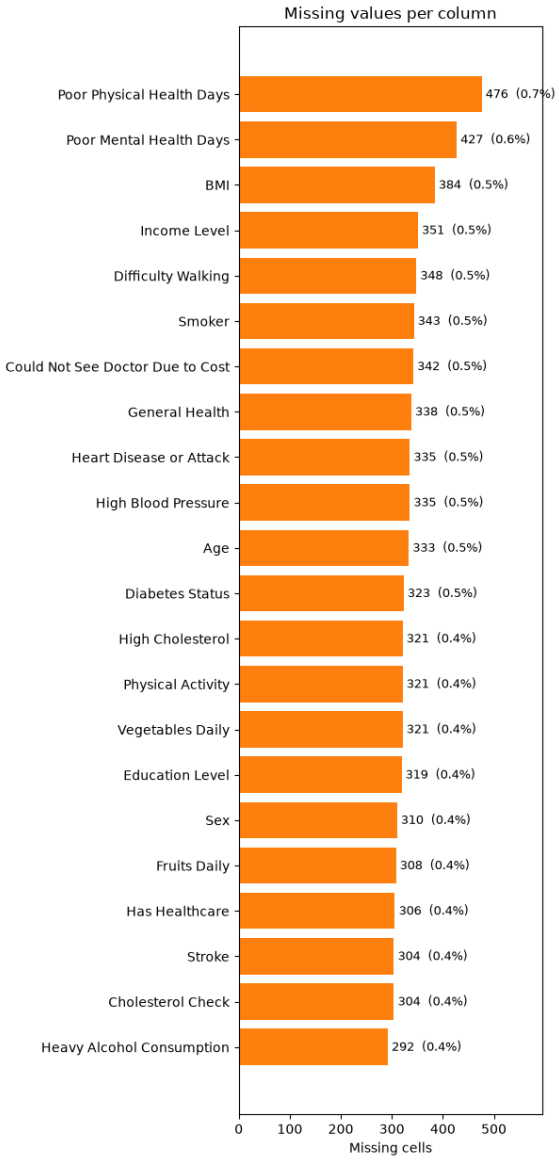
در این بخش، تابع trace_pathبرای نمایش مسیر حرکت یک نمونه در درخت تصمیم پیاده‌سازی شده است. ابتدا نام ویژگی‌ها مشخص شده و نمونه‌ی ورودی به آرایه‌ای عددی تبدیل می‌شود. سپس حرکت از ریشه‌ی درخت آغاز می‌شود و در هر گره داخلی، مقدار ویژگی مربوطه با آستانه‌ی تعیین‌شده مقایسه می‌شود. بر اساس نتیجه‌ی این مقایسه، مسیر حرکت به سمت فرزند چپ یا راست انتخاب می‌شود و اطلاعات هر مرحله، شامل نام ویژگی، مقدار آن، آستانه، جهت حرکت و تعداد نمونه‌های گره، چاپ می‌شود.

این فرایند تا رسیدن به یک برگ ادامه پیدا می‌کند. در پایان، کلاس پیش‌بینی‌شده به همراه تعداد نمونه‌ها و توزیع کلاس‌های موجود در برگ نمایش داده می‌شود و مقدار کلاس به‌عنوان خروجی تابع برگردانده می‌شود.

پارت B : آماده سازی داده و Baseline

قسمت B1

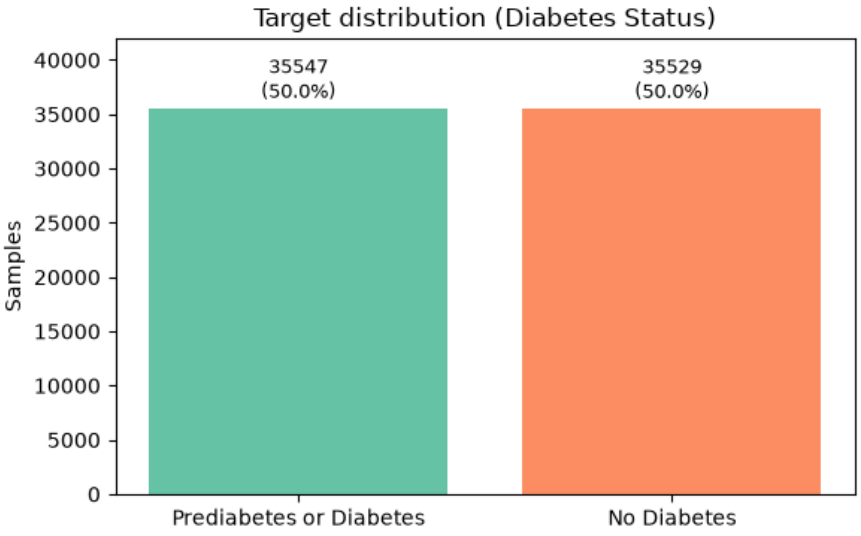
در این بخش دیتا ست پروژه لود شده است و ۵ ردیف اول دیتا ست ، نمودار مقادیر گم شده هر اتریبیوت و نمودار توزیع هر کلاس نمایش داده شده است.



این تصویر missing value های هر ستون را نمایش داده است.

در مراحل بعد این داده های گم شده مدیریت می شود و تبدیل به مد یا میانه هر ستون میشوند.

در این قسمت هم توزیع کلاس های دیتا ست به تعداد و درصد نمایش داده شده است.



قسمت B2 : پاکسازی داده

در این قسمت در چند مرحله دیتا ست لود شده ، پاکسازی و مرتب می شود.

مرحله اول : Duplicate Records و ID و کلاس دیتا ست

```
# TODO

print("Initial shape:", full_data.shape)

duplicate_count = full_data.duplicated().sum()
print("Duplicate records:", duplicate_count)


full_data = full_data.drop_duplicates().copy()


missing = full_data.isnull().sum()
missing = missing[missing > 0].sort_values(ascending=False)

print("\nMissing values:")
print(missing)


full_data = full_data.dropna(
    subset=["Diabetes Status"]
).copy()

full_data = full_data.drop(
    columns=["id"],
    errors="ignore"
)
print("\nShape after cleaning:", full_data.shape)
```

در این بخش، داده‌ها برای استفاده در مراحل بعدی یادگیری ماشین پاک‌سازی می‌شوند. ابتدا ابعاد اولیه‌ی مجموعه‌داده و تعداد رکوردهای تکراری بررسی می‌شود. سپس رکوردهای تکراری حذف می‌شوند تا از تکرار غیرضروری اطلاعات جلوگیری شود. در ادامه، مقادیر گمشده‌ی هر ستون شناسایی و بر اساس تعداد آن‌ها مرتب می‌شوند تا وضعیت داده‌ها مشخص شود.

از آنجا که ستون Diabetes Statusمتغیر هدف مدل است، نمونه‌هایی که مقدار این ستون در آن‌ها مشخص نیست حذف می‌شوند. در نهایت، ستون idنیز از مجموعه‌داده کنار گذاشته می‌شود، زیرا شناسه‌ی هر رکورد معمولاً اطلاعات مفیدی برای پیش‌بینی فراهم نمی‌کند. پس از انجام این مراحل، ابعاد مجموعه‌داده‌ی پاک‌سازی‌شده نمایش داده می‌شود تا تغییرات ایجادشده قابل بررسی باشند.

خروجی در صفحه بعد:

```
Initial shape: (71399, 23)
Duplicate records: 707

Missing values:
Poor Physical Health Days          470
Poor Mental Health Days            420
BMI                                380
Income Level                       350
Difficulty Walking                 345
Smoker                             340
Could Not See Doctor Due to Cost   335
Heart Disease or Attack            330
General Health                     330
High Blood Pressure                330
Age                                330
Physical Activity                  320
High Cholesterol                   320
Diabetes Status                    320
Education Level                    315
Vegetables Daily                   315
Sex                                310
Fruits Daily                       305
Stroke                             300
Cholesterol Check                  300
Has Healthcare                     300
Heavy Alcohol Consumption          290
dtype: int64

Shape after cleaning: (70372, 22)
```

مرحله دوم: مقدار گذاری ویژگی های دسته بندی طور

```python
binary_mappings = {
    "High Blood Pressure": {"Yes": 1, "No": 0},
    "High Cholesterol": {"Yes": 1, "No": 0},
    "Smoker": {"Yes": 1, "No": 0},
    "Heavy Alcohol Consumption": {"Yes": 1, "No": 0},
    "Has Healthcare": {"Yes": 1, "No": 0},
    "Could Not See Doctor Due to Cost": {"Yes": 1, "No": 0},
    "Difficulty Walking": {"Yes": 1, "No": 0},
    "Physical Activity": {"Active": 1, "Inactive": 0},
    "Sex": {"Male": 1, "Female": 0},
}
for col, mapping in binary_mappings.items():
    if col in full_data.columns:
        full_data[col] = full_data[col].map(mapping)

full_data["General Health"] = full_data["General Health"].map({
    "Poor": 0,
    "Fair": 1,
    "Good": 2,
    "Very Good": 3,
    "Excellent": 4
}).astype(float)

print(full_data.isnull().sum().sort_values(ascending=False))
```

در این بخش، ویژگی‌های دسته‌ای مجموعه‌داده به مقادیر عددی تبدیل می شود. برای ویژگی‌های دودویی مانند فشار خون بالا، کلسترول بالا، سیگار کشیدن و جنسیت، از نگاشت مقادیر متنی به صفر و یک استفاده شده است. همچنین، ویژگی Physical Activity بر اساس فعال یا غیرفعال بودن و سایر ویژگی‌های دودویی بر اساس مقادیر مشخص‌شده کدگذاری می‌شوند. این تبدیل فقط برای ستون‌هایی انجام می‌شود که در مجموعه‌داده وجود دارند.

ویژگی General Healthنیز به‌صورت ترتیبی کدگذاری می‌شود؛ به این صورت که وضعیت سلامت از Poor با مقدار صفر تا Excellentبا مقدار چهار نمایش داده می‌شود.

مرحله سوم : داده های پرت

روش‌های شناسایی داده‌های پرت

برای شناسایی داده‌های پرت در این مجموعه‌داده، از دو روش **IQR**و **Z-score**استفاده شده است. هر کدام از این روش‌ها با معیار متفاوتی مقادیر غیرعادی را مشخص می‌کنند.

روش(Interquartile Range) IQR :

در این روش، ابتدا چارک اول ((Q_1)) و چارک سوم ((Q_3)) محاسبه می‌شوند. چارک اول مقداری است که ۲۵ درصد داده‌ها کمتر یا مساوی آن هستند و چارک سوم مقداری است که ۷۵ درصد داده‌ها کمتر یا مساوی آن قرار دارند. سپس فاصله‌ی بین این دو چارک به دست می‌آید:

$$[IQR = Q_3 - Q_1]$$

با استفاده از این مقدار، محدوده‌ی داده‌های عادی تعیین می‌شود:

$$[\text{Lower Bound} = Q_1 - 1.5 \times IQR]$$

$$[\text{Upper Bound} = Q_3 + 1.5 \times IQR]$$

هر مقداری که کمتر از کران پایین یا بیشتر از کران بالا باشد، به‌عنوان داده‌ی پرت شناسایی می‌شود. این روش بر اساس چارک‌ها عمل می‌کند و معمولاً نسبت به وجود مقادیر بسیار بزرگ یا کوچک حساسیت کمتری دارد.

روش Z-score :

در این روش، مشخص می‌شود هر مقدار چقدر از میانگین داده‌ها فاصله دارد. برای این کار، اختلاف مقدار هر نمونه با میانگین محاسبه شده و بر انحراف معیار تقسیم می‌شود.

$$\left[z = \frac{x - \mu}{\sigma}\right]$$

در این رابطه، (x)مقدار نمونه، (μ) میانگین و(σ) انحراف معیار داده‌ها است. اگر قدر مطلق Z-score بیشتر از ۳ باشد، آن مقدار به‌عنوان داده‌ی پرت در نظر گرفته می‌شود؛ یعنی مقدار موردنظر بیش از سه انحراف معیار با میانگین فاصله دارد.

در کد پیاده‌سازی‌شده، تعداد داده‌های پرت با هر دو روش محاسبه و در کنار هم نمایش داده می‌شود. مقایسه‌ی نتایج این دو روش کمک می‌کند دید بهتری نسبت به مقادیر غیرعادی داشته باشیم. البته شناسایی یک مقدار به‌عنوان داده‌ی پرت لزوماً به معنی اشتباه بودن آن نیست و بهتر است قبل از حذف یا تغییر آن، شرایط و ماهیت داده نیز بررسی شود.

```
continuous_cols = [
    "BMI",
    "Poor Mental Health Days",
    "Poor Physical Health Days",
    "Age"
]


day_cols = [
    "Poor Mental Health Days",
    "Poor Physical Health Days"
]

for col in day_cols:
    full_data.loc[
        (full_data[col] < 0) | (full_data[col] > 30),
        col
    ] = float("nan")

print("BMI values below 10:", (full_data["BMI"] < 10).sum())
print("BMI values above 80:", (full_data["BMI"] > 80).sum())
print("Age values below 0:", (full_data["Age"] < 0).sum())
print("Age values above 120:", (full_data["Age"] > 120).sum())

outlier_results = []

for col in continuous_cols:
    data = full_data[col].dropna()

    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    iqr_count = (
        (data < lower_bound) | (data > upper_bound)
    ).sum()

    mean = data.mean()
    std = data.std()

    if std > 0:
        z_count = (((data - mean) / std).abs() > 3).sum()
    else:
        z_count = 0

    outlier_results.append({
        "Feature": col,
        "IQR Lower Bound": round(lower_bound, 2),
        "IQR Upper Bound": round(upper_bound, 2),
        "IQR Outliers": int(iqr_count),
        "Z-score Outliers": int(z_count)
    })

outlier_df = pd.DataFrame(outlier_results)
print("\nOutlier Detection Results:")
print(outlier_df.to_string(index=False))

for col in continuous_cols:
    plt.figure(figsize=(8, 3))

    plt.boxplot(full_data[col].dropna(), vert=False)

    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)
    plt.tight_layout()
    plt.show()
```

در این بخش، ویژگی‌های پیوسته‌ی مجموعه‌داده شامل BMI، تعداد روزهای سلامت نامناسب جسمی و روانی و Ageبررسی می‌شوند. ابتدا برای ستون‌های مربوط به تعداد روزها، مقادیر خارج از بازه‌ی صفر تا ۳۰ به‌عنوان مقادیر نامعتبر شناسایی شده و به‌صورت مقدار گمشده ثبت می‌شوند. همچنین، تعداد مقادیر غیرعادی در ستون‌های BMIو Ageبا استفاده از چند شرط مشخص محاسبه می‌شود تا وضعیت اولیه‌ی این ویژگی‌ها بررسی شود.

در ادامه، برای شناسایی داده‌های پرت از دو روش IQR و Z-score استفاده می‌شود. در روشIQR ، چارک اول و سوم محاسبه شده و بر اساس فاصله‌ی بین آن‌ها، کران پایین و بالای داده‌های عادی تعیین می‌شود. مقادیری که خارج از این محدوده باشند، به‌عنوان داده‌ی پرت در نظر گرفته می‌شوند. در روش Z-score نیز فاصله‌ی هر مقدار از میانگین، بر حسب انحراف معیار، محاسبه می‌شود و مقادیری که قدر مطلق امتیاز آن‌ها بیشتر از سه باشد، به‌عنوان داده‌ی پرت شناسایی می‌شوند.

خروجی ها:

```
BMI values below 10: 0
BMI values above 80: 369
Age values below 0: 0
Age values above 120: 296

Outlier Detection Results:
                Feature  IQR Lower Bound  IQR Upper Bound  IQR Outliers  Z-score Outliers
                    BMI             13.0             45.0          2457               416
  Poor Mental Health Days             -3.0              5.0         11686              4326
Poor Physical Health Days             -9.0             15.0         10510                 0
                    Age             22.5             98.5           978               296
```

تحلیل نتایج شناسایی داده‌های پرت

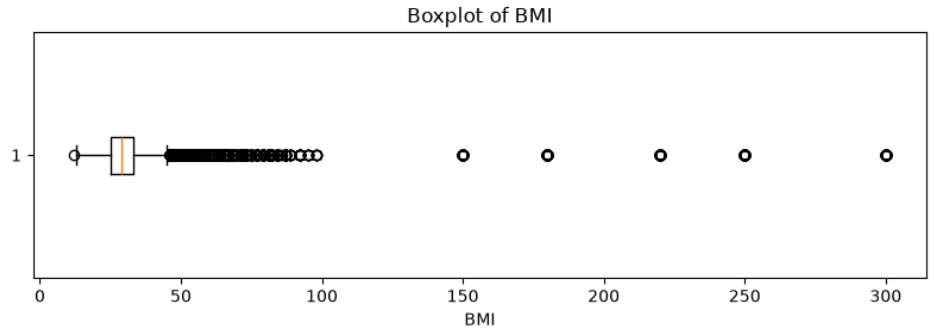
نتایج اولیه نشان می‌دهد که در ستون BMIهیچ مقداری کمتر از ۱۰ وجود ندارد، اما ۳۶۹ مقدار بیشتر از ۸۰ شناسایی شده است. همچنین، در ستون Ageهیچ مقدار منفی وجود ندارد، ولی ۲۹۶ مقدار بیشتر از ۱۲۰ مشاهده می‌شود.

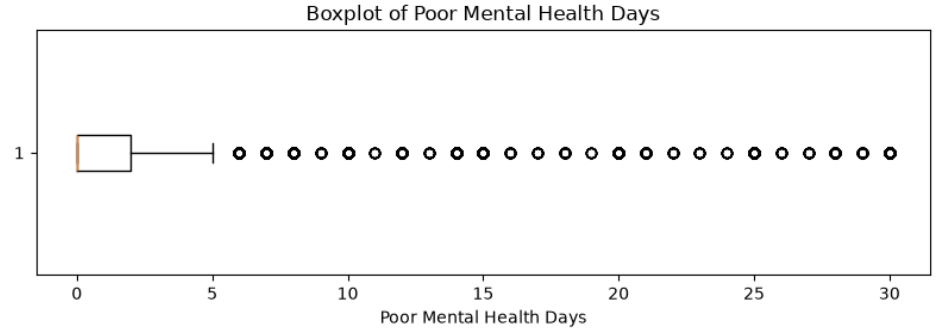
در جدول نتایج، روش IQR برای ستون BMIتعداد ۲۴۵۷ مقدار پرت را شناسایی کرده است؛ در حالی که روش Z-score تعداد ۴۱۶ مقدار را پرت تشخیص داده است. این اختلاف نشان می‌دهد که دو روش معیارهای متفاوتی برای شناسایی داده‌های غیرعادی دارند. در ستون Ageنیز روش IQR تعداد ۹۷۸ مقدار و روش Z-scoreتعداد ۲۹۶ مقدار را به‌عنوان داده‌ی پرت مشخص کرده است.

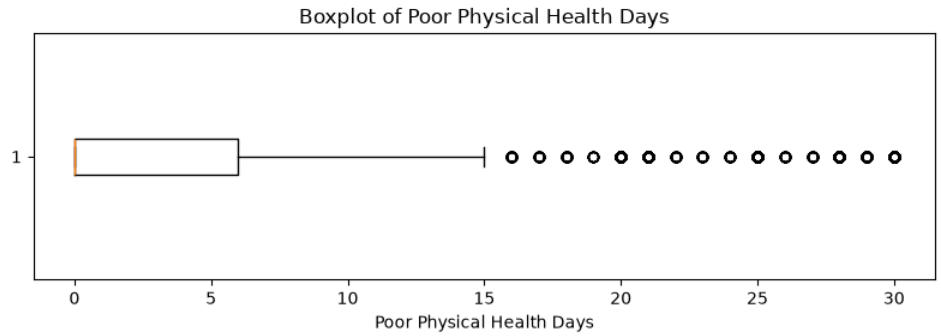
در ستون Poor Mental Health Days، روش IQR تعداد ۱۱۶۸۶ مقدار پرت را شناسایی کرده، در حالی که این تعداد با روش Z-score برابر با ۴۳۲۶ است. برای ستون Poor Physical Health Daysنیز به‌ترتیب ۱۰۵۱۰ و صفر مقدار پرت شناسایی شده است. صفر بودن تعداد داده‌های پرت در روش Z-score به این معنی است که هیچ مقداری در این ستون بیش از سه انحراف معیار از میانگین فاصله نداشته است؛ با این حال، روش IQRمقادیر بیشتری را به دلیل قرار گرفتن خارج از محدوده‌ی چارک‌ها شناسایی کرده است.

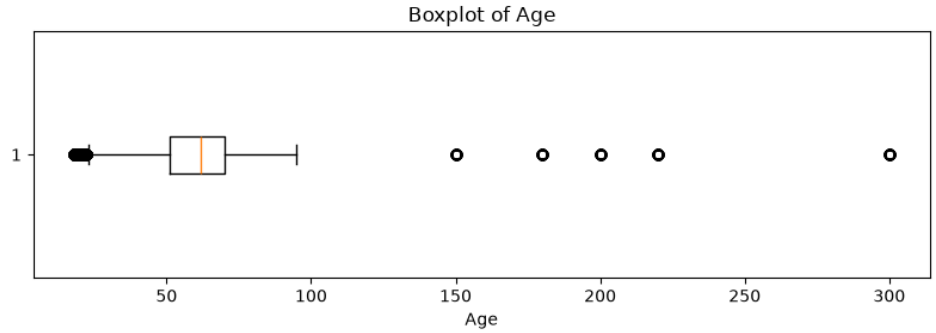
به‌طور کلی، تفاوت نتایج نشان می‌دهد که روش IQR در این مجموعه‌داده مقادیر بیشتری را به‌عنوان داده‌ی پرت شناسایی می‌کند. یکی از دلایل این موضوع می‌تواند توزیع نامتقارن داده‌ها یا تجمع تعداد زیادی از نمونه‌ها در مقادیر خاص باشد.

نمودار ها:









این تصـــاویر نمودار های boxplot ۴ ویژگی بالا را نمایش میدهد. هر نمودار شـــامل چارک های بالا و پایین و میانه است و داده های پرت بعد از چارک های بالا نمایش داده شده است. در نمودار های روز های ضعف فیزیکی و اعصاب چون پایین ترین داده صفر است و میانه هم برابر با صفر به دست آمده است ، چارک پایین و میانه روی هم افتاده است و باعث تفاوت در شکل نمودار در مقایسه با دیگر نمودار ها شده است.

مرحله چهارم: هندل کردن مقادیر پرت و نامعتبر

```python
# 1. BMI
invalid_bmi = full_data["BMI"] > 50

print("Invalid BMI values:", invalid_bmi.sum())
full_data.loc[invalid_bmi, "BMI"] = float("nan")


# 2. Poor Mental Health Days
invalid_mental = (full_data["Poor Mental Health Days"] < 0) | (full_data["Poor Mental Health Days"] > 30)
full_data.loc[invalid_mental, "Poor Mental Health Days"] = float("nan")
print("Invalid Poor Mental Health Days values:", invalid_mental.sum())

#3. Poor Physical Health Days
invalid_physical = (full_data["Poor Physical Health Days"] < 0) | (full_data["Poor Physical Health Days"] > 30)
full_data.loc[invalid_physical, "Poor Physical Health Days"] = float("nan")
print("Invalid Poor Physical Health Days values:", invalid_physical.sum())

# 3. Age
invalid_age = (full_data["Age"] < 18) | (full_data["Age"] > 100)
print("Invalid Age values:", invalid_age.sum())
full_data.loc[invalid_age, "Age"] = float("nan")
```

✓ 0.0s

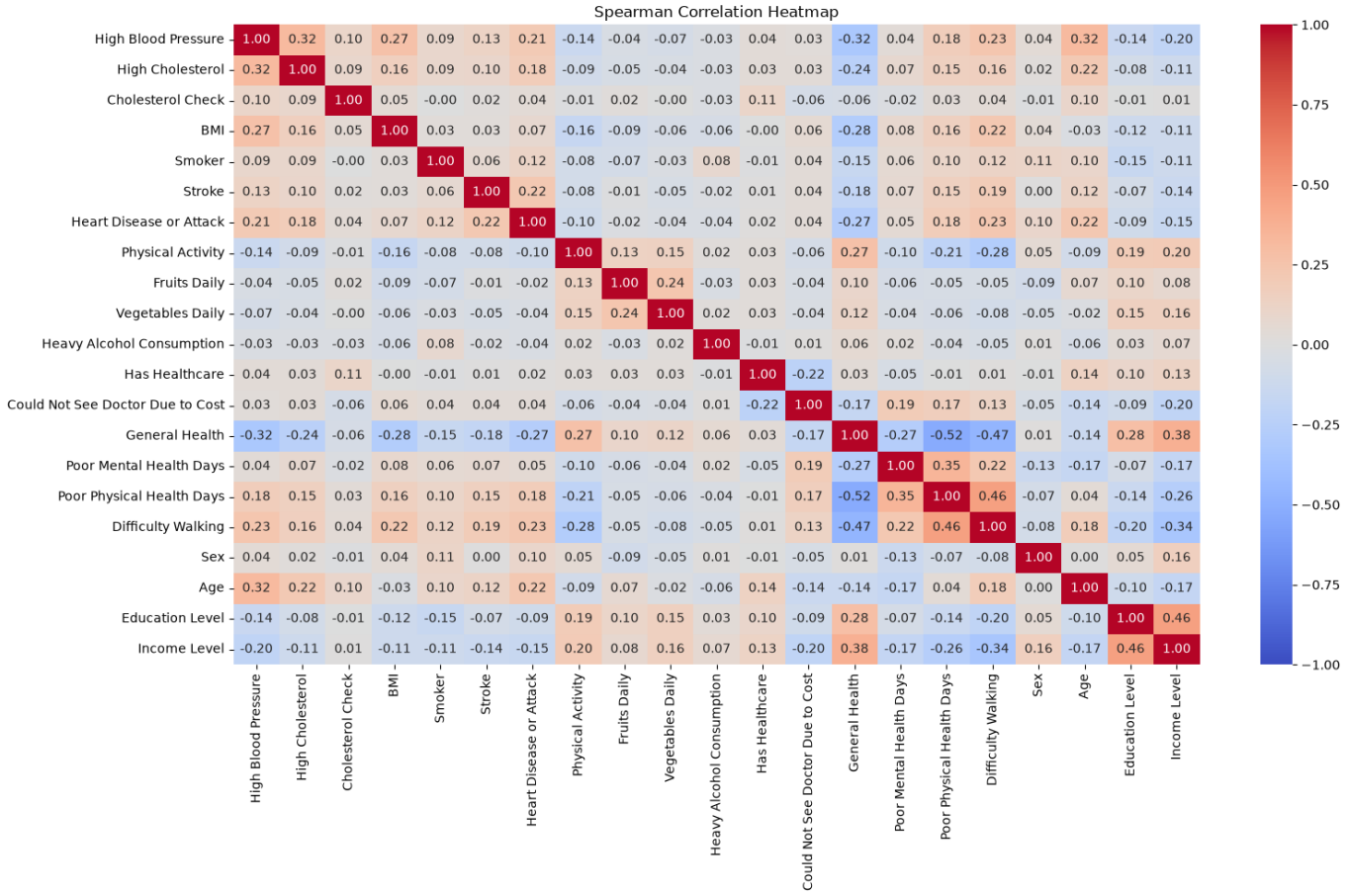
```
Invalid BMI values: 1209
Invalid Poor Mental Health Days values: 0
Invalid Poor Physical Health Days values: 0
Invalid Age values: 296
```

اصلاح مقادیر نامعتبر در ویژگی‌های عددی

در این مرحله، مقادیر نامعتبر در چهار ویژگی عددی مجموعه‌داده بررسی و اصلاح می‌شوند. در ستون BMI، مقادیر بیشتر از ۵۰ نامعتبر در نظر گرفته شده‌اند و به‌صورت مقدار گمشده ثبت می‌شوند. برای ستون‌های Poor Mental Health Days و Poor Physical Health Daysنیز مقادیر خارج از بازه‌ی صفر تا ۳۰ شناسایی و به NaNتبدیل می‌شوند، زیرا تعداد روزها باید در همین محدوده قرار داشته باشد.

همچنین، در ستون Age، مقادیر کمتر از ۱۸ یا بیشتر از ۱۰۰ سال نامعتبر در نظر گرفته شده‌اند و به‌صورت مقدار گمشده ثبت می‌شوند. تعداد مقادیر نامعتبر در هر ویژگی نیز محاسبه و چاپ می‌شود تا میزان اصلاحات انجام‌شده مشخص باشد.

در این روش، مقادیر نامعتبر مستقیماً حذف نمی‌شوند، بلکه به مقادیر گمشده تبدیل می‌شوند تا در مراحل بعدی ، بتوان آن‌ها را با روش‌هایی مانند جایگزینی با میانه مدیریت کرد. این کار باعث می‌شود سایر اطلاعات معتبر همان رکوردها حفظ شوند.



بررسی همبستگی بین ویژگی‌ها(Spearman Correlation Heatmap)

در این مرحله، از نمودار حرارتی همبستگی اسپیرمن برای بررسی ارتباط بین ویژگی‌های مجموعه‌داده استفاده شده است. همبستگی اسپیرمن میزان و جهت رابطه‌ی یکنواخت بین دو متغیر را نشان می‌دهد این روش با استفاده از رتبه‌ی مقادیر داده‌ها، میزان همبستگی را محاسبه می‌کند و برای بررسی روابط بین ویژگی‌های عددی و ترتیبی نیز مناسب است.

در این نمودار، هر سطر و ستون نشان‌دهنده‌ی یک ویژگی است و هر خانه مقدار ضریب همبستگی بین دو ویژگی را نمایش می‌دهد. مقادیر این ضریب بین منفی یک و مثبت یک قرار دارند. مقادیر مثبت نشان‌دهنده‌ی رابطه‌ی مستقیم، مقادیر منفی نشان‌دهنده‌ی رابطه‌ی معکوس و مقادیر نزدیک به صفر نشان‌دهنده‌ی ضعیف بودن رابطه‌ی یکنواخت بین دو ویژگی هستند. همچنین، رنگ قرمز بیانگر همبستگی مثبت و رنگ آبی نشان‌دهنده‌ی همبستگی منفی است. مقادیر روی قطر اصلی نمودار برابر با یک هستند، زیرا هر ویژگی با خودش همبستگی کامل دارد.

با توجه به نتایج، یکی از روابط قابل توجه بین General Healthو Poor Physical Health Daysبا ضریب حدود 0.52-مشاهده می‌شود. این مقدار نشان می‌دهد با بهتر شدن وضعیت سلامت عمومی، تعداد روزهای سلامت جسمی نامناسب معمولاً کاهش می‌یابد. همچنین، بین Education Levelو Income Levelهمبستگی مثبت 0.46وجود دارد که نشان‌دهنده‌ی رابطه‌ی مستقیم بین سطح تحصیلات و سطح درآمد است. رابطه‌ی General Healthبا Poor Mental Health Daysنیز منفی و برابر با 0.27-است.

در مقابل، بعضی از ویژگی‌ها مانند Sexو Ageهمبستگی بسیار ضعیفی با یکدیگر دارند؛ ضریب همبستگی آن‌ها در این نمودار تقریباً صفر است. این موضوع نشان می‌دهد رابطه‌ی یکنواخت قابل توجهی بین این دو ویژگی مشاهده نمی‌شود.

به‌طور کلی، این نمودار به شناخت بهتر روابط بین ویژگی‌ها، شناسایی ارتباط‌های قوی‌تر و بررسی احتمال وجود ویژگی‌های مرتبط یا تکراری کمک می‌کند . با این حال نمیتوان اتریبیوتی را حذف کرد چون شاخص همبستگی بالا تر از 0.7 یا پایین تر از 0.7- برای هیچ یک از ستون ها وجود ندارد تا بتوان ویژگی را حذف کرد.

پ . ن : در رسم نمودار های باکس پلات و ماتریس همبستگی از هوش مصنوعی استفاده شده است و کد های رسم نمودار با استفاده از CHAT GPT نوشته شده است.

```python
y = full_data["Diabetes Status"].copy()
X = full_data.drop(columns=["Diabetes Status"]).copy()
X = X.reset_index(drop=True)
y = y.reset_index(drop=True)
```

جداسازی ویژگی‌ها و متغیر هدف

در نهایت، مجموعه‌داده به دو بخش تقسیم می‌شود. ستون Diabetes Statusبه‌عنوان متغیر هدف در متغیر yذخیره می‌شود و سایر ستون‌ها به‌عنوان ویژگی‌های ورودی مدل در متغیر Xقرار می‌گیرند. به این ترتیب، مدل از ویژگی‌های موجود برای پیش‌بینی وضعیت دیابت استفاده می‌کند.

```python
# Checks for your cleaned data — do not modify
assert isinstance(X, pd.DataFrame) and isinstance(y, pd.Series), "X must be a DataFrame and y a Series"
assert len(X) == len(y) and X.index.equals(y.index), "X and y must have the same rows / index"
assert X.index.is_unique, "The index must be unique — use reset_index(drop=True)"
assert all(pd.api.types.is_numeric_dtype(t) for t in X.dtypes), "X must contain numeric columns only (encode the categorical ones)"
assert TARGET_COLUMN not in X.columns, "The target column must not be a feature"
assert not y.isna().any(), "y must not contain missing values"

FEATURE_NAMES = list(X.columns)
print("X:", X.shape, " y:", y.shape, "| classes:", sorted(y.unique().tolist()))
print("Missing numeric cells still in X (to be filled in B.4 with Train statistics):", int(X.isna().sum().sum()))
```

✓ 0.0s

```
X: (70372, 21)  y: (70372,) | classes: ['No Diabetes', 'Prediabetes or Diabetes']
Missing numeric cells still in X (to be filled in B.4 with Train statistics): 8495
```

بررسی نهایی داده‌های پاک‌سازی‌شده

در این مرحله، با استفاده از چند دستور assert، صحت ساختار داده‌ها پس از پاک‌سازی بررسی می‌شود.

نتایج نشـــان می‌دهد که مجموعه‌داده شـــامل ۷۰۳۷۲ نمونه و ۲۱ ویژگی ورودی اســـت. متغیر هدف نیز دارای ۷۰۳۷۲ مقدار اســت و دو کلاس No Diabetesو Prediabetes or Diabetesرا شـامل می‌شـود. با این حال، در ویژگی‌های ورودی هنوز ۸۴۹۵ مقدار گمشـده وجود دارد. این مقادیر در این مرحله حذف نشـده‌اند، زیرا قرار است در مرحله‌ی بعد با استفاده از آمار محاسبه‌شده از داده‌های آموزشی جایگزین شوند.

قسمت B3 : Train / Validation / Test split

```python
def stratified_train_test_split(X, y, test_size=0.3, random_state=None):
    # TODO
    rng = np.random.RandomState(random_state)
    train_idx = []
    test_idx = []
    for c in np.unique(y):
        idx = np.where(np.asarray(y) == c)[0]
        rng.shuffle(idx)
        n_test = round(test_size * len(idx))
        if len(idx) >=2 :
            n_test = max(1 , min(n_test , len(idx) - 1))

        test_idx.extend(idx[:n_test])
        train_idx.extend(idx[n_test:])

    train_idx = rng.permutation(train_idx)
    test_idx = rng.permutation(test_idx)

    X_train = X.iloc[train_idx]
    X_test = X.iloc[test_idx]
    y_train = y.iloc[train_idx]
    y_test = y.iloc[test_idx]

    return X_train, X_test, y_train, y_test
```

تقسیم داده‌ها به روش Stratified Train-Test Split

در این بخش، تابع stratified_train_test_split برای تقسیم مجموعه‌داده به دو بخش آموزش و آزمون پیاده‌سازی شده است. هدف از این روش، حفظ تقریباً یکسان نسبت کلاس‌های متغیر هدف در هر دو بخش است تا توزیع کلاس‌ها پس از تقسیم تغییر زیادی نکند. این موضوع به‌خصوص در مجموعه‌داده‌هایی که تعداد نمونه‌های کلاس‌ها برابر نیست، اهمیت دارد.

ابتدا یک مولد اعداد تصادفی با استفاده از random_state ساخته می‌شود تا در صورت استفاده از مقدار ثابت، تقسیم داده‌ها قابل تکرار باشد. سپس نمونه‌های هر کلاس به‌صورت جداگانه انتخاب و به‌طور تصادفی مرتب می‌شوند. بر اساس مقدار test_size، تعداد مشخصی از نمونه‌های هر کلاس به مجموعه‌ی آزمون اختصاص داده شده و بقیه در مجموعه‌ی آموزش قرار می‌گیرند. همچنین، اگر یک کلاس حداقل دو نمونه داشته باشد، تلاش می‌شود حداقل یک نمونه از آن در هر دو بخش قرار گیرد.

در ادامه، اندیس‌های جمع‌آوری‌شده‌ی آموزش و آزمون دوباره به‌صورت تصادفی مرتب می‌شوند تا نمونه‌های کلاس‌های مختلف با یکدیگر ترکیب شوند. در نهایت، داده‌ها و برچسب‌های متناظر از Xو yاستخراج شده و به ترتیب X_train، X_test، y_trainو y_testبرگردانده می‌شوند.

```
X_train, X_temp, y_train, y_temp = stratified_train_test_split(X, y, test_size=0.30, random_state=RANDOM_STATE)
X_val, X_test, y_val, y_test     = stratified_train_test_split(X_temp, y_temp, test_size=0.50, random_state=RANDOM_STATE)

print("Train:", X_train.shape, "| Validation:", X_val.shape, "| Test:", X_test.shape)
```

✓ 0.0s

```
Train: (49260, 21) | Validation: (10556, 21) | Test: (10556, 21)
```

```
# Checks for your split — do not modify
n = len(X)
assert len(X_train) + len(X_val) + len(X_test) == n, "No sample may be lost or duplicated"
assert not (set(X_train.index) & set(X_val.index)) and not (set(X_train.index) & set(X_test.index)) \
       and not (set(X_val.index) & set(X_test.index)), "Train / Validation / Test must not overlap"
assert X_train.index.equals(y_train.index) and X_val.index.equals(y_val.index) and X_test.index.equals(y_test.index), \
       "X and y must stay aligned"
for name, part, frac in (("Train", X_train, 0.70), ("Validation", X_val, 0.15), ("Test", X_test, 0.15)):
    assert abs(len(part) / n - frac) < 0.02, f"{name} should contain about {frac:.0%} of the data"
overall = y.value_counts(normalize=True)
for name, part in (("Train", y_train), ("Validation", y_val), ("Test", y_test)):
    gap = (part.value_counts(normalize=True).reindex(overall.index, fill_value=0) - overall).abs().max()
    assert gap < 0.03, f"{name}: class proportions differ from the full data by {gap:.3f} — is the split stratified?"
print("Split checks passed.")
plot_class_proportions({"Full data": y, "Train": y_train, "Validation": y_val, "Test": y_test})
```

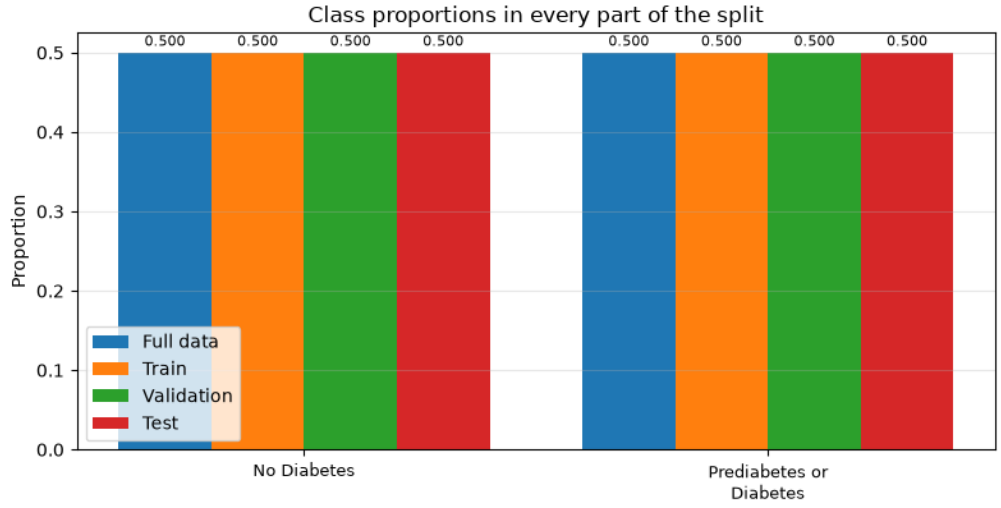
✓ 0.1s

```
Split checks passed.
```

تقسیم نهایی داده‌ها به مجموعه‌های آموزش، اعتبارسنجی و آزمون

در این مرحله، مجموعه‌داده ابتدا به دو بخش آموزش و بخش موقت تقسیم می‌شود؛ ۷۰ درصد داده‌ها برای آموزش و ۳۰ درصد برای ادامه‌ی تقسیم در نظر گرفته می‌شوند. سپس بخش موقت به دو قسمت مساوی تقسیم می‌شود تا مجموعه‌های اعتبارسنجی و آزمون هرکدام حدود ۱۵ درصد از کل داده‌ها را تشکیل دهند.

نتایج نشان می‌دهد که مجموعه‌ی آموزش شامل ۴۹۲۶۰ نمونه، مجموعه‌ی اعتبارسنجی شامل ۱۰۵۵۶ نمونه و مجموعه‌ی آزمون نیز شامل ۱۰۵۵۶ نمونه است.



قسمت B4 : Baseline and handling missing value

```
  # TODO
  majority_class = y_train.mode()[0]
  baseline_val_acc = accuracy_score_manual(y_val, np.full(len(y_val), majority_class))
  print(f"Majority-class baseline (Validation): {baseline_val_acc:.3f}")

  # Filling missing values
  cols = [
      "High Blood Pressure", "High Cholesterol", "Cholesterol Check",
      "Smoker", "Stroke", "Heart Disease or Attack", "Physical Activity",
      "Fruits Daily", "Vegetables Daily", "Heavy Alcohol Consumption",
      "Has Healthcare", "Could Not See Doctor Due to Cost",
      "Difficulty Walking", "Sex", "General Health",
      "Education Level", "Income Level"
  ]
  modes = X_train[cols].mode().iloc[0]
  for X in [X_train, X_val, X_test]:
      X[cols] = X[cols].fillna(modes)

  # # BMI
  bmi_median = X_train["BMI"].median()
  X_train["BMI"] = X_train["BMI"].fillna(bmi_median)
  X_val["BMI"] = X_val["BMI"].fillna(bmi_median)
  X_test["BMI"] = X_test["BMI"].fillna(bmi_median)

  # #Poor Mental Health Days
  median_value = X_train["Poor Mental Health Days"].median()
  X_train["Poor Mental Health Days"] = X_train["Poor Mental Health Days"].fillna(median_value)
  X_val["Poor Mental Health Days"] = X_val["Poor Mental Health Days"].fillna(median_value)
  X_test["Poor Mental Health Days"] = X_test["Poor Mental Health Days"].fillna(median_value)

  # #Poor Physical Health Days
  median_Health = X_train["Poor Physical Health Days"].median()
  X_train["Poor Physical Health Days"] = X_train["Poor Physical Health Days"].fillna(median_Health)
  X_val["Poor Physical Health Days"] = X_val["Poor Physical Health Days"].fillna(median_Health)
  X_test["Poor Physical Health Days"] = X_test["Poor Physical Health Days"].fillna(median_Health)

# #Age
median_Age = X_train["Age"].median()
X_train["Age"] =  X_train["Age"].fillna(median_Age)
X_val["Age"] =  X_val["Age"].fillna(median_Age)
X_test["Age"] =  X_test["Age"].fillna(median_Age)
```

```
Majority-class baseline (Validation): 0.500
```

در این مرحله، ابتدا یک مدل پایه بر اساس کلاس اکثریت ایجاد می‌شود تا معیار اولیه‌ای برای ارزیابی عملکرد مدل اصلی در اختیار باشد. برای این کار، پرتکرارترین کلاس در مجموعه‌ی آموزش با استفاده از mode() مشخص می‌شود و فرض می‌شود که تمام نمونه‌های مجموعه‌ی اعتبارسنجی متعلق به همین کلاس هستند.

سپس دقت این پیش‌بینی‌ها محاسبه می‌شود تا در مراحل بعدی بتوان عملکرد درخت تصمیم را با این معیار مقایسه کرد.

در ادامه، مقادیر گمشده‌ی ویژگی‌ها مدیریت می‌شوند. برای ویژگی‌های دسته‌ای و ویژگی‌هایی که به‌صورت مقادیر گسسته کدگذاری شده‌اند، مقدار پرتکرار هر ستون در مجموعه‌ی آموزش محاسبه شده و برای جایگزینی مقادیر گمشده استفاده می‌شود. در مقابل، برای ویژگی‌های عددی مانند BMI، تعداد روزهای سلامت نامناسب جسمی و روانی و Age، میانه‌ی مقادیر مجموعه‌ی آموزش به‌عنوان مقدار جایگزین در نظر گرفته می‌شود. استفاده از میانه برای ویژگی‌های عددی می‌تواند تأثیر مقادیر بسیار بزرگ یا کوچک را نسبت به میانگین کاهش دهد.

نکته‌ی مهم این است که آمار موردنیاز برای جایگزینی مقادیر گمشده فقط از مجموعه‌ی آموزش محاسبه می‌شود و همان مقادیر برای مجموعه‌های آموزش، اعتبارسنجی و آزمون به کار می‌روند. این کار از نشت اطلاعات جلوگیری می‌کند؛ زیرا اطلاعات مجموعه‌ی اعتبارسنجی و آزمون در تعیین مقادیر جایگزین دخالتی ندارند. در نتیجه، داده‌ها برای آموزش و ارزیابی مدل آماده‌تر می‌شوند.

پ . ن : در این مرحله از روش گسسته سازی مقادیر پیوسته ستون هایی مثل سن و bmi و سلامت روان و جسمانی استفاده کردم ولی تقاوت چشمگیری در دقت داده های ولیدیشن و ترین مشاهده نشد ولی با این روش فاصله و گپ بین ترین و ولیدیشن اندکی بهتر شد .

پاسخ سؤالات بخشB

۱ .چه پیش‌پردازش‌هایی روی داده‌ها انجام شد و چرا؟

ابتدا رکوردهای تکراری حذف شدند، نمونه‌هایی که مقدار متغیر هدف آن‌ها مشخص نبود کنار گذاشته شدند و ستون idبه دلیل نداشتن کاربرد مستقیم در پیش‌بینی حذف شد. سپس ویژگی‌های دسته‌ای به مقادیر عددی تبدیل شدند تا امکان استفاده از آن‌ها در مدل فراهم شود. در ادامه، مقادیر نامعتبر در ویژگی‌های عددی شناسایی و به‌صورت مقدار گمشده ثبت شدند و داده‌های پرت با روش‌های IQR و Z-score بررسی شدند. در نهایت، داده‌ها به مجموعه‌های آموزش، اعتبارسنجی و آزمون تقسیم شدند و مقادیر گمشده با استفاده از مد برای ویژگی‌های دسته‌ای و میانه برای ویژگی‌های عددی جایگزین شدند. هدف از این مراحل، بهبود کیفیت داده‌ها و آماده‌سازی آن‌ها برای آموزش و ارزیابی مدل است.

۲ .چرا از همان ابتدا یک مجموعه‌ی آزمون جداگانه نگه می‌داریم؟

مجموعه‌ی آزمون برای ارزیابی نهایی مدل روی داده‌هایی اســتفاده می‌شــود که در فرایند آموزش و تنظیم مدل دخالتی نداشته‌اند. اگر از این مجموعه برای انتخاب تنظیمات مدل یا تصمیم‌گیری درباره‌ی پیش‌پردازش استفاده شود، ممکن است اطلاعات آن به‌طور غیرمستقیم وارد فرایند ساخت مدل شود و باعث نشت اطلاعات شود. در نتیجه، عملکرد مدل بهتر از مقدار واقعی تخمین زده خواهد شــد. جدا نگه داشــتن مجموعه‌ی آزمون کمک می‌کند ارزیابی نهایی، دید قابل‌اعتمادتری از عملکرد مدل روی داده‌های جدید ارائه دهد.

۳ .اگر دقت درخت تصمیم نزدیک به دقت خط پایه باشد، آیا درخت مفید است؟

اگر دقت درخت تصمیم به دقت خط پایه نزدیک باشد، احتمال دارد درخت نسبت به پیش‌بینی همیشگی کلاس اکثریت بهبود قابل‌توجهی ایجاد نکرده باشــد. با این حال، تنها معیار دقت برای تصــمیم‌گیری کافی نیســت، به‌خصوص زمانی که تعداد نمونه‌های کلاس‌ها نامتوازن باشد. در چنین شرایطی، معیارهایی مانندPrecision ، Recallو F1-score به همراه ماتریس درهم‌ریختگی باید بررسی شوند. برای مثال، ممکن است درخت با وجود دقت مشابه خط پایه، در شناسایی افراد مبتلا به دیابت عملکرد بهتری داشته باشد.

۴ .چرا به بخش B.4 نیاز داریم؟ نمی‌توانیم قبل از تقسیم داده‌ها مقادیر گمشده را جایگزین کنیم؟

خیر، بهتر است مقادیر گمشده پس از تقسیم داده‌ها جایگزین شوند. در این روش، آماری مانند مد و میانه فقط از مجموعه‌ی آموزش محاسبه می‌شود و سپس همان مقادیر برای جایگزینی داده‌های گمشده در مجموعه‌های آموزش، اعتبارسنجی و آزمون استفاده می‌شوند. اگر قبل از تقسیم داده‌ها این آمار از کل مجموعه‌داده محاسبه شود، اطلاعات مجموعه‌های اعتبارسنجی و آزمون نیز در فرایند پیش‌پردازش دخالت خواهد داشت. این موضوع باعث نشت اطلاعات می‌شود و می‌تواند ارزیابی مدل را غیرواقعی کند. بنابراین، انجام این مرحله پس از تقسیم داده‌ها به حفظ اعتبار نتایج کمک می‌کند.

قسمت C : Experiment 1: Let the Tree Grow

```
# TODO — Experiment 1: an UNRESTRICTED tree

unrestricted = run_experiment(make_config())


print(f"Train accuracy:      {unrestricted['train_acc']:.3f}")
print(f"Validation accuracy: {unrestricted['val_acc']:.3f}")
print(f"Majority baseline:   {baseline_val_acc:.3f}")

print(f"Tree depth:          {unrestricted['depth']}")
print(f"Number of nodes:     {unrestricted['nodes']}")
print(f"Number of leaves:    {unrestricted['leaves']}")
```

✓ 1m 15.6s

```
Train accuracy:      0.999
Validation accuracy: 0.656
Majority baseline:   0.500
Tree depth:          38
Number of nodes:     25503
Number of leaves:    12752

Train - Validation gap: 0.343
```

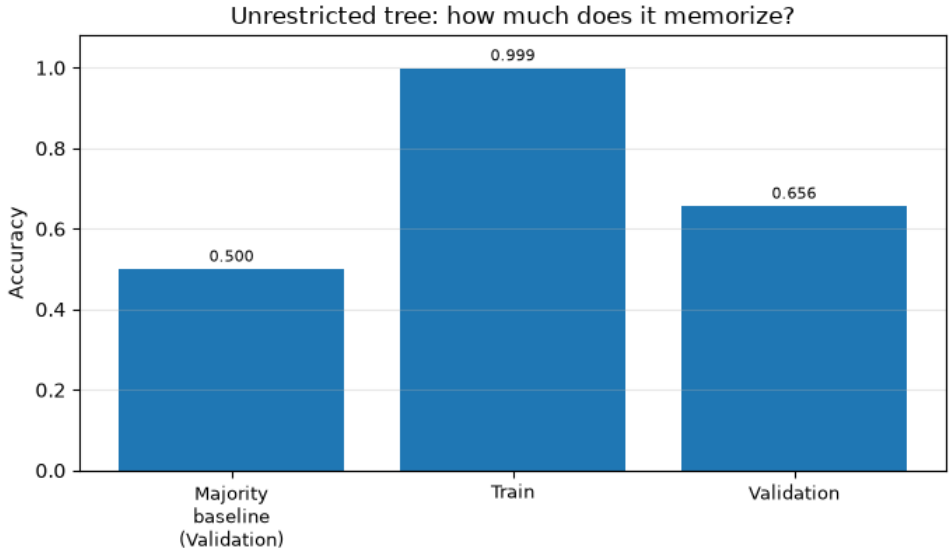
آزمایش اول: درخت تصمیم بدون محدودیت(Unrestricted Tree)

در این آزمایش، یک درخت تصمیم بدون محدودیت مشخص برای عمق یا حداقل تعداد نمونه‌های لازم برای تقسیم گره آموزش داده می‌شود. هدف از این آزمایش، بررسی عملکرد درختی است که آزادی زیادی برای یادگیری الگوهای موجود در داده‌های آموزشی دارد.

نتایج نشان می‌دهد که دقت مدل روی داده‌های آموزش برابر با 0.999و دقت آن روی داده‌های اعتبارسنجی برابر با 0.656است. در حالی که دقت خط پایه، یعنی پیش‌بینی همیشگی کلاس اکثریت، برابر با 0.500است. بنابراین، درخت تصمیم نسبت به مدل پایه عملکرد بهتری دارد؛ اما اختلاف زیاد بین دقت آموزش و اعتبارسنجی نشان می‌دهد که مدل دچار **بیش‌برازش (Overfitting)** شده است. به این معنا که مدل داده‌های آموزشی را تقریباً به‌طور کامل یاد گرفته، اما در تعمیم الگوها به داده‌های جدید عملکرد ضعیف‌تری دارد.

از نظر پیچیدگی نیز، عمق درخت برابر با 38، تعداد کل گره‌ها برابر با 25503و تعداد برگ‌ها برابر با 12752 است. این اعداد نشان می‌دهند که درخت ساختار بسیار بزرگی پیدا کرده است. در مجموع، این آزمایش نشان

می‌دهد که آزاد گذاشتن کامل درخت می‌تواند دقت آموزش را بسیار بالا ببرد، اما لزوماً به عملکرد مناسب روی داده‌های دیده‌نشده منجر نمی‌شود.



پاسخ سؤالات بخشC

۱ .آیا دقت آموزش و اعتبارسنجی به هم نزدیک هستند؟ اختلاف آن‌ها چه چیزی را نشان می‌دهد؟

خیر، دقت آموزش برابر با 0.999و دقت اعتبارسنجی برابر با 0.656است. این اختلاف قابل‌توجه نشان می‌دهد که مدل روی داده‌های آموزشی عملکرد بسیار خوبی دارد، اما نمی‌تواند همان عملکرد را روی داده‌های جدید حفظ کند. این مسئله نشانه‌ی بیش‌برازش (Overfitting) است؛ یعنی مدل به‌جای یادگیری الگوهای عمومی، تا حد زیادی جزئیات و ویژگی‌های خاص داده‌های آموزشی را حفظ کرده است.

۲ .درخت چقدر بزرگ شده است؟ آیا این اندازه برای تعداد داده‌ها منطقی است؟

درخت دارای 25503گره، 12752برگ و عمق 38است. با توجه به اینکه مجموعه‌ی آموزش شامل 49260 نمونه است، این ساختار نسبتاً بزرگ و پیچیده به نظر می‌رسد. چنین درختی می‌تواند تعداد زیادی از جزئیات داده‌های آموزشی را یاد بگیرد و حتی به نمونه‌های خاص واکنش نشان دهد. بنابراین، این پیچیدگی در کنار اختلاف زیاد دقت آموزش و اعتبارسنجی نشان می‌دهد که محدود کردن اندازه‌ی درخت می‌تواند به بهبود تعمیم‌پذیری مدل کمک کند.

۳ .چرا یک درخت تصمیم می‌تواند به‌راحتی به دقت تقریباً ۱۰۰ درصد در آموزش برسد؟

درخت تصمیم با تقسیم مکرر داده‌ها بر اساس ویژگی‌ها و آستانه‌های مختلف، نمونه‌ها را به گروه‌های کوچک‌تر تقسیم می‌کند. اگر محدودیتی برای عمق درخت یا حداقل تعداد نمونه‌های لازم برای تقسیم وجود نداشته باشد، درخت می‌تواند تقسیم‌های زیادی انجام دهد و حتی نمونه‌های آموزشی را تا حد زیادی از یکدیگر جدا کند. در نتیجه، ممکن است تقریباً تمام نمونه‌های آموزشی را درست پیش‌بینی کند. با این حال، چنین مدلی لزوماً روی داده‌های جدید عملکرد خوبی ندارد؛ زیرا ممکن است به جزئیات و نویز داده‌های آموزش بیش از حد وابسته شده باشد. به همین دلیل، استفاده از محدودیت‌های رشد درخت و هرس کردن آن برای کنترل بیش‌برازش اهمیت دارد.

قسمت D : Experiment 2: Underfitting vs Overfitting

```
depth_values = [1, 2, 3, 4, 5, 6, 8, 10, float("inf")]      # float("inf") = no depth limit

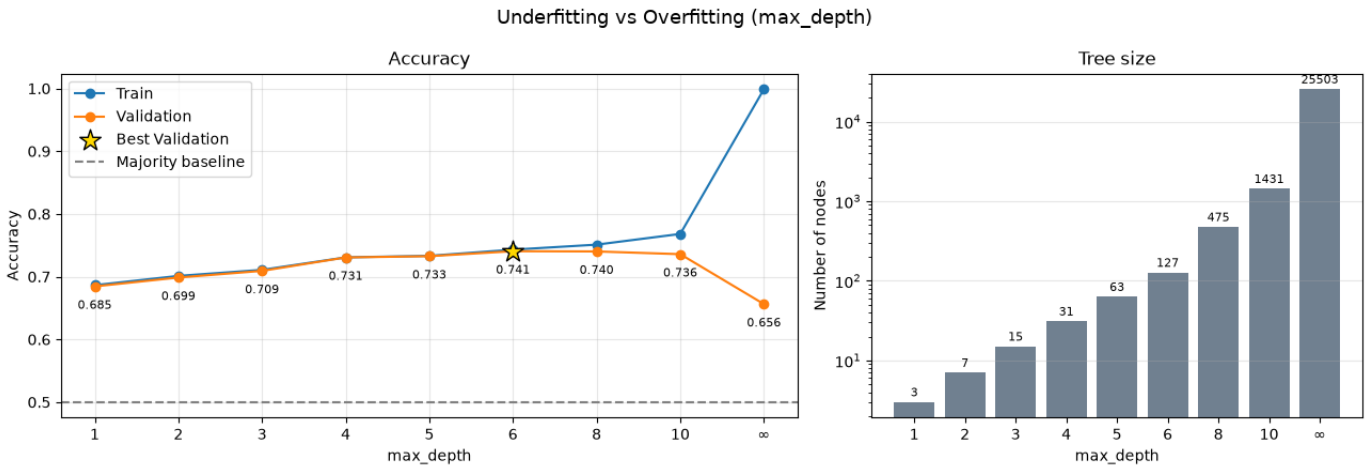
# TODO — one experiment per depth value
depth_results = []
for depth in depth_values:
    result = run_experiment(make_config(max_depth=depth))
    depth_results.append(result)

depth_df = to_df(depth_results)
```

بررسی تأثیر عمق درخت تصمیم بر عملکرد مدل

در این مرحله، آزمایش‌هایی با عمق‌های مختلف برای درخت تصمیم انجام می‌شود تا تأثیر محدود کردن عمق بر عملکرد و پیچیدگی مدل بررسی شود. مقادیر عمق شامل 1، 2، 3، 4، 5، 6، 8، 10و حالت بدون محدودیت است که با float("inf")مشخص می‌شود.در هر مرحله، تابع run_experimentبا مقدار عمق موردنظر اجرا شده و نتایج در فهرست depth_resultsذخیره می‌شوند.

هدف این آزمایش، بررسی رابطه‌ی بین عمق درخت و دقت آن روی داده‌های آموزش و اعتبارسنجی است. انتظار می‌رود با افزایش عمق، مدل بتواند الگوهای پیچیده‌تری را یاد بگیرد و دقت آموزش افزایش پیدا کند؛ اما اگر درخت بیش از حد رشد کند، ممکن است دقت اعتبارسنجی کاهش یابد یا بهبود قابل‌توجهی نداشته باشد.



نمودار نشـــان می‌دهد که با افزایش پارامتر max_depth، دقت مدل روی داده‌های آموزش و اعتبارســـنجی در ابتدا افزایش پیدا می‌کند. در عمق‌های کم، درخت ســـاختار ســـاده‌ای دارد و نمی‌تواند تمام الگوهای موجود در داده‌ها را یاد بگیرد؛ این وضـعیت می‌تواند نشـانه‌ی کم‌برازش (Underfitting) باشـد. برای مثال، در عمق ۱، دقت اعتبارسنجی برابر با 0.685است.

با افزایش عمق، عملکرد مدل بهتر می‌شـود و بیشـترین دقت اعتبارسـنجی در عمق ۶ با مقدار 0.741به دسـت می‌آید. پس از این نقطه، دقت اعتبارسنجی بهبود قابل‌توجهی ندارد و با افزایش عمق به ۱۰، به 0.736می‌رسد. در حالت بدون محدودیت نیز دقت آموزش تقریباً به 1.000می‌رسـد، اما دقت اعتبارسـنجی به 0.656کاهش پیدا می‌کند. این اختلاف نشان می‌دهد که درخت بیش از حد پیچیده شده و دچار بیش‌برازش شده است.

نمودار سـمت راسـت نیز افزایش شـدید پیچیدگی درخت را نشـان می‌دهد. تعداد گره‌ها از ۳ گره در عمق ۱ به ۱۲۷ گره در عمق ۶ می‌رسـد و در حالت بدون محدودیت به ۲۵۵۰۳ گره افزایش پیدا می‌کند. بنابراین، افزایش عمق همیشـه به معنای بهتر شـدن عملکرد مدل نیسـت و می‌تواند باعث افزایش هزینه‌ی محاسـباتی و کاهش توانایی تعمیم مدل شود.

در مجموع، **عمق ۶ بهترین نتیجه را در این آزمایش دارد**؛ زیرا بیشـترین دقت اعتبارسـنجی را با پیچیدگی بسـیار کمتر از درخت بدون محدودیت ارائه می‌دهد.

پاسخ سؤالات بخشD

۱ .کدام عمق‌ها نشانه‌ی کم‌برازش و کدام‌یک نشانه‌ی بیش‌برازش هستند؟ از کجا می‌توان تشخیص داد؟

عمق‌های پایین، به‌خصوص عمق ۱ و ۲، نشانه‌هایی از کم‌برازش دارند؛ زیرا مدل هنوز ساختار پیچیده‌ای ندارد و دقت اعتبارسنجی آن نسبتاً پایین است. با افزایش عمق، دقت آموزش و اعتبارسنجی بهتر می‌شود. در عمق‌های

۸ و ۱۰، فاصله‌ی بین دقت آموزش و اعتبارسنجی بیشتر می‌شود و درخت بدون محدودیت نیز با دقت آموزش نزدیک به ۱ و دقت اعتبارسنجی برابر با 0.656دچار بیش‌برازش واضحی است. بنابراین، افزایش فاصله‌ی دو منحنی و کاهش دقت اعتبارسنجی با وجود افزایش دقت آموزش، نشانه‌ی بیش‌برازش است.

۲ .آیا عمیق‌ترین درخت بهترین عملکرد را روی داده‌های اعتبارسنجی دارد؟

خیر. عمیق‌ترین درخت، یعنی درخت بدون محدودیت، دقت آموزش تقریباً برابر با 1.000دارد، اما دقت اعتبارسنجی آن فقط 0.656است. بیشترین دقت اعتبارسنجی در عمق ۶ با مقدار 0.741به دست آمده است. بنابراین، درخت عمیق‌تر لزوماً عملکرد بهتری روی داده‌های جدید ندارد.

۳ .آیا می‌توان فقط بر اساس دقت آموزش بهترین مدل را انتخاب کرد؟ چرا؟

خیر. دقت آموزش به‌تنهایی معیار مناسبی برای انتخاب مدل نیست؛ زیرا با افزایش پیچیدگی درخت، مدل ممکن است داده‌های آموزشی را تقریباً به‌طور کامل یاد بگیرد، اما در پیش‌بینی داده‌های جدید عملکرد ضعیف‌تری داشته باشد. این مسئله همان بیش‌برازش است. بنابراین، برای انتخاب مدل مناسب باید دقت اعتبارسنجی و میزان پیچیدگی درخت نیز بررسی شوند تا مدلی انتخاب شود که علاوه بر یادگیری مناسب داده‌ها، قدرت تعمیم خوبی داشته باشد.

قسمت E : Experiment 3: Pre-Pruning

```
min_split_values = [2, 5, 10, 20, 50, 100]

# TODO — sweep min_samples_split (all other hyperparameters stay at their defaults)
minsplit_results = []
for m in min_split_values:
    result = run_experiment(make_config(min_samples_split=m))
    minsplit_results.append(result)

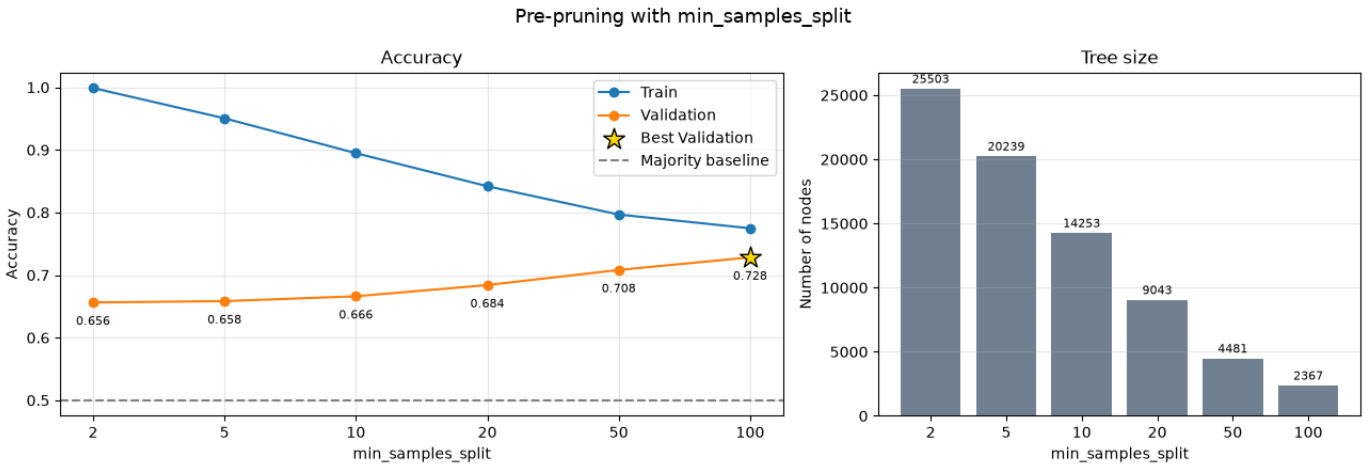
minsplit_df = to_df(minsplit_results)
```

بررسی تأثیر پارامترmin_samples_split

در این مرحله، تأثیر پارامتر min_samples_splitبر عملکرد و پیچیدگی درخت تصمیم بررسی می‌شود. این پارامتر حداقل تعداد نمونه‌هایی را مشخص می‌کند که یک گره باید داشته باشد تا امکان تقسیم آن به گره‌های فرزند وجود داشته باشد. در این آزمایش، مقادیر 2، 5، 10، 20، 50و 100بررسی می‌شوند و سایر پارامترهای مدل در حالت پیش‌فرض باقی می‌مانند.

برای هر مقدار، تابع run_experimentاجرا شده و نتایج در فهرست minsplit_resultsذخیره می‌شوند.

انتظار می‌رود با افزایش min_samples_split، تقسیم گره‌ها محدودتر شود و در نتیجه، اندازه و پیچیدگی درخت کاهش پیدا کند. این محدودیت می‌تواند از بیش‌برازش جلوگیری کند؛



نمودار نشان می‌دهد که با افزایش مقدار min_samples_split، دقت مدل روی داده‌های آموزش کاهش پیدا می‌کند، در حالی که دقت اعتبارسنجی به‌تدریج افزایش می‌یابد. برای مثال، با مقدار ۲، دقت آموزش تقریباً 1.000و دقت اعتبارسنجی 0.656است؛ اما با افزایش این پارامتر به ۱۰۰، دقت آموزش به 0.773و دقت اعتبارسنجی به 0.728می‌رسد. این روند نشان می‌دهد که محدود کردن تقسیم گره‌ها می‌تواند از بیش‌برازش جلوگیری کند و توانایی مدل در تعمیم به داده‌های جدید را بهبود دهد.

نمودار سمت راست نیز نشان می‌دهد که افزایش این پارامتر باعث کاهش قابل‌توجه اندازه‌ی درخت می‌شود. تعداد گره‌ها از 25503در مقدار ۲ به 2367در مقدار ۱۰۰ کاهش پیدا می‌کند. در نتیجه، درخت ساده‌تر شده و پیچیدگی محاسباتی آن کمتر می‌شود.

بیشترین دقت اعتبارسنجی در بین مقادیر آزمایش‌شده، برابر با 0.728و مربوط به min_samples_split = 100است. در این مقدار، اختلاف دقت آموزش و اعتبارسنجی نیز نسبت به مقادیر کوچک‌تر کاهش یافته است. بنابراین، در محدوده‌ی آزمایش‌شده، مقدار ۱۰۰ بهترین نتیجه را از نظر دقت اعتبارسنجی ارائه می‌دهد. البته برای انتخاب نهایی مدل باید سایر پارامترها و معیارهای ارزیابی نیز در نظر گرفته شوند.

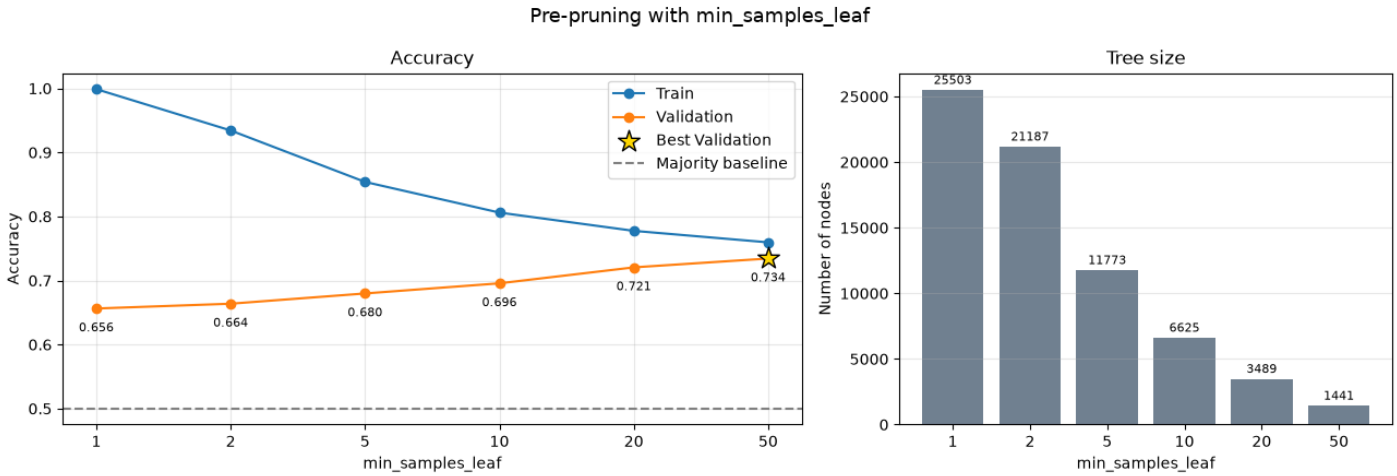
```python
min_leaf_values = [1, 2, 5, 10, 20, 50]

# TODO — sweep min_samples_leaf (all other hyperparameters stay at their defaults)
minleaf_results = []
for m in min_leaf_values:
    result = run_experiment(make_config(min_samples_leaf=m))
    minleaf_results.append(result)

minleaf_df = to_df(minleaf_results)
```

بررسی تأثیر پارامتر min_samples_leaf

در این مرحله، تأثیر پارامتر min_samples_leaf بر عملکرد و پیچیدگی درخت تصمیم بررسی می‌شود. این پارامتر حداقل تعداد نمونه‌هایی را مشخص می‌کند که باید در هر برگ درخت وجود داشته باشند. در این آزمایش، مقادیر 1، 2، 5، 10، 20 و 50 بررسی می‌شوند و سایر پارامترهای مدل در حالت پیش‌فرض باقی می‌مانند.برای هر مقدار، تابع run_experiment اجرا شده و نتایج در فهرست minleaf_results ذخیره می‌شوند. با افزایش min_samples_leaf، مدل مجبور می‌شود برگ‌هایی با تعداد نمونه‌ی بیشتری ایجاد کند؛ در نتیجه، تقسیم‌های بسیار جزئی محدود می‌شوند و معمولاً اندازه‌ی درخت کاهش پیدا می‌کند. این کار می‌تواند از بیش‌برازش جلوگیری کند



نمودار نشان می‌دهد که با افزایش مقدار min_samples_leaf، دقت مدل روی داده‌های آموزش کاهش می‌یابد، اما دقت اعتبارسنجی به‌تدریج بهتر می‌شود. برای مثال، در مقدار ۱، دقت آموزش تقریباً 1.000 و دقت اعتبارسنجی 0.656 است؛ در حالی که با افزایش این پارامتر به ۵۰، دقت آموزش به 0.760 و دقت اعتبارسنجی به 0.734 می‌رسد. این روند نشان می‌دهد که محدود کردن حداقل تعداد نمونه‌های هر برگ می‌تواند از تقسیم‌های بیش از حد جزئی جلوگیری کند و توانایی مدل در تعمیم به داده‌های جدید را بهبود دهد.

نمودار سمت راست نیز نشان می‌دهد که با افزایش این پارامتر، اندازه‌ی درخت به‌طور قابل‌توجهی کاهش پیدا می‌کند. تعداد گره‌ها از 25503در مقدار ۱ به 1441در مقدار ۵۰ می‌رسد. بنابراین، درخت ساده‌تر می‌شود و پیچیدگی آن کاهش می‌یابد.

بیشترین دقت اعتبارسنجی در بین مقادیر بررسی‌شده، برابر با 0.734و مربوط به min_samples_leaf = 50است. همچنین فاصله‌ی دقت آموزش و اعتبارسنجی نسبت به مقادیر کوچک‌تر کمتر شده است. در نتیجه، در محدوده‌ی این آزمایش، مقدار ۵۰ بهترین عملکرد اعتبارسنجی را ارائه می‌دهد و در عین حال اندازه‌ی درخت را نیز کاهش می‌دهد. البته انتخاب نهایی باید با در نظر گرفتن سایر تنظیمات و معیارهای ارزیابی انجام شود.

```python
# TODO — pick ONE representative pre-pruned tree for the comparison in Part G
all_prepruning_results = minsplit_results + minleaf_results
best_val_acc = max(r["val_acc"] for r in all_prepruning_results)

ok_trees = [
    r for r in all_prepruning_results
    if r["val_acc"] >= best_val_acc - 0.01
]

pre_pruned = ok_trees[0]

for r in ok_trees:
    if r["nodes"] < pre_pruned["nodes"]:
        pre_pruned = r

print("Configuration:")
print({
    "criterion": pre_pruned["criterion"],
    "max_depth": pre_pruned["max_depth"],
    "min_samples_split": pre_pruned["min_samples_split"],
    "min_samples_leaf": pre_pruned["min_samples_leaf"]
})


print("\nMetrics:")
print(fmt({
    "train_acc": pre_pruned["train_acc"],
    "val_acc": pre_pruned["val_acc"],
    "depth": pre_pruned["depth"],
    "nodes": pre_pruned["nodes"],
    "leaves": pre_pruned["leaves"]
}))
```

```
Configuration:
{'criterion': 'entropy', 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf': 50}

Metrics:
{'train_acc': 0.7596, 'val_acc': 0.7344, 'depth': 19, 'nodes': 1441, 'leaves': 721}
```

انتخاب درخت هرس‌شده برای مقایسه‌ی نهایی

در این مرحله، نتایج تمام آزمایش‌های هرس اولیه، شامل تغییر پارامترهای min_samples_split و min_samples_leaf، با یکدیگر مقایسه می‌شوند تا یک درخت مناسب برای مقایسه در بخش G انتخاب شود.

ابتدا بیشترین دقت اعتبارسنجی در بین تمام آزمایش‌ها محاسبه می‌شود. سپس درخت‌هایی انتخاب می‌شوند که دقت اعتبارسنجی آن‌ها حداکثر 0.01کمتر از بهترین دقت به دست آمده باشد. از بین این درخت‌ها، مدلی انتخاب می‌شود که کمترین تعداد گره را داشته باشد. به این ترتیب، هدف فقط دستیابی به دقت بالا نیست، بلکه انتخاب درختی با پیچیدگی کمتر و عملکرد اعتبارسنجی قابل‌قبول نیز اهمیت دارد.

در پایان، تنظیمات مدل شامل معیار تقسیم(criterion) ، حداکثر عمق(max_depth) ، حداقل تعداد نمونه برای تقسیم گره (min_samples_split) و حداقل تعداد نمونه در هر برگ (min_samples_leaf) نمایش داده می‌شوند. همچنین دقت آموزش، دقت اعتبارسنجی، عمق، تعداد گره‌ها و تعداد برگ‌های درخت گزارش می‌شوند تا مدل انتخاب‌شده در بخش G با سایر روش‌ها مقایسه شود.

پاسخ سؤالات بخشE

۱ .با افزایش min_samples_split، اندازه‌ی درخت و اختلاف دقت آموزش و اعتبارسنجی چه تغییری می‌کنند؟

با افزایش min_samples_split، تعداد گره‌های درخت کاهش پیدا می‌کند؛ زیرا فقط گره‌هایی که تعداد نمونه‌ی کافی دارند اجازه‌ی تقسیم شدن خواهند داشت. در آزمایش انجام‌شده، تعداد گره‌ها از 25503به 2367کاهش یافت. همچنین دقت آموزش از 1.000به 0.773رسید، در حالی که دقت اعتبارسنجی از 0.656به 0.728افزایش پیدا کرد. بنابراین، اختلاف دقت آموزش و اعتبارسنجی کمتر شد و نشانه‌های بیش‌برازش کاهش یافتند.

۲ .آیا درخت بسیار کوچک‌تری با دقت اعتبارسنجی مشابه پیدا شد؟

بله. با افزایش min_samples_splitبه ۱۰۰، تعداد گره‌ها از 25503به 2367کاهش پیدا کرد و دقت اعتبارسنجی به 0.728رسید. همچنین با افزایش min_samples_leafبه ۵۰، تعداد گره‌ها به 1441 کاهش یافت و دقت اعتبارسنجی 0.734شد. بنابراین، می‌توان درختی بسیار کوچک‌تر از درخت اولیه داشت که عملکرد اعتبارسنجی آن همچنان مناسب باشد.

۳ .هرس اولیه (Pre-Pruning) در چه مرحله‌ای از ساخت درخت انجام می‌شود؟

هرس اولیه هنگام ساخت درخت و پیش از تقسیم هر گره انجام می‌شود. در این روش، شرایط تقسیم گره بررسی می‌شوند و اگر شرایط تعیین‌شده برقرار نباشند، تقسیم انجام نمی‌شود. برای مثال، در پارامتر min_samples_splitاگر تعداد نمونه‌های یک گره کمتر از حداقل مقدار تعیین‌شده باشد، آن گره دیگر تقسیم نخواهد شد. به این ترتیب، از ابتدا جلوی رشد بیش از حد درخت گرفته می‌شود.

۴ .تفاوت min_samples_splitو min_samples_leafچیست؟ کدام‌یک در نتایج آزمایش تأثیر بیشتری داشت؟

پارامتر min_samples_splitحداقل تعداد نمونه‌های لازم برای تقسیم یک گره را مشخص می‌کند؛ اما min_samples_leafحداقل تعداد نمونه‌هایی را تعیین می‌کند که باید در هر برگ باقی بمانند. به عبارت دیگر، اولی شرایط تقسیم گره را محدود می‌کند و دومی مانع ایجاد برگ‌هایی با تعداد نمونه‌ی بسیار کم می‌شود.

در نتایج این آزمایش، min_samples_leafتأثیر بیشتری بر کاهش اندازه‌ی درخت داشت. با افزایش این پارامتر به ۵۰، تعداد گره‌ها به 1441و دقت اعتبارسنجی به 0.734رسید؛ در حالی که با min_samples_split = 100تعداد گره‌ها 2367و دقت اعتبارسنجی 0.728بود.

قسمت F : Post-Pruning

```
# TODO — Experiment 4: Post-Pruning

post_tree =DecisionTree(**make_config())
post_tree.fit(X_train , y_train)

before = snapshot(post_tree)
print("Before pruning:")
print(fmt(before))

post_prune(post_tree , X_val , y_val)
after = snapshot(post_tree)
print("After pruning:")
print(fmt(after))

removed_nodes = before["nodes"] - after["nodes"]
print("\n Nodes removed:" , removed_nodes)

post_pruned = {
    "train_acc" : after["train_acc"],
    "val_acc" : after["val_acc"],
    "depth" : after["depth"],
    "nodes" : after["nodes"],
    "leaves" : after["leaves"]
}
```

✓ 1m 17.0s

```
Before pruning:
{'train_acc': 0.9988, 'val_acc': 0.6563, 'depth': 38, 'nodes': 25503, 'leaves': 12752}
After pruning:
{'train_acc': 0.8727, 'val_acc': 0.7501, 'depth': 35, 'nodes': 10281, 'leaves': 5141}

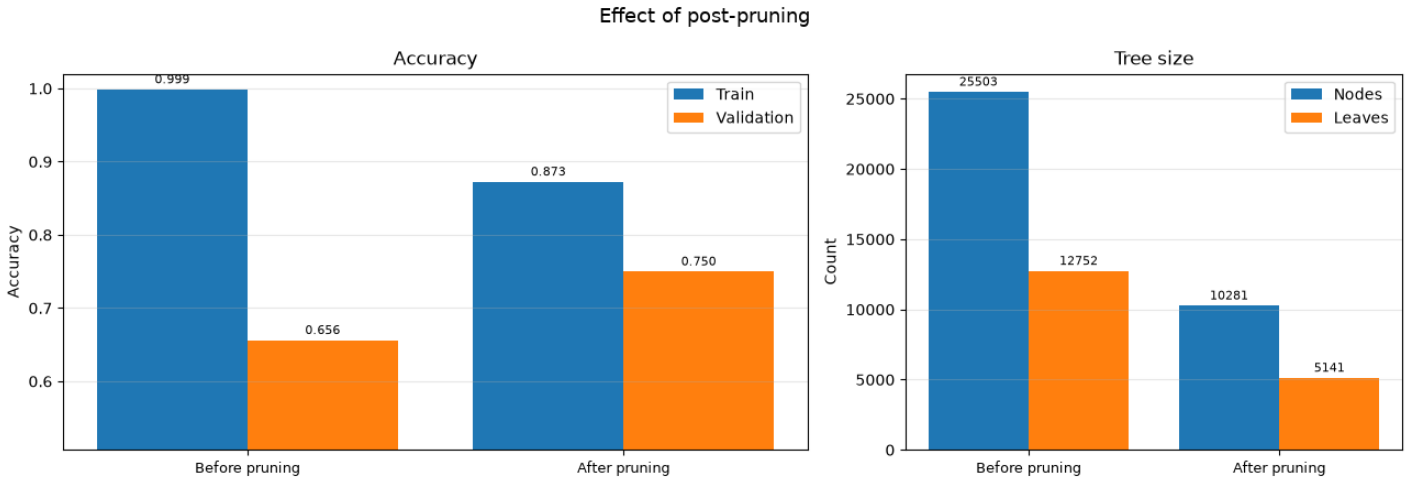
 Nodes removed: 15222
```

آزمایش چهارم: هرس پس از ساخت درخت(Post-Pruning)

در این آزمایش، ابتدا یک درخت تصمیم جدید و بدون محدودیت با استفاده از مجموعه‌ی آموزش ساخته می‌شود. سپس با تابع snapshot، وضعیت اولیه‌ی درخت شامل دقت آموزش، دقت اعتبارسنجی، عمق، تعداد گره‌ها و تعداد برگ‌ها ثبت می‌شود تا امکان مقایسه‌ی وضعیت درخت قبل و بعد از هرس وجود داشته باشد.

در مرحله‌ی بعد، تابع post_prune با استفاده از مجموعه‌ی اعتبارسنجی اجرا می‌شود. در هرس پس از ساخت، ابتدا درخت به‌طور کامل ساخته می‌شود و سپس گره‌هایی که حذف آن‌ها باعث بدتر شدن عملکرد مدل روی داده‌های اعتبارسنجی نمی‌شود، حذف می‌شوند. در این روش، تصمیم‌گیری برای حذف گره‌ها پس از بررسی زیردرخت‌های آن‌ها انجام می‌شود و هدف، ساده‌تر کردن ساختار درخت و کاهش بیش‌برازش است.

پس از هرس، وضعیت جدید درخت دوباره با تابع snapshot ثبت می‌شود. سپس تعداد گره‌های حذف‌شده از اختلاف تعداد گره‌ها قبل و بعد از هرس محاسبه می‌شود. در پایان، معیارهای نهایی شامل دقت آموزش، دقت اعتبارسنجی، عمق، تعداد گره‌ها و تعداد برگ‌ها در متغیر post_pruned ذخیره می‌شوند تا در بخش G با نتایج روش‌های دیگر مقایسه شوند.



نمودار نشان می‌دهد که هرس پس از ساخت توانسته است هم پیچیدگی درخت را کاهش دهد و هم دقت آن را روی داده‌های اعتبارسنجی بهبود ببخشد. پیش از هرس، دقت آموزش برابر با 0.999و دقت اعتبارسنجی برابر با 0.656بود. پس از هرس، دقت آموزش به 0.873کاهش یافت، اما دقت اعتبارسنجی به 0.750افزایش پیدا کرد. این تغییر نشان می‌دهد که حذف بخش‌های غیرضروری درخت باعث شده مدل کمتر به جزئیات داده‌های آموزشی وابسته باشد و عملکرد بهتری روی داده‌های جدید داشته باشد.

از نظر اندازه‌ی درخت نیز تغییر قابل‌توجهی مشاهده می‌شود. تعداد گره‌ها از 25503به 10281و تعداد برگ‌ها از 12752به 5141کاهش یافته است. بنابراین، بخش بزرگی از ساختار اولیه‌ی درخت حذف شده و مدل ساده‌تر شده است.

پاسخ سؤالات بخشF

۱ .چند گره حذف شد؟ دقت اعتبارسنجی چه تغییری کرد؟

تعداد گره‌ها قبل از هرس 25503و پس از هرس 10281بود؛ بنابراین، در مجموع 15222گره حذف شد. دقت اعتبارسنجی نیز از 0.656به 0.750افزایش یافت؛ یعنی حدود 9.4واحد درصد بهبود پیدا کرد. این نتیجه نشان می‌دهد که هرس باعث ساده‌تر شدن درخت و بهبود عملکرد آن روی داده‌های اعتبارسنجی شده است.

۲ .آیا کوچک‌تر بودن درخت لزوماً به معنی ضعیف‌تر بودن مدل است؟

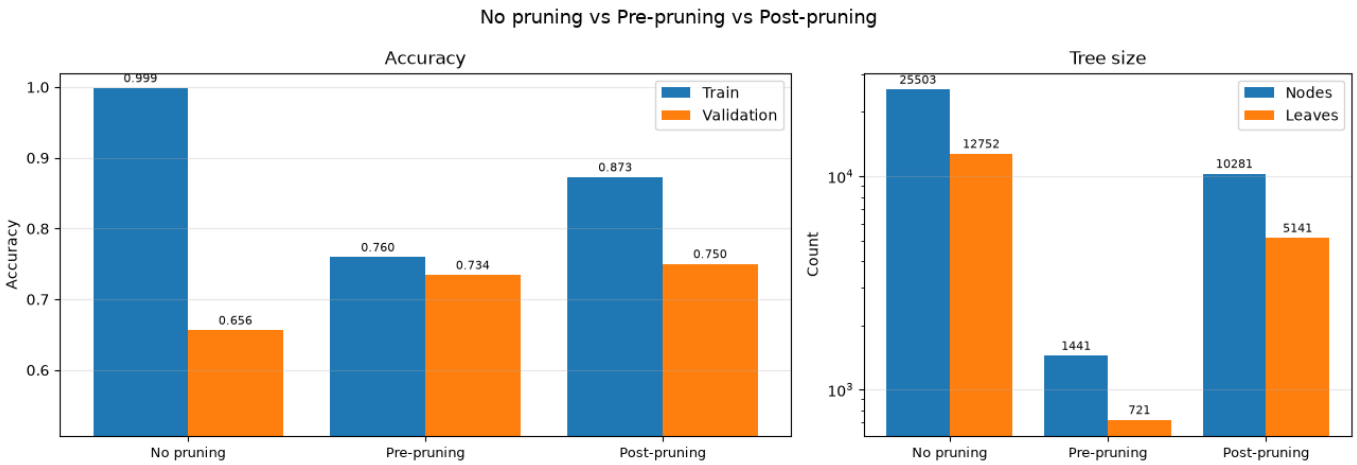
خیر. کوچک‌تر بودن درخت لزوماً به معنی ضعیف‌تر بودن آن نیست. یک درخت بزرگ ممکن است جزئیات و نویز داده‌های آموزشی را بیش از حد یاد بگیرد و دچار بیش‌برازش شود. در مقابل، یک درخت کوچک‌تر می‌تواند با حذف تقسیم‌های غیرضروری، الگوهای عمومی‌تر را یاد بگیرد و روی داده‌های جدید عملکرد بهتری داشته باشد. در این آزمایش نیز با کاهش اندازه‌ی درخت، دقت اعتبارسنجی افزایش یافته است.

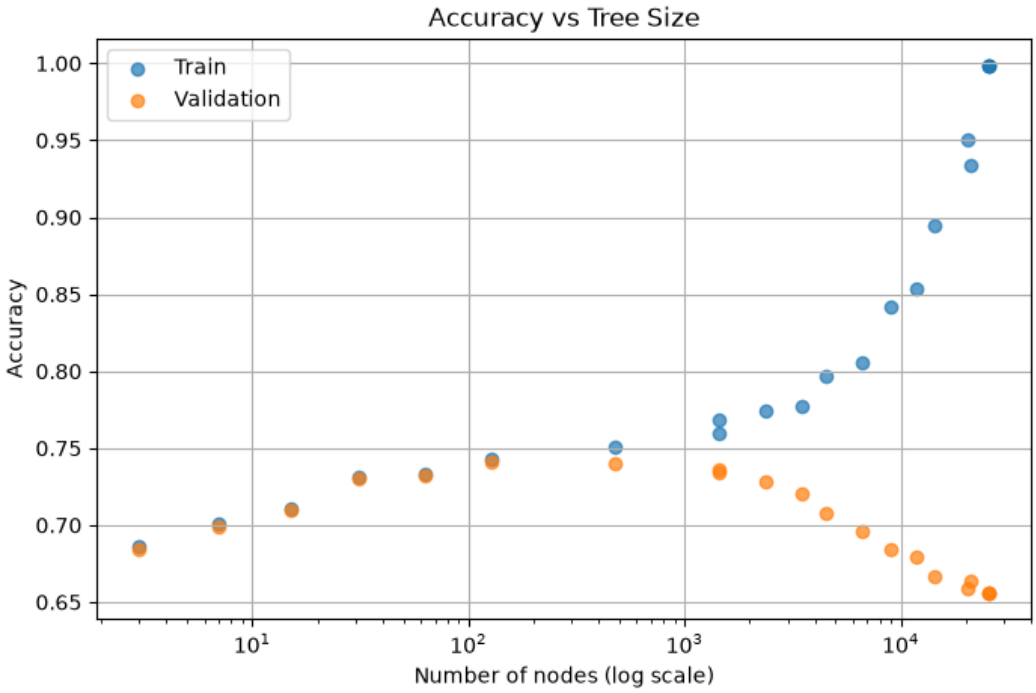
۳ .چرا دقت اعتبارسنجی پس از هرس کمی خوش‌بینانه است؟ کدام مجموعه ارزیابی منصفانه‌تری ارائه می‌دهد؟

دقت اعتبارسنجی پس از هرس کمی خوش‌بینانه است؛ زیرا از همین مجموعه برای تصمیم‌گیری درباره‌ی حذف گره‌ها استفاده شده است. به عبارت دیگر، ساختار درخت با توجه به عملکرد آن روی داده‌های اعتبارسنجی تغییر کرده و به‌نوعی با این داده‌ها تنظیم شده است. بنابراین، نتیجه‌ی به‌دست‌آمده ممکن است عملکرد واقعی مدل روی داده‌های کاملاً جدید را کمی بهتر از مقدار واقعی نشان دهد.

برای ارزیابی منصفانه‌تر، باید از **مجموعه‌ی آزمون (Test Set)** استفاده کرد؛ زیرا این مجموعه در آموزش مدل و تصمیم‌گیری درباره‌ی هرس دخالت نداشته است. در نتیجه، دقت مدل روی مجموعه‌ی آزمون تخمین قابل‌اعتمادتری از عملکرد آن روی داده‌های دیده‌نشده ارائه می‌دهد.

قسمت G : Experiment 5: No Pruning vs Pre-Pruning vs Post-Pruning





پاسخ سؤالات بخشG

۱ .کدام روش ساده‌ترین درخت را ایجاد کرد؟ آیا دقت اعتبارسنجی آن خیلی کاهش پیدا کرد؟

روش هرس اولیه (Pre-Pruning) ســاده‌ترین درخت را ایجاد کرد؛ به‌طوری‌که تعداد گره‌ها به 1441و تعداد برگ‌ها به 721رسـید. دقت اعتبارسـنجی این مدل برابر با 0.734بود، در حالی که دقت اعتبارسـنجی درخت بدون هرس 0.656و درخت هرس‌شـده پس از سـاخت 0.750بود. بنابراین، هرس اولیه نه‌تنها باعث کاهش

قابل‌توجه پیچیدگی درخت شد، بلکه دقت اعتبارسنجی آن نیز نسبت به درخت بدون هرس بهتر بود. اختلاف دقت آن با روش هرس پس از ساخت فقط 0.016است؛ بنابراین، توانسته با ساختاری بسیار کوچک‌تر، عملکرد نسبتاً نزدیکی به بهترین نتیجه ارائه دهد.

۲ .تفاوت مفهومی هرس اولیه و هرس پس از ساخت چیست؟ کدام‌یک هزینه‌ی کمتری دارد؟

در روش هرس اولیه، محدودیت‌ها هنگام ساخت درخت اعمال می‌شوند و از تقسیم گره‌هایی که شرایط لازم را ندارند جلوگیری می‌کنند. برای مثال، پارامترهای min_samples_splitو min_samples_leafاز رشد بیش از حد درخت جلوگیری می‌کنند. این روش معمولاً از نظر زمان و حافظه کم‌هزینه‌تر است؛ زیرا بسیاری از گره‌ها از ابتدا ساخته نمی‌شوند.

در مقابل، در روش هرس پس از ساخت، ابتدا درخت رشد می‌کند و سپس با بررسی عملکرد زیردرخت‌ها روی مجموعه‌ی اعتبارسنجی، بخش‌های غیرضروری حذف می‌شوند. این روش ممکن است هزینه‌ی محاسباتی بیشتری داشته باشد، چون ابتدا باید درخت بزرگ‌تری ساخته شود؛ اما می‌تواند فرصت بیشتری برای بررسی ساختار درخت و حذف بخش‌های اضافی فراهم کند.

۳ .اگر دو مدل دقت اعتبارسنجی مشابهی داشته باشند، چرا باید مدل کوچک‌تر را ترجیح دهیم؟

اگر دو مدل عملکرد مشابهی داشته باشند، معمولاً مدل کوچک‌تر انتخاب مناسب‌تری است؛ زیرا ساختار ساده‌تری دارد، حافظه‌ی کمتری مصرف می‌کند و پیش‌بینی با آن می‌تواند سریع‌تر انجام شود. همچنین، مدل‌های ساده‌تر معمولاً راحت‌تر قابل‌تفسیر هستند و ممکن است کمتر به جزئیات غیرضروری داده‌های آموزشی وابسته شوند. بنابراین، وقتی بهبود دقت مدل بزرگ‌تر ناچیز باشد، انتخاب مدل کوچک‌تر می‌تواند تعادل بهتری بین عملکرد، پیچیدگی و قابلیت تعمیم ایجاد کند.

قسمت H : Experiment 6: Gini vs Information Gain

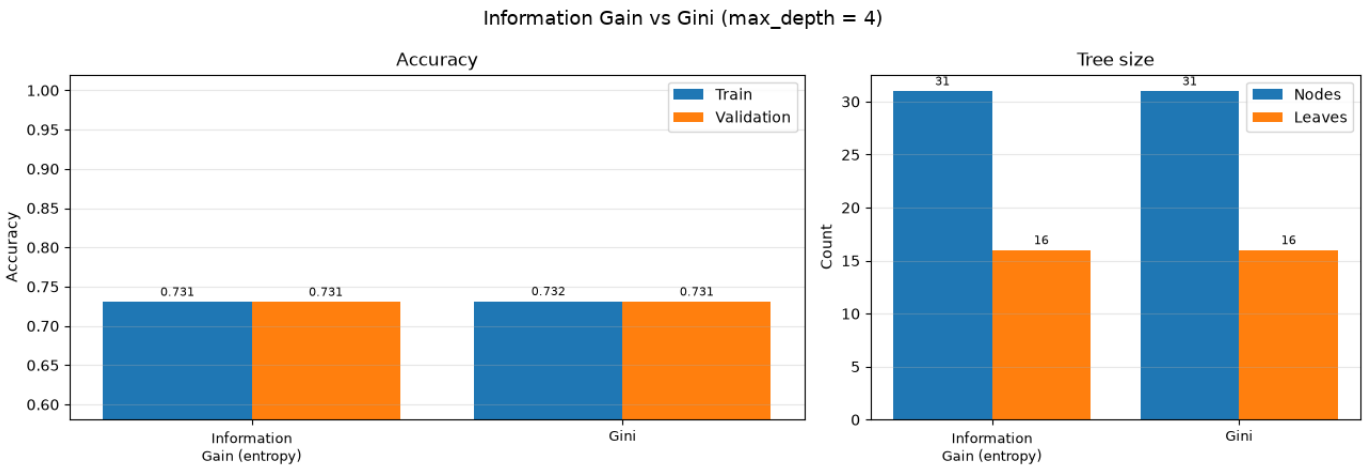
```
# TODO — Gini vs Information Gain (same data, same restrictions, only `criterion` changes)
#
gain_res = run_experiment(make_config(criterion= "entropy" , max_depth=4))
gini_res = run_experiment(make_config(criterion="gini" , max_depth=4))
```

مقایسه‌ی معیارهای Gini وInformation Gain

در این آزمایش، دو درخت تصمیم با استفاده از یک مجموعه‌داده و محدودیت‌های یکسان ساخته می‌شوند و تنها معیار انتخاب تقسیم در آن‌ها متفاوت است. درخت اول از معیار entropy استفاده می‌کند که مبنای محاسبه‌ی Information Gain است و درخت دوم از معیار gini برای ارزیابی کیفیت تقسیم‌ها استفاده می‌کند. در هر دو آزمایش، حداکثر عمق درخت برابر با ۴ تعیین شده است تا مقایسه در شرایط یکسان انجام شود.

معیار Information Gain میزان کاهش آنتروپی یا بی‌نظمی داده‌ها را پس از تقسیم اندازه‌گیری می‌کند؛ هرچه این کاهش بیشتر باشد، تقسیم مناسب‌تر است. در مقابل، معیار Gini میزان ناخالصی کلاس‌ها را در یک گره محاسبه می‌کند و تقسیم‌هایی را ترجیح می‌دهد که گره‌های حاصل را از نظر کلاس‌ها خالص‌تر کنند.

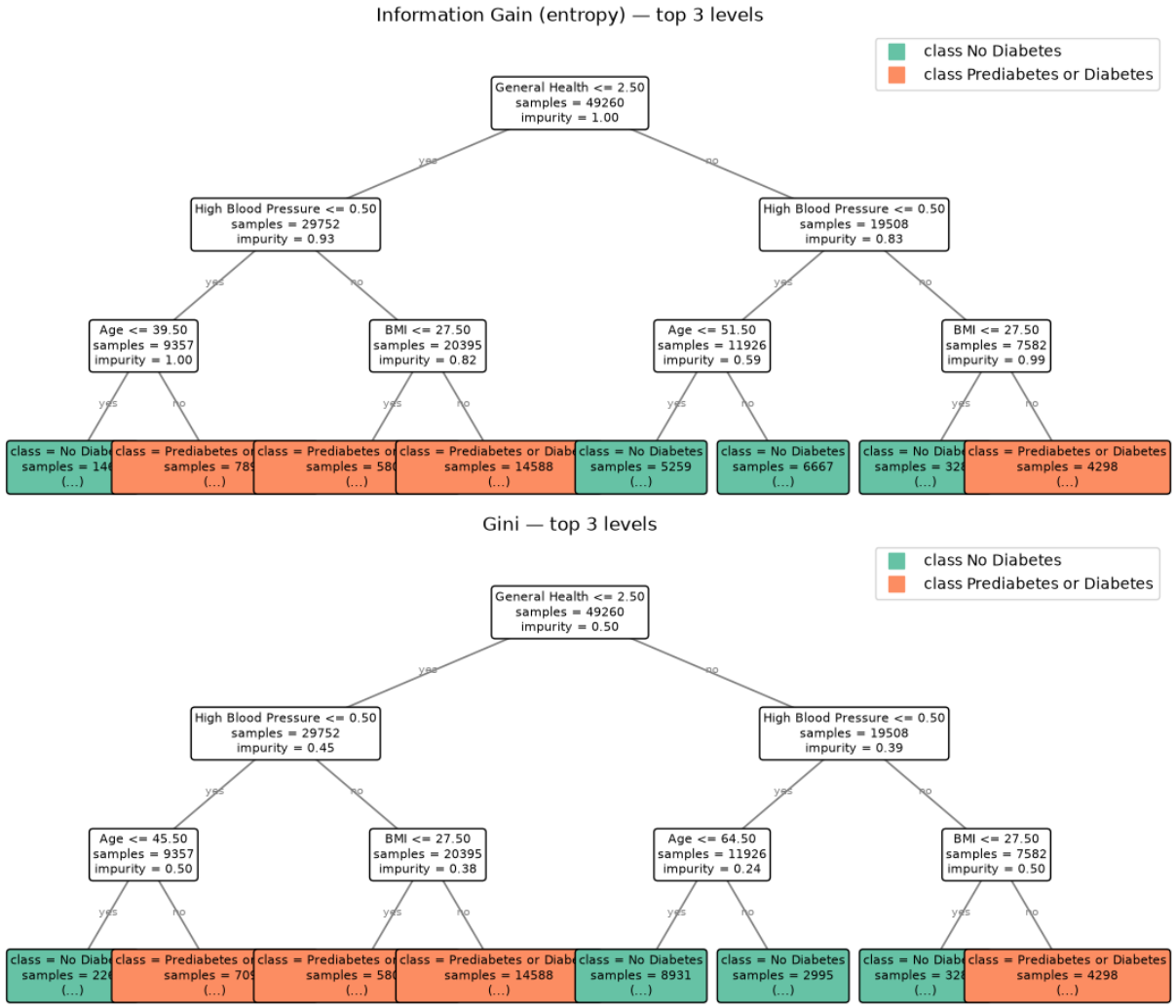
در پایان، نتایج دو آزمایش در متغیرهای gain_res و gini_res ذخیره می‌شوند تا دقت آموزش، دقت اعتبارسنجی و پیچیدگی درخت‌ها با یکدیگر مقایسه شوند



نمودار، عملکرد دو معیار Information Gain و Gini را در ساخت درخت تصمیم با حداکثر عمق ۴ مقایسه می‌کند. همان‌طور که مشاهده می‌شود، دقت آموزش برای معیار Information Gain برابر با 0.731 و برای Gini برابر با 0.732 است. دقت اعتبارسنجی هر دو مدل نیز تقریباً برابر با 0.731 است. بنابراین، اختلاف عملکرد این دو معیار بسیار ناچیز است و در این آزمایش هیچ‌کدام برتری قابل‌توجهی نسبت به دیگری ندارند.

از نظر پیچیدگی نیز هر دو درخت ساختار یکسانی دارند؛ تعداد گره‌ها در هر دو برابر با 31 و تعداد برگ‌ها برابر با 16 است. این موضوع نشان می‌دهد که با وجود تفاوت در معیار انتخاب تقسیم‌ها، در این آزمایش هر دو روش درخت‌هایی با اندازه‌ی یکسان ایجاد کرده‌اند.

در مجموع، نتایج نشـــان می‌دهند که در این مجموعه‌داده و با محدودیت عمق تعیین‌شـــده، انتخاب بین Information Gainو Gini تأثیر قابل‌توجهی بر دقت یا اندازه‌ی درخت نداشته است.



این دو نمودار، سـه سـطح ابتدایی درخت تصـمیم را با اسـتفاده از دو معیار Information Gain (مبتنی بر آنتروپی) و Gini نشـــان می‌دهند. در هر دو درخت، ویژگی General Healthدر ریشـــه قرار گرفته و اولین تقسـیم بر اسـاس شـرط General Health <= 2.50انجام شـده اسـت. این موضـوع نشـان می‌دهد که هر دو معیار در این مرحله، این ویژگی را برای تقسیم داده‌ها مناسب تشخیص داده‌اند.

پس از آن، ویژگی High Blood Pressureدر هر دو شـــاخه اســـتفاده شـــده اســـت. در ســـطوح بعدی نیز ویژگی‌های Ageو BMIبرای تقسـیم بیشـتر داده‌ها به کار رفته‌اند. با این حال، آسـتانه‌ی تقسـیم مربوط به سـن در دو روش متفاوت اسـت؛ برای مثال، در یکی از شـاخه‌ها آسـتانه‌ی سـن در درخت مبتنی بر Information

Gainبرابر با 39.50و در درخت مبتنی بر Gini برابر با 45.50اســـت. این تفاوت نشـــان می‌دهد که معیار انتخاب تقسـیم می‌تواند بر جزئیات سـاختار درخت تأثیر بگذارد، حتی اگر ویژگی‌های اصـلی انتخاب‌شـده مشـابه باشند.

در مجموع، هر دو معیار درختی مشابه با ویژگی‌های اصلی یکسان ایجاد کرده‌اند، اما در بعضی آستانه‌ها و نحوه‌ی تقسیم نمونه‌ها تفاوت دارند.

قسمت I : Experiment 7: Opening the Box (Explainability)

```
# TODO — explain ONE prediction
explain_tree = DecisionTree(**make_config(max_depth=4))
explain_tree.fit(X_train , y_train)
sample = X_val.iloc[0]
print(f"True label : {y_val.iloc[0]} , Predicted label: {explain_tree.predict(X_val.iloc[[0]])}" )
print(sample.to_string())
trace_path(explain_tree , sample)
```

✓ 26.1s

```
True label : No Diabetes , Predicted label: ['No Diabetes']
High Blood Pressure                 0.0
High Cholesterol                    1.0
Cholesterol Check                   1.0
BMI                                24.0
Smoker                              0.0
Stroke                              0.0
Heart Disease or Attack             0.0
Physical Activity                   0.0
Fruits Daily                        1.0
Vegetables Daily                    1.0
Heavy Alcohol Consumption           0.0
Has Healthcare                      1.0
Could Not See Doctor Due to Cost    0.0
General Health                      4.0
Poor Mental Health Days             0.0
Poor Physical Health Days           0.0
Difficulty Walking                  0.0
Sex                                 0.0
Age                                62.0
Education Level                     6.0
Income Level                        8.0
Step 1: General Health = 4 > 2.500 -> go right (node samples = 49260)
Step 2: High Blood Pressure = 0 <= 0.500 -> go left (node samples = 19508)
Step 3: Age = 62 > 51.500 -> go right (node samples = 11926)
Step 4: General Health = 4 > 3.500 -> go right (node samples = 6667)
Leaf reached -> predict No Diabetes (samples = 2279, counts = {'No Diabetes': np.int64(2057), 'Prediabetes or Diabetes': np.int64(222)})

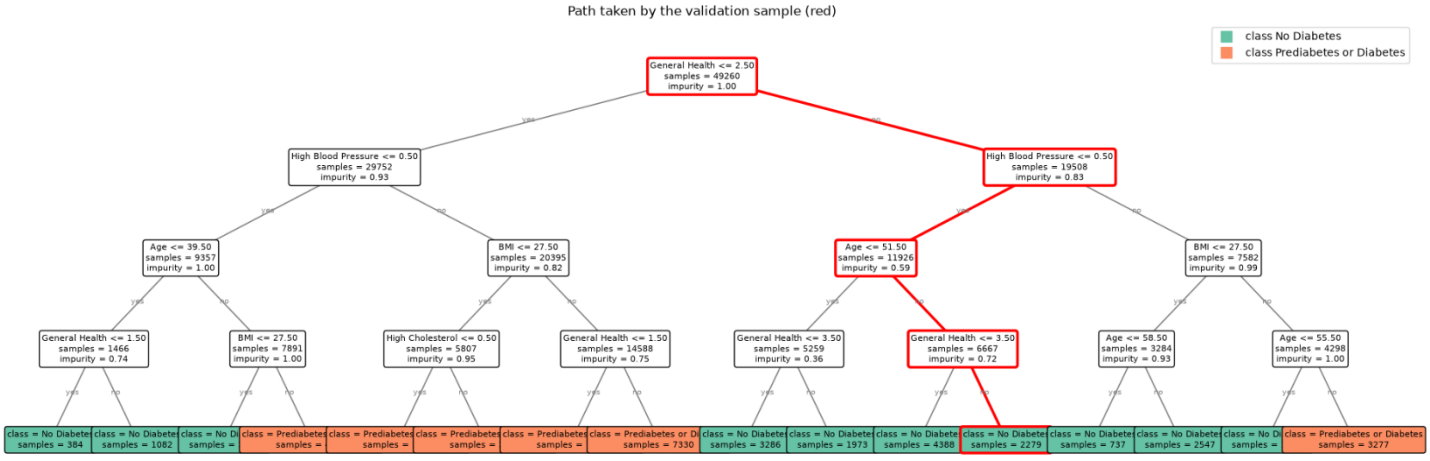
'No Diabetes'
```

توضیح مسیر پیش‌بینی یک نمونه

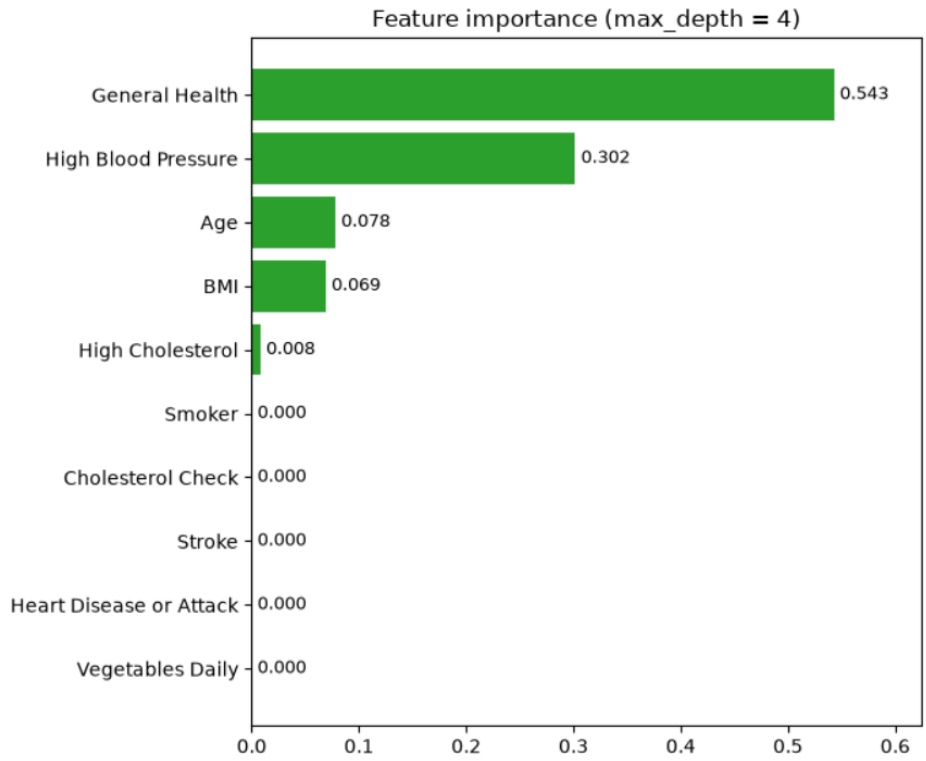
در این بخش، یک درخت تصمیم با حداکثر عمق ۴ ساخته و با استفاده از داده‌های آموزشی آموزش داده می‌شود. سپس اولین نمونه از مجموعه‌ی اعتبارسنجی انتخاب می‌شود و برچسب واقعی آن با برچسب پیش‌بینی‌شده توسط مدل مقایسه می‌شود.

با استفاده از تابع trace_path، مسیر حرکت نمونه در درخت نیز نمایش داده می‌شود. این تابع در هر گره، ویژگی مورد بررسی، مقدار آن برای نمونه، آستانه‌ی تقسیم و جهت حرکت به شاخه‌ی چپ یا راست را چاپ می‌کند. این فرایند تا رسیدن به یک برگ ادامه پیدا می‌کند؛ سپس کلاس پیش‌بینی‌شده و اطلاعات مربوط به برگ نمایش داده می‌شود.

هدف از این آزمایش، علاوه بر بررسی نتیجه‌ی پیش‌بینی، مشخص کردن نحوه‌ی تصمیم‌گیری مدل است تا بتوان فهمید که درخت بر اساس چه ویژگی‌هایی نمونه را به کلاس نهایی نسبت داده است.



```
importance = feature_importance(explain_tree)
plot_feature_importance(importance, top=10, title="Feature importance (max_depth = 4)")
```
✓ 0.1s



این نمودار میزان اهمیت هر ویژگی را در پیش‌بینی مدل درخت تصمیم با حداکثر عمق ۴ نشان می‌دهد. اهمیت ویژگی مشخص می‌کند که هر ویژگی تا چه اندازه در تقسیم گره‌های درخت و کاهش ناخالصی داده‌ها نقش داشته است. هرچه مقدار اهمیت بیشتر باشد، آن ویژگی نقش بیشتری در تصمیم‌گیری مدل داشته است.

همان‌طور که مشاهده می‌شود، ویژگی General Healthبا اهمیت 0.543بیشترین تأثیر را در مدل دارد و پس از آن High Blood Pressureبا مقدار 0.302قرار گرفته است. این دو ویژگی در مجموع حدود ۸۴٫۵ درصد از اهمیت کل ویژگی‌ها را تشکیل می‌دهند. ویژگی‌های Ageو BMIنیز به‌ترتیب با مقادیر 0.078و 0.069در رتبه‌های بعدی قرار دارند. در مقابل، اهمیت ویژگی High Cholesterolفقط 0.008است و ویژگی‌هایی مانند Smoker، Cholesterol Check، Stroke، Heart Disease or Attackو Vegetables Dailyدر این درخت اهمیتی برابر با صفر دارند.

صفر بودن اهمیت یک ویژگی به این معنی است که آن ویژگی در تقسیم‌های مؤثر درخت استفاده نشده است؛

پاسخ سؤالات بخش۱

۱ .کدام ویژگی‌ها و شرط‌ها در تعیین پیش‌بینی این نمونه نقش داشتند؟

. ویژگی هایی مانند General Healthیا High Blood Pressure ،Age ،BMI بررسی می‌شود و بر اساس مقدار نمونه، مسیر به شاخه‌ی چپ یا راست ادامه پیدا می‌کند.

۲ .آیا مهم‌ترین ویژگی مدل در مسیر پیش‌بینی تمام نمونه‌ها استفاده می‌شود؟ چرا؟

خیر. مهم‌ترین ویژگی در کل مدل لزوماً در مسیر پیش‌بینی تمام نمونه‌ها استفاده نمی‌شود. برای مثال، اگر General Healthدر ریشه‌ی درخت قرار داشته باشد، تمام نمونه‌ها ابتدا بر اساس این ویژگی تقسیم می‌شوند؛ اما ویژگی‌های مراحل بعدی به شاخه‌ای بستگی دارند که هر نمونه وارد آن می‌شود. بنابراین، ممکن است یک ویژگی در یک شاخه استفاده شود، ولی در شاخه‌ی دیگر اصلاً وجود نداشته باشد. همچنین، اهمیت کلی ویژگی‌ها نشان‌دهنده‌ی نقش آن‌ها در کل درخت است، نه اینکه هر ویژگی حتماً در تصمیم‌گیری درباره‌ی هر نمونه نقش داشته باشد.

قسمت J : Is One Validation Split Reliable? K-Fold Cross-Validation

```python
def cross_validate(config):

    # TODO
    train_accs = []
    val_accs = []
    node_counts = []
    fold_accs = []
    for train_idx , val_idx in folds:
        tree = DecisionTree(**config)
        X_train_fold = X_dev.iloc[train_idx]
        y_train_fold = y_dev.iloc[train_idx]

        X_val_fold = X_dev.iloc[val_idx]
        y_val_fold = y_dev.iloc[val_idx]

        tree.fit(X_train_fold , y_train_fold)

        train_pred = tree.predict(X_train_fold)
        val_pred = tree.predict(X_val_fold)

        train_acc = accuracy_score_manual(y_train_fold , train_pred)
        val_acc = accuracy_score_manual(y_val_fold , val_pred)

        train_accs.append(train_acc)
        val_accs.append(val_acc)

        node_counts.append(tree.n_nodes_)

        fold_accs.append(val_acc)

    return {
        "cv_train_acc": np.mean(train_accs),
        "cv_val_acc": np.mean(val_accs),
        "cv_val_std": np.std(val_accs),
        "cv_nodes": np.mean(node_counts),
        "fold_accs": fold_accs
    }
```

پیاده‌سازی اعتبارسنجی متقابل(Cross-Validation)

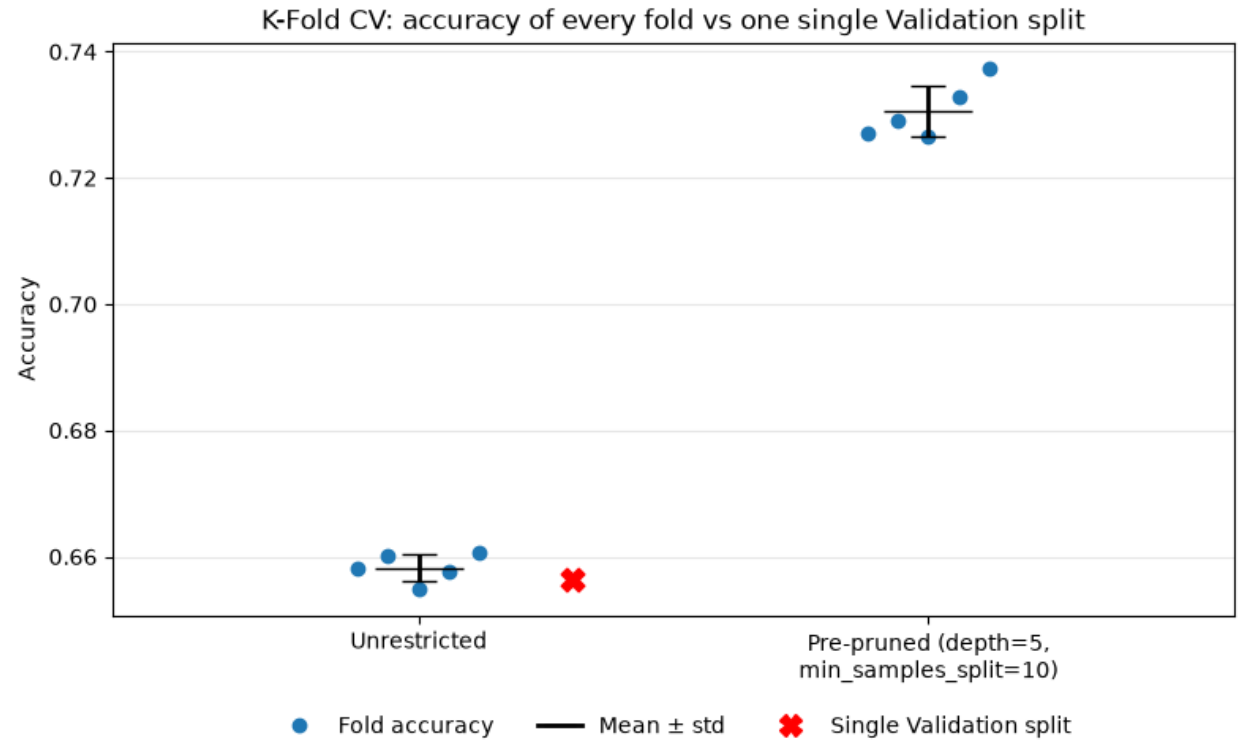
در این بخش، عملکرد یک پیکربندی از درخت تصمیم با استفاده از روش اعتبارسنجی متقابل ارزیابی می‌شود. در هر مرحله، یک درخت جدید ساخته می‌شود و با بخشی از داده‌های توسعه آموزش می‌بیند؛ سپس دقت آن

هم روی داده‌های آموزش همان مرحله و هم روی داده‌های اعتبارسنجی محاسبه می‌شود. این فرایند برای تمام تقسیم‌بندی‌های از پیش تعیین‌شده در folds تکرار می‌شود.

علاوه بر دقت آموزش و اعتبارسنجی، تعداد گره‌های هر درخت نیز ثبت می‌شود تا پیچیدگی مدل بررسی شود. در پایان، میانگین دقت آموزش، میانگین دقت اعتبارسنجی، انحراف معیار دقت‌های اعتبارسنجی و میانگین تعداد گره‌ها محاسبه می‌شوند. همچنین، دقت اعتبارسنجی هر مرحله به‌صورت جداگانه در fold_accs نگهداری می‌شود.

این روش کمک می‌کند عملکرد مدل فقط به یک تقسیم‌بندی خاص از داده‌ها وابسته نباشد. میانگین دقت، تصویری کلی از عملکرد مدل ارائه می‌دهد و انحراف معیار نشان می‌دهد که نتایج مدل در تقسیم‌بندی‌های مختلف چقدر تغییر می‌کنند.

```
Time for 10 fits: 529.5s
Unrestricted: mean = 0.6582, std = 0.0021
Pre-pruned (depth=5, min_samples_split=10): mean = 0.7306, std = 0.0040
```



این نمودار دقت مدل را در چند مرحله‌ی اعتبارسنجی متقابل با دقت حاصل از یک تقسیم‌بندی واحد از داده‌ها مقایسه می‌کند. نقاط آبی نشان‌دهنده‌ی دقت هر مرحله یا Fold هستند و خط‌های سیاه میانگین دقت به‌همراه یک انحراف معیار را نمایش می‌دهند. علامت ضربدر قرمز نیز دقت به‌دست‌آمده از یک مجموعه‌ی اعتبارسنجی ثابت را نشان می‌دهد.

در مدل بدون محدودیت(Unrestricted) ، دقتFold ها تقریباً بین 0.655تا 0.660قرار دارد و میانگین آن‌ها حدود 0.658است. دقت در تقسیم‌بندی واحد نیز نزدیک به 0.656است. این مقادیر نشان می‌دهند که نتیجه‌ی مدل در تقسیم‌بندی‌های مختلف نسبتاً نزدیک است، اما دقت کلی آن پایین‌تر از مدل هرس‌شده است.

در مدل هرس اولیه (Pre-pruned) با max_depth=5و min_samples_split=10، دقتFold ها تقریباً بین 0.727تا 0.737قرار دارد و میانگین آن‌ها حدود 0.730است. این مقدار به‌وضوح از دقت مدل بدون محدودیت بیشتر است. همچنین، پراکندگی نسبتاً کم نقاط نشان می‌دهد که عملکرد مدل درFold های مختلف تغییر زیادی ندارد.

در مجموع، مدل هرس اولیه در مقایسه با مدل بدون محدودیت، هم دقت بالاتری دارد و هم در تقسیم‌بندی‌های مختلف عملکرد نسبتاً پایداری نشان می‌دهد. مقایسه‌ی اعتبارسنجی تقسیم بندی شده با یک تقسیم‌بندی واحد نیز اهمیت دارد؛ زیرا نتیجه‌ی یک تقسیم‌بندی ممکن است تا حدی تحت تأثیر نحوه‌ی انتخاب داده‌های آموزش و اعتبارسنجی قرار بگیرد.

پاسخ سؤالات بخشJ

۱ .دقتFold ها برای یک پیکربندی چقدر تغییر می‌کند؟ این موضوع چه چیزی درباره‌ی اعتماد به یک تقسیم‌بندی واحد نشان می‌دهد؟

در مدل بدون محدودیت، دقتFold ها تقریباً بین 0.655و 0.660و در مدل هرس اولیه بین 0.727و 0.737تغییر می‌کند. بنابراین، پراکندگی دقت‌ها در هر دو مدل نسبتاً کم است و عملکرد آن‌ها در تقسیم‌بندی‌های مختلف تفاوت زیادی ندارد. با این حال،اعتماد به یک تقسیم‌بندی واحد ممکن است نتیجه‌ای وابسته به انتخاب داده‌های آموزش و اعتبارسنجی ارائه دهد. استفاده از اعتبارسنجی تقسیم بندی شده، با بررسی چند تقسیم‌بندی، ارزیابی قابل‌اعتمادتری از عملکرد مدل فراهم می‌کند.

۲ .کدام مدل انحراف معیار کمتری دارد و پایدارتر است؟ چرا؟

از روی نمودار، پراکندگی دقت‌ها در مدل بدون محدودیت کمی کمتر به نظر می‌رسد؛ بنابراین، انحراف معیار آن کوچک‌تر است. مدل هرس اولیه با وجود دقت بالاتر، لندکی پراکندگی بیشتر دارد. درخت بدون محدودیت، آزادی بیشتری برای پیدا کردن تقسیم‌های مناسب دارد و می‌تواند الگوهای بیشتری را در داده‌های آموزشی یاد بگیرد .در این آزمایش، ظاهراً این تقسیم‌ها باعث شده‌اند عملکرد مدل در Foldهای مختلف به هم نزدیک‌تر باشد و انحراف معیار کمتری به دست بیاید و پایدار تر باشد.

۳ .چرا همه‌ی پیکربندی‌ها باید رویFold های یکسان ارزیابی شوند؟

چون تفاوت عملکرد آن‌ها بیشتر به تنظیمات مدل مربوط می‌شود، نه به تفاوت در نمونه‌های آموزش و اعتبارسنجی. اگر برای هر پیکربندیFold های متفاوتی استفاده شود، ممکن است یک مدل صرفاً به دلیل آسان‌تر بودن تقسیم‌بندی‌ها نتیجه‌ی بهتری بگیرد. بنابراین، یکسان بودنFold ها مقایسه را منصفانه‌تر و قابل‌اعتمادتر می‌کند.

قسمت K : Hyperparameter Tuning: Grid Search + K-Fold CV

```
from itertools import product

param_grid = {
    "criterion": ["gini", "entropy"],
    "max_depth": [4, 5, 6, None],
    "min_samples_split": [50, 75, 100],
    "min_samples_leaf": [40, 50]
}
# You may add more values or `min_samples_leaf` (see Part E) — but watch the runtime!

combos = list(product(*param_grid.values()))
print(f"{len(combos)} combinations x {N_FOLDS} folds = {len(combos) * N_FOLDS} fits")

t0 = time.time()
cross_validate(make_config(max_depth=5, min_samples_split=10))          # time a single CV round
print(f"Estimated total time: ~{(time.time() - t0) * len(combos) / 60:.1f} min")
```

✓ 2m 32.2s

```
48 combinations x 5 folds = 240 fits
Estimated total time: ~121.8 min
```

در این بخش، از روش (Grid Search)برای بررسی ترکیب‌های مختلف پارامترهای درخت تصمیم استفاده می‌شود. پارامترهای مورد بررسی شامل (criterion)، حداکثر عمق درخت(max_depth) ، حداقل تعداد نمونه برای تقسیم یک گره (min_samples_split) و حداقل تعداد نمونه در هر برگ (min_samples_leaf)هستند.

در این آزمایش، ۴۸ ترکیب مختلف وجود دارد و هر ترکیب باید در تمام مراحل اعتبارسنجی متقابل ارزیابی شود. بنابراین، تعداد کل دفعات آموزش مدل برابر با تعداد ترکیب‌ها ضرب‌در تعدادFold ها خواهد بود. در مجموع در این آزمایش ۲۴۰ فیت داریم که در کل ۱۲۱.۸ دقیقه تایم جست و جوی آن تخمین زده شده است.(که البته این آزمایش در حدود ۱۹۴ دقیقه به انجام رسید.)

```
# TODO — run the grid search

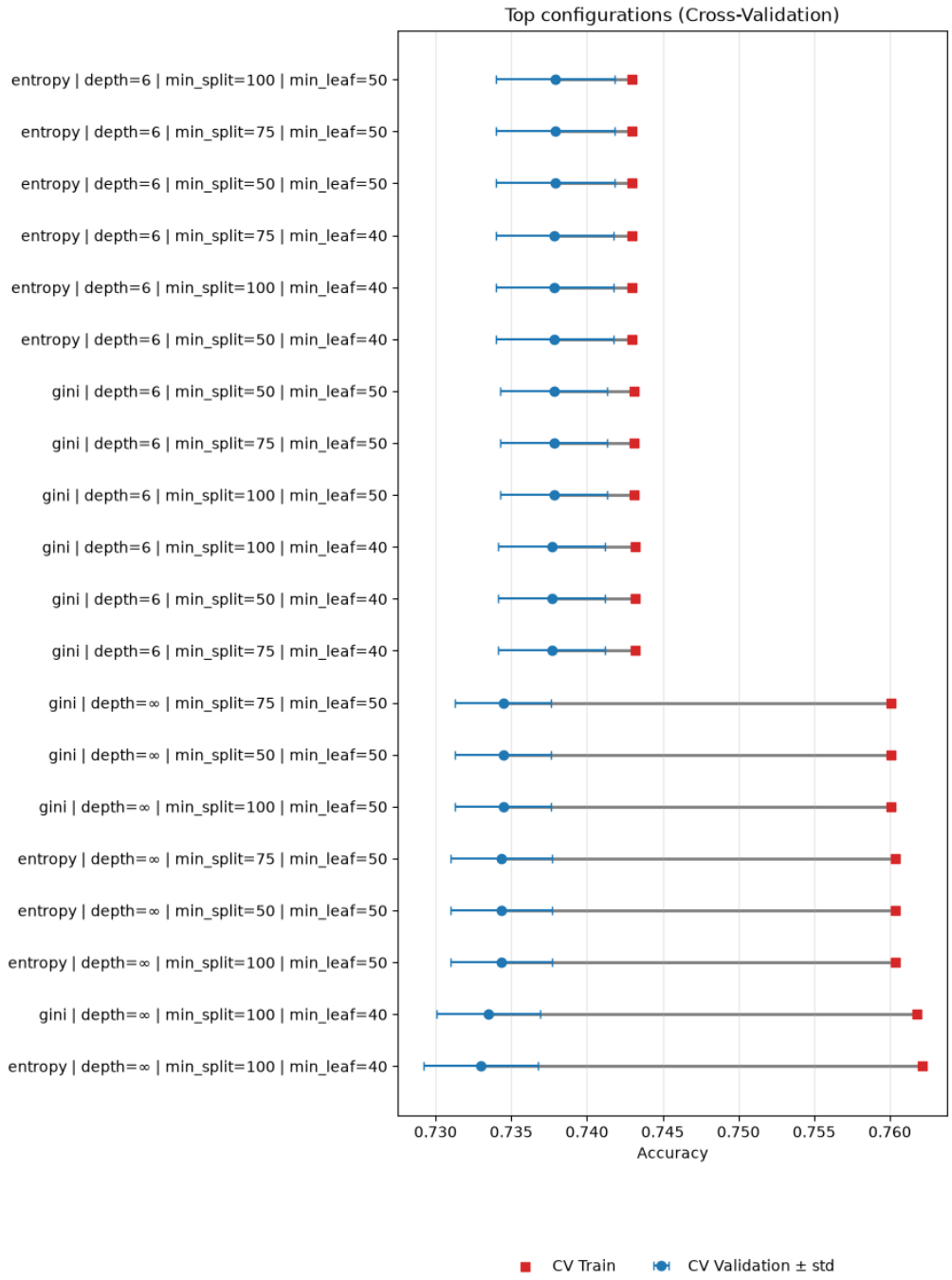
rows = []
for criterion , depth , min_s , min_l  in combos:
    cv = cross_validate(make_config(criterion=criterion , max_depth=depth , min_samples_split=min_s , min_samples_leaf=min_l))
    row = {
    "criterion": criterion,
    "max_depth": depth,
    "min_samples_split": min_s,
    "min_samples_leaf" : min_l,
    "cv_train_acc": cv["cv_train_acc"],
    "cv_val_acc": cv["cv_val_acc"],
    "cv_val_std": cv["cv_val_std"],
    "cv_nodes": cv["cv_nodes"]
}
    rows.append(row)

grid_df = pd.DataFrame(rows)
grid_df["gap"] = grid_df["cv_train_acc"] - grid_df["cv_val_acc"]
```

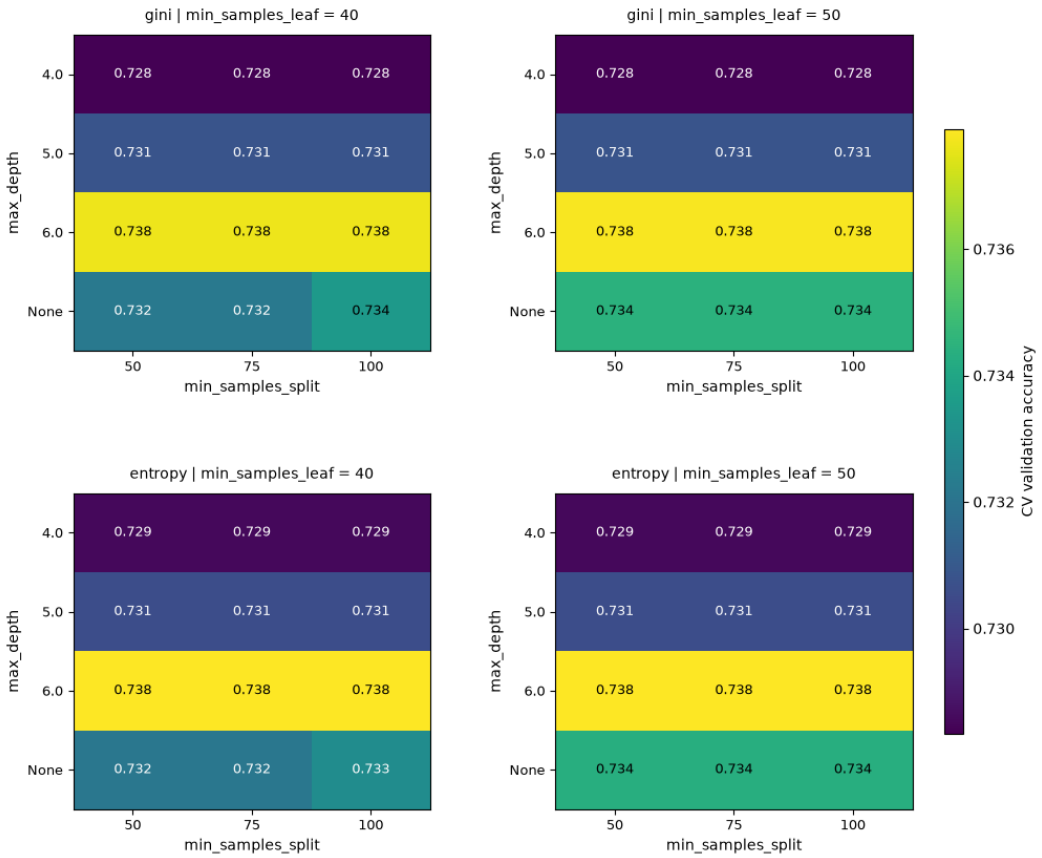
در این بخش، جست‌وجوی شبکه‌ای به‌صورت کامل اجرا می‌شود تا بهترین ترکیب پارامترهای درخت تصمیم بر اساس نتایج اعتبارسنجی متقابل مشخص شود. برای هر ترکیب از معیار تقسیم‌بندی، حداکثر عمق درخت، حداقل تعداد نمونه برای تقسیم گره و حداقل تعداد نمونه در برگ، مدل ارزیابی شده و نتایج آن در قالب یک سطر ذخیره می‌شوند.

برای هر پیکربندی، میانگین دقت آموزش(cv_train_acc) ، میانگین دقت اعتبارسنجی(cv_val_acc) ، انحراف معیار دقت اعتبارسنجی (cv_val_std) و میانگین تعداد گره‌ها (cv_nodes) ثبت می‌شود.

در پایان، اختلاف میان دقت آموزش و اعتبارسنجی با نام gapمحاسبه می‌شود. این مقدار معیاری برای بررسی بیش‌برازش است.



در این نمودار، عملکرد ۲۰ پیکربندی مختلف درخت تصمیم بر اساس اعتبارسنجی متقابل مقایسه شده است. مربع‌های قرمز نشان‌دهنده دقت آموزش و دایره‌های آبی نشان‌دهنده میانگین دقت اعتبارسنجی هستند. خطوط اطراف نقاط آبی نیز میزان تغییرپذیری دقت اعتبارسنجی را با استفاده از انحراف معیار نشان می‌دهند.



در این نمودار، تاثیر پارامترهای max_depth، min_samples_split و min_samples_leafبر میانگین دقت اعتبارسنجی در دو معیار giniو entropyبررسی شده است. نتایج نشان می‌دهند که در هر چهار حالت، عمق ۶ بهترین عملکرد را دارد و دقت اعتبارسنجی به حدود ۰٫۷۳۸ می‌رسد. در عمق‌های ۴ و ۵، دقت به‌ترتیب حدود ۰٫۷۲۸ تا ۰٫۷۲۹ و ۰٫۷۳۱ است.

همچنین، تغییر مقدار min_samples_splitاز ۵۰ به ۱۰۰ در عمق‌های محدود تأثیر محسوسی بر دقت ندارد. تغییر min_samples_leafاز ۴۰ به ۵۰ نیز نتایج بسیار مشابهی ایجاد می‌کند. در درخت‌های بدون محدودیت عمق، دقت اعتبارسنجی حدود ۰٫۷۳۲ تا ۰٫۷۳۴ است که کمی پایین‌تر از بهترین نتیجه به‌دست‌آمده در عمق ۶ قرار دارد.

در مجموع، این نتایج نشان می‌دهند که در آزمایش حاضر، انتخاب عمق مناسب درخت تأثیر بیشتری بر دقت اعتبارسنجی دارد و تفاوت میان دو معیار giniو entropyو همچنین مقادیر مختلف پارامترهای دیگر نسبتاً کم است. بنابراین، عمق ۶ گزینه مناسبی برای ادامه بررسی و انتخاب مدل نهایی محسوب می‌شود.

پ . ن : کد مربوط به رسم نمودار این دو شکل رو کمی تغییر دادم تا ویژگی min_samples_leaf هم وارد بازی شود . در ضمن از CHAT GPT برای این کار استفاده شده است.

```python
TOLERANCE = 0.01

# TODO – "simplest model that is almost as good as the best" rule
best = grid_df["cv_val_acc"].max()
candidates = grid_df[
    grid_df["cv_val_acc"] >= best - TOLERANCE
    ]

candidates["max_depth"] = candidates["max_depth"].fillna(float("inf"))
candidates = candidates.sort_values(
    by=["cv_nodes" , "max_depth"],
    ascending=True
    )
suggested =candidates.iloc[0]


print("Best CV accuracy:", round(float(best), 4))
print("Number of candidates:", len(candidates))

print("Suggested configuration:")
print("criterion:", suggested["criterion"])
print("max_depth:", suggested["max_depth"])
print("min_samples_split:", suggested["min_samples_split"])
print("min_samples_leaf:", suggested["min_samples_leaf"])
print(f"cv_val_acc: {suggested['cv_val_acc']:.4f} ± {suggested['cv_val_std']:.4f}")
print("cv_nodes:", suggested["cv_nodes"])
```

✓ 0.0s

```
Best CV accuracy: 0.7379
Number of candidates: 48
Suggested configuration:
criterion: gini
max_depth: 4.0
min_samples_split: 50
min_samples_leaf: 40
cv_val_acc: 0.7283 ± 0.0047
cv_nodes: 31.0
```

انتخاب مدل نهایی با توجه به دقت و پیچیدگی

در این بخش، مدلی انتخاب می‌شود که ضمن داشتن دقت اعتبارسنجی مناسب، پیچیدگی کمتری نیز داشته باشد. ابتدا بیشترین دقت اعتبارسنجی در میان تمام پیکربندی‌ها محاسبه می‌شود. سپس، پیکربندی‌هایی که دقت آن‌ها حداکثر به‌اندازه ۰٫۰۱ کمتر از بهترین دقت است، به‌عنوان گزینه‌های قابل قبول در نظر گرفته می‌شوند.

در مرحله بعد، از میان این گزینه‌ها مدلی انتخاب می‌شود که کمترین تعداد گره را داشته باشد. اگر تعداد گره‌های چند مدل برابر باشد، مدلی با عمق کمتر ترجیح داده می‌شود. برای مقایسه، عمق نامحدود با مقدار بی‌نهایت جایگزین می‌شود تا در مرتب‌سازی، درخت‌های دارای عمق محدود اولویت داشته باشند.

در پایان، معیار تقسیم‌بندی، عمق درخت، حداقل تعداد نمونه برای تقسیم گره، حداقل تعداد نمونه در برگ، میانگین دقت اعتبارسنجی به‌همراه انحراف معیار و تعداد گره‌های مدل پیشنهادی نمایش داده می‌شوند. این روش تلاش می‌کند مدلی ساده‌تر انتخاب کند که عملکرد آن همچنان نزدیک به بهترین نتیجه آزمایش باشد.

پاسخ سؤالات بخشK

1.کدام بخش از شبکه بهترین عملکرد را دارد؟ در عمق زیاد چه اتفاقی می‌افتد؟

بهترین ناحیه مربوط به عمق ۶ است که دقت اعتبارسنجی حدود ۰٫۷۳۸ دارد. در عمق‌های بسیار زیاد یا نامحدود، به‌خصوص زمانی که min_samples_splitکوچک باشد، درخت می‌تواند پیچیده‌تر شود و جزئیات بیشتری از داده‌های آموزش را یاد بگیرد. در نتیجه، احتمال بیش‌برازش افزایش می‌یابد و دقت اعتبارسنجی لزوماً بهتر نمی‌شود.

2.با مقایسه cv_train_accو cv_val_accکدام مدل‌ها بیش‌برازش دارند؟

مدل‌هایی که فاصله بیشتری بین دقت آموزش و اعتبارسنجی دارند، نشانه‌های بیشتری از بیش‌برازش نشان می‌دهند. در نتایج این آزمایش، درخت‌های بدون محدودیت عمق فاصله بیشتری بین این دو دقت دارند؛ بنابراین، نسبت به درخت‌های محدودشده به عمق ۶، بیشتر مستعد بیش‌برازش هستند.

3.آیا پیکربندی‌های برتر واقعاً تفاوت زیادی با هم دارند؟

خیر، اختلاف دقت اعتبارسنجی پیکربندی‌های برتر بسیار کم است و بسیاری از آن‌ها دقتی نزدیک به ۰٫۷۳۸ دارند. با توجه به انحراف معیار دقت اعتبارسنجی در اعتبارسنجی متقابل، این اختلاف‌های کوچک لزوماً نشان‌دهنده برتری واقعی یک مدل بر مدل دیگر نیستند. بنابراین، انتخاب مدل ساده‌تر می‌تواند منطقی‌تر باشد.

4.کدام پیکربندی را انتخاب می‌کنید و چرا؟

من پیکربندی دارای معیار gini، عمق ۶ و min_samples_leaf=40 و min_samples_split = 50را انتخاب می‌کنم؛ زیرا دقت اعتبارسنجی آن حدود ۰٫۷۳۸ است و در مقایسه با درخت‌های بدون محدودیت عمق، پیچیدگی کمتری دارد. همچنین، با توجه به شباهت عملکرد پیکربندی‌های برتر، افزایش پیچیدگی برای بهبود ناچیز دقت توجیه چندانی ندارد. البته معیار entropy با همین پیکربندی هم میتوان انتخاب کرد زیرا در آزمایش های قبل مشاهده شد که تفاوت میان معیار entropy و gini در داده های ترین و ولیدیشن بسیار اندک است.

5.چرا این روش از انتخاب بهترین نتیجه در یک تقسیم‌بندی اعتبارسنجی قابل‌اعتمادتر است؟

در اعتبارسنجی تقسیم بندی شده، مدل روی چند تقسیم‌بندی مختلف از داده‌ها ارزیابی می‌شود؛ بنابراین، نتیجه فقط به یک مجموعه اعتبارسنجی خاص وابسته نیست. محاسبه میانگین دقت، عملکرد کلی مدل را نشان می‌دهد و انحراف معیار میزان تغییرپذیری نتایج را مشخص می‌کند. در نتیجه، انتخاب مدل بر اساس این روش معمولاً قابل‌اعتمادتر از انتخاب مدلی است که صرفاً در یک تقسیم‌بندی بهترین دقت را کسب کرده است.

قسمت L : Final Model, Test Evaluation & Visualization

```
# TODO — final model

FINAL_CRITERION = suggested["criterion"]
FINAL_MAX_DEPTH = suggested["max_depth"]
FINAL_MIN_SAMPLES_SPLIT = suggested["min_samples_split"]
FINAL_MIN_SAMPLES_LEAF = suggested["min_samples_leaf"]

final_config = make_config(
    criterion=FINAL_CRITERION,
    max_depth=FINAL_MAX_DEPTH,
    min_samples_split=FINAL_MIN_SAMPLES_SPLIT,
    min_samples_leaf=FINAL_MIN_SAMPLES_LEAF
)


final_tree = DecisionTree(**final_config)
final_tree.fit(X_dev, y_dev)


y_dev_pred = final_tree.predict(X_dev)
dev_acc = accuracy_score_manual(y_dev, y_dev_pred)


cv = cross_validate(final_config)


model_complexity = complexity(final_tree)

print("Final configuration:", final_config)
print("Dev accuracy:", round(dev_acc, 4))
print(
    "CV validation accuracy:",
    f"{cv['cv_val_acc']:.4f} +/- {cv['cv_val_std']:.4f}"
)
print("Model complexity:", model complexity)
```

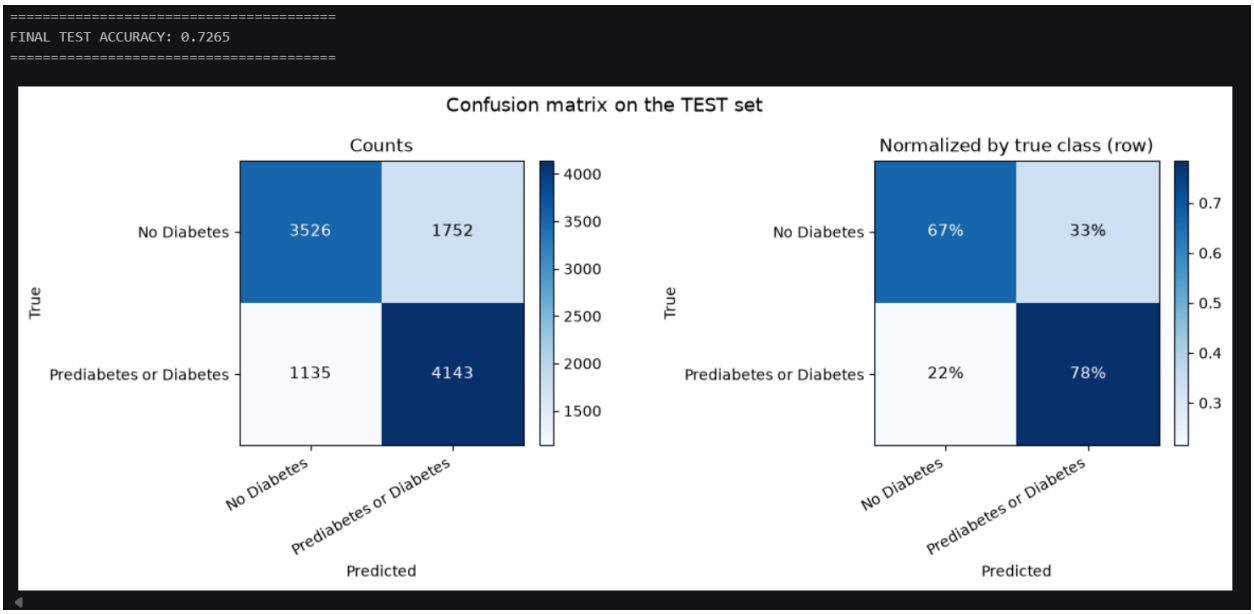
```
Final configuration: {'criterion': 'gini', 'max_depth': np.float64(4.0), 'min_samples_split': np.int64(50), 'min_samples_leaf': np.int64(40)}
Dev accuracy: 0.7315
CV validation accuracy: 0.7283 +/- 0.0047
Model complexity: {'depth': 4, 'nodes': 31, 'leaves': 16}
```

آموزش و ارزیابی مدل نهایی

در این مرحله، مدل نهایی بر اساس پیکربندی انتخاب‌شده از مرحله جست‌وجوی شبکه‌ای ساخته می‌شود. پارامترهای مدل شامل معیار تقسیم‌بندی gini، عمق حداکثر ۴، حداقل ۵۰ نمونه برای تقسیم هر گره و حداقل ۴۰ نمونه در هر برگ هستند. سپس مدل با استفاده از کل داده‌های توسعه (X_dev و y_dev)آموزش داده می‌شود.

پس از آموزش، دقت مدل روی داده‌های dev محاسبه شده و عملکرد آن با استفاده از اعتبارسنجی متقابل نیز ارزیابی می‌شود. دقت آموزش برابر با **۰٫۷۳۱۵** و میانگین دقت اعتبارسنجی برابر با **۰٫۷۲۸۳** با انحراف معیار **۰٫۰۰۴۷** است. نزدیک بودن این دو مقدار نشان می‌دهد که اختلاف عملکرد مدل روی داده‌های آموزش و اعتبارسنجی کم است.

از نظر پیچیدگی، درخت نهایی دارای عمق ۴، تعداد ۳۱ گره و ۱۶ برگ است که نشان دهنده ساختار نسبتا ساده درخت تصمیم ساخته شده است.



ارزیابی نهایی مدل روی مجموعه آزمون

در این مرحله، عملکرد مدل نهایی روی مجموعه تست مستقل بررسی می‌شود. دقت نهایی مدل برابر با **۰٫۷۲۶۵** است. ماتریس درهم‌ریختگی نشان می‌دهد که از میان نمونه‌های واقعی گروه «بدون دیابت»، تعداد ۳۵۲۶ نمونه به‌درستی در همین گروه قرار گرفته‌اند و ۱۷۵۲ نمونه به‌اشتباه در گروه «پیش‌دیابت یا دیابت» طبقه‌بندی

شـده‌اند. در گروه دوم نیز ۴۱۴۳ نمونه به‌درسـتی شـناسـایی شـده‌اند، درحالی‌که ۱۱۳۵ نمونه به‌اشـتباه در گروه «بدون دیابت» قرار گرفته‌اند.

نمودار نرمال‌شـــده نشـــان می‌دهد که مدل حدود **۶۷ درصـــد** از افراد بدون دیابت و **۷۸ درصـــد** از افراد دارای پیش‌دیابت یا دیابت را به‌درسـتی شـناسـایی می‌کند. بنابراین، مدل در تشـخیص گروه دوم عملکرد بهتری دارد و بخشی از افراد بدون دیابت را به‌اشتباه در گروه دارای پیش‌دیابت یا دیابت قرار می‌دهد.

```python
# TODO — precision, recall and F1 for every class, computed from the TEST set
y_test = np.asarray(y_test).ravel()
test_pred = np.asarray(test_pred).ravel()
rows = []
for c in all_classes:

    TP = np.sum((y_test == c) & (test_pred == c))
    FP = np.sum((y_test != c) & (test_pred == c))
    FN = np.sum((y_test == c) & (test_pred != c))
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0.0
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0.0
    f1 =( 2 * precision * recall / (precision + recall) if precision + recall > 0 else 0.0 )
    support = TP + FN

    rows.append({
        "class": c,
        "precision": precision,
        "recall": recall,
        "f1": f1,
        "support": support
    })

report_df = pd.DataFrame(rows).set_index("class")

macro_avg = report_df[["precision", "recall", "f1"]].mean()

weighted_avg = (
    report_df[["precision", "recall", "f1"]]
    .multiply(report_df["support"], axis=0)
    .sum()
    / report_df["support"].sum()
)

avg_df = pd.DataFrame(
    [macro_avg, weighted_avg],
    index=["macro avg", "weighted avg"]
)

print(report_df.round(3))
print(avg_df.round(3))
```

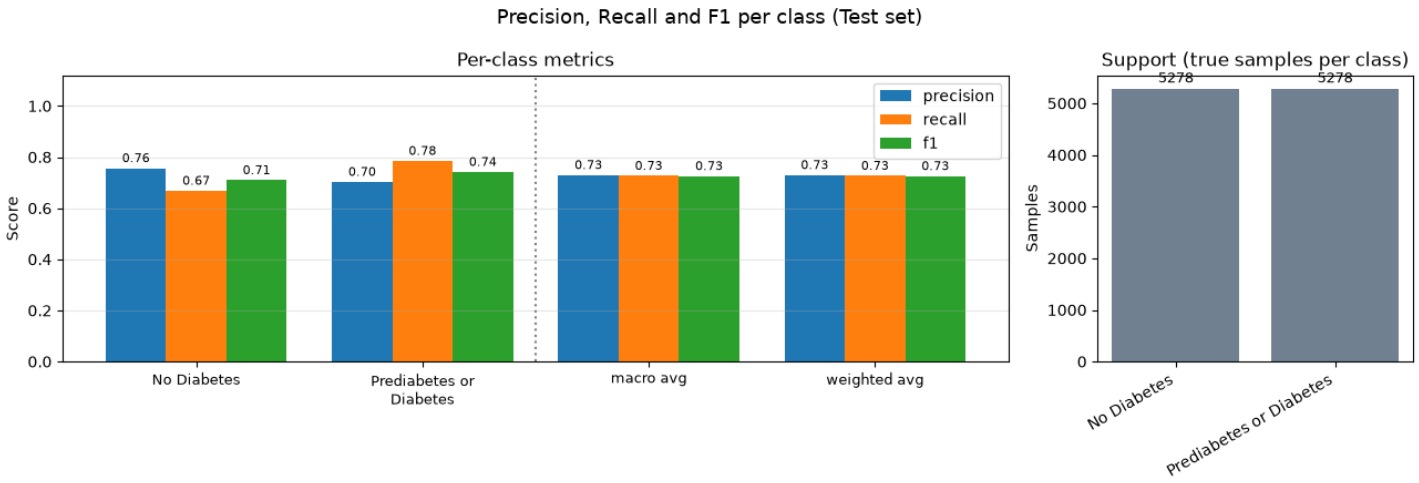
```
                     precision  recall     f1  support
class
No Diabetes              0.756   0.668  0.710     5278
Prediabetes or Diabetes  0.703   0.785  0.742     5278
              precision  recall     f1
macro avg          0.73   0.727  0.726
weighted avg       0.73   0.727  0.726
```

محاسبه معیارهای ارزیابی برای هر کلاس

در این بخش، معیارهای Precision، Recallو F1-scoreبرای هر کلاس با اســتفاده از پیش‌بینی‌های مدل روی مجموعه آزمون محاســبه می‌شــوند. برای هر کلاس، تعداد پیش‌بینی‌های صــحیح مثبت(TP) ، مثبت‌های اشـتباه (FP) و منفی‌های اشـتباه (FN) به دست می‌آید. سـپس Precision میزان صحت پیش‌بینی‌های مربوط به آن کلاس، Recallمیزان نمونه‌های واقعی شــناســایی‌شــده و F1-score میانگین هماهنگ این دو معیار را نشـــان می‌دهد. همچنین، مقدار supportتعداد نمونه‌های واقعی هر کلاس در مجموعه آزمون را مشـــخص می‌کند.

در ادامه، دو معیار میانگین نیز محاسـبه می‌شـوند Macro avg .میانگین سـاده معیارها در تمام کلاس‌هاسـت و به هر کلاس وزن یکسـانی می‌دهد؛ اما Weighted avgبا توجه به تعداد نمونه‌های واقعی هر کلاس محاسـبه می‌شود. در این آزمایش، هر دو کلاس دارای ۵۲۷۸ نمونه هسـتند؛ بنابراین، میانگین وزنی و میانگین سـاده نتایج یکسانی دارند.

بر اساس خروجی، برای کلاس «بدون دیابت» مقدار Precision برابر با ۰٫۷۵۶، مقدار Recall برابر با ۰٫۶۶۸ و مقدار F1-score برابر با ۰٫۷۱۰ اســـت. برای کلاس «پیش‌دیابت یا دیابت» نیز این مقادیر به‌ترتیب ۰٫۷۰۳، ۰٫۷۸۵ و ۰٫۷۴۲ هســـتند. این نتایج نشـــان می‌دهند که مدل نمونه‌های گروه پیش‌دیابت یا دیابت را بهتر شناسایی می‌کند، اما در تشخیص افراد بدون دیابت، Recallپایین‌تری دارد.




```
Check passed: weighted recall = accuracy = 0.7265
```

پاسخ سؤالات بخشPrecision ، RecallوF1

1. مقایسه Accuracy وMacro F1

دقت کلی مدل (Accuracy) برابر با ۰٫۷۲۶۵ و مقدار Macro F1برابر با ۰٫۷۲۶ است. بنابراین، Accuracy اندکی بالاتر است، اما اختلاف بسیار ناچیز است. این نزدیکی نشان می‌دهد که عملکرد کلی مدل و میانگین عملکرد آن در دو کلاس تقریباً مشابه است؛ با این حال، برای بررسی تفاوت عملکرد بین کلاس‌ها باید Precisionو Recall هر کلاس را جداگانه بررسی کرد.

2 .کدام کلاس کمترین Recall و کدام کلاس کمترین Precision را دارد؟

- **کمترین :Recall** کلاس «بدون دیابت» با مقدار ۰٫۶۶۸. یعنی مدل تعدادی از افراد واقعاً بدون دیابت را به‌اشتباه در گروه پیش‌دیابت یا دیابت قرار می‌دهد؛ در واقع، این افراد از کلاس واقعی خود جا می‌مانند.
- **کمترین :Precision** کلاس «پیش‌دیابت یا دیابت» با مقدار ۰٫۷۰۳. یعنی تعدادی از افرادی که مدل در این گروه قرار می‌دهد، در واقع بدون دیابت هستند؛ بنابراین، مدل برای این کلاس هشدارهای اشتباه ایجاد می‌کند.

3 .توضیح یک معیار با استفاده از ماتریس درهم‌ریختگی

برای کلاس (پیش‌دیابت یا دیابت)

- TP = 4143: نمونه‌های واقعی این کلاس که درست پیش‌بینی شده‌اند؛ خانه پایین سمت راست ماتریس.
- FP = 1752: نمونه‌های واقعی «بدون دیابت» که اشتباهاً در این کلاس پیش‌بینی شده‌اند؛ خانه بالا سمت راست.
- FN = 1135: نمونه‌های واقعی «پیش‌دیابت یا دیابت» که اشتباهاً «بدون دیابت» پیش‌بینی شده‌اند؛ خانه پایین سمت چپ.

بنابراین، Precisionاین کلاس برابر است با $(4143/(4143 + 1752) \approx 0.703)$ و Recall آن برابر است با.$(4143/(4143 + 1135) \approx 0.785)$

4 .مثال‌هایی که Recall یا Precision اهمیت بیشتری دارند

- **اهمیت بیشتر Recall:** غربالگری بیماری‌ها؛ بهتر است تعداد کمتری از افراد بیمار از دست بروند، حتی اگر تعدادی از افراد سالم اشتباهاً مشکوک تشخیص داده شوند.
- **اهمیت بیشتر Precision:** شناسایی پیام‌های اسپم در صندوق ایمیل کاری مهم؛ بهتر است پیام‌های عادی به‌اشتباه اسپم شناخته نشوند، چون ممکن است پیام مهمی از دست برود.

5 .چرا F1 از میانگین هارمونیک استفاده می‌کند؟

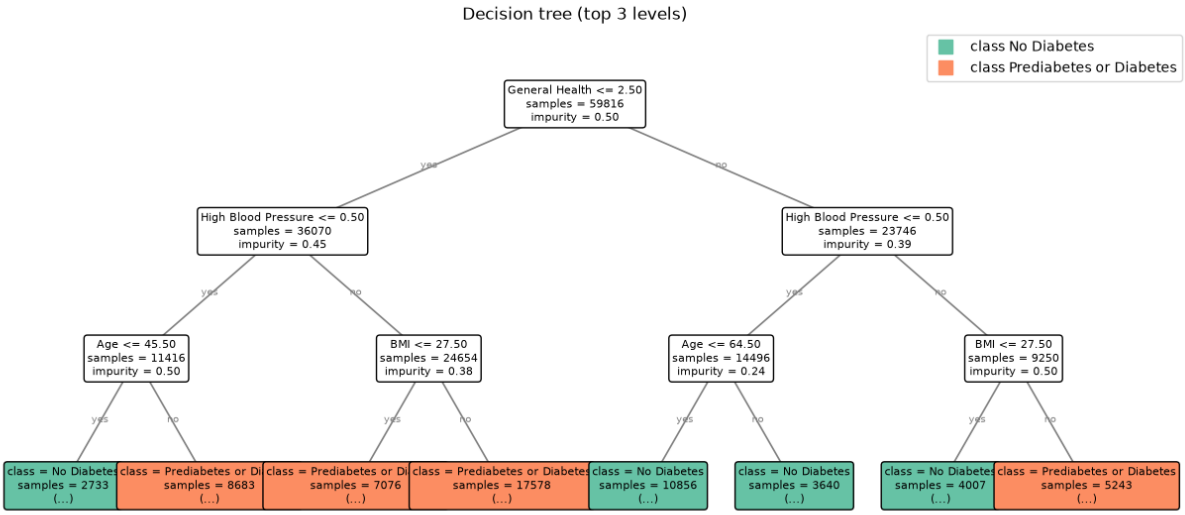
میانگین هارمونیک باعث می‌شود اگر یکی از دو معیار Precision یا Recall خیلی پایین باشد، مقدار F1 نیز پایین بماند. برای مثال، اگر Precision برابر با ۱ و Recall برابر با ۰٫۰۱ باشد، میانگین معمولی آن‌ها برابر با ۰٫۵۰۵ است؛ اما F1 تقریباً برابر با ۰٫۰۱۹۸ خواهد بود. این مقدار نشان می‌دهد که عملکرد مدل با وجود Precisionکامل، به دلیل Recall بسیار پایین مناسب نیست.

6 .چه زمانی Macro و Weighted Average تفاوت زیادی دارند؟

زمانی که داده‌ها نامتوازن باشند و تعداد نمونه‌های کلاس‌ها تفاوت زیادی داشته باشد، این دو میانگین ممکن است اختلاف زیادی پیدا کنند Macro avg .به همه کلاس‌ها وزن یکسان می‌دهد، اما Weighted avgبه کلاس‌های دارای نمونه‌های بیشتر وزن بالاتری می‌دهد. در داده‌های نامتوازن، گزارش هر دو معیار مفید است؛ به‌ویژه Macro Average برای بررسی عملکرد روی کلاس‌های کم‌نمونه اهمیت دارد، زیرا اجازه نمی‌دهد عملکرد ضعیف یک کلاس کوچک پشت عملکرد کلاس بزرگ پنهان شود.

7 . اگر Precision و Recall یک کلاس هر دو صفر باشند، چه معنایی دارد؟ آیا Accuracy این مشکل را نشان می‌دهد؟

این وضعیت می‌تواند به این معنا باشد که مدل هیچ نمونه‌ای را به‌درستی به آن کلاس نسبت نداده است؛ برای مثال، ممکن است اصلاً آن کلاس را پیش‌بینی نکرده باشد یا تمام پیش‌بینی‌های مربوط به آن اشتباه باشند. در این حالت، F1نیز صفر می‌شود. Accuracy لزوماً این مشکل را نشان نمی‌دهد؛ به‌خصوص در داده‌های نامتوازن، مدل ممکن است با پیش‌بینی درست کلاس بزرگ‌تر، Accuracyبالایی داشته باشد، درحالی‌که کلاس کوچک‌تر را کاملاً نادیده می‌گیرد.



بررسی ساختار درخت تصمیم نهایی

این نمودار سه سطح ابتدایی درخت تصمیم نهایی را نمایش می‌دهد. اولین ویژگی انتخاب‌شده در ریشه درخت، General Healthبا آستانه ۲٫۵ است؛ پس از آن، در هر دو شاخه، ویژگی High Blood Pressureبررسی می‌شود و در مراحل بعدی، ویژگی‌های Ageو BMIبرای تقسیم بیشتر نمونه‌ها به کار می‌روند.

پاسخ سؤالات بخشL

1 . آیا دقت آزمون به نتیجه اعتبارسنجی متقابل نزدیک است؟ اگر اختلاف زیادی داشته باشند، چه معنایی دارد؟

بله، دقت آزمون برابر با ۰٫۷۲۶۵ و میانگین دقت اعتبارسنجی متقابل برابر با ۰٫۷۲۸۳ است. اگر اختلاف زیادی وجود داشت، ممکن بود نشان‌دهنده تفاوت توزیع مجموعه‌ها یا تعمیم‌پذیری ضعیف مدل باشد.

2 . چرا انتخاب پیکربندی با توجه به دقت مجموعه آزمون اشتباه است؟

مجموعه آزمون باید فقط برای ارزیابی نهایی مدل استفاده شود. اگر پیکربندی‌ها را بر اساس دقت آزمون انتخاب کنیم، اطلاعات این مجموعه وارد فرایند انتخاب مدل می‌شود و دیگر ارزیابی کاملاً مستقلی نخواهیم داشت. بنابراین، بهتر است پارامترها با استفاده از داده‌های توسعه و اعتبارسنجی متقابل انتخاب شوند و مجموعه آزمون فقط یک‌بار برای سنجش عملکرد نهایی به کار رود.

3 . در ماتریس درهم‌ریختگی، مدل کدام کلاس را بیشتر اشتباه می‌گیرد؟

مدل در تشخیص کلاس «بدون دیابت» خطای بیشتری دارد. تعداد ۱۷۵۲ نمونه از این کلاس به‌اشتباه در گروه «پیش‌دیابت یا دیابت» قرار گرفته‌اند، درحالی‌که ۱۱۳۵ نمونه از گروه «پیش‌دیابت یا دیابت» به‌اشتباه «بدون دیابت» پیش‌بینی شده‌اند. بنابراین، مدل بیشتر افراد بدون دیابت را به‌اشتباه در گروه پیش‌دیابت یا دیابت قرار می‌دهد.

Final Reflection

1 . Overfitting و Underfitting

در آزمایش‌ها، درخت بدون محدودیت عمق دقت آموزش تقریباً ٪۱۰۰ و دقت اعتبارسنجی حدود ٪۶۵،۶ داشت که نشان‌دهنده بیش‌برازش بود و حفظ داده های آموزشی بود. در مقابل، محدود کردن بیش از حد عمق می‌تواند باعث کم‌برازش شود؛ زیرا مدل برای یادگیری الگوهای داده پیچیدگی کافی ندارد.

2. تفاوت Pre-Pruning و Post-Pruning

درPre-Pruning ، رشد درخت از ابتدا با محدودیت‌هایی مثل عمق و حداقل تعداد نمونه کنترل می‌شود. در Post-Pruning، ابتدا درخت کامل ساخته شده و سپس بخش‌هایی از آن حذف می‌شوند. در آزمایش من، مدل هرس اولیه با ۱۴۴۱ گره به دقت اعتبارسنجی ۰،۷۳۴ رسید، درحالی‌که هرس پس از ساخت، تعداد گره‌ها را از ۲۵۵۰۳ به ۱۰۲۸۱ کاهش داد و دقت اعتبارسنجی را به ۰،۷۵۰ رساند.

3. آیا Gini و Information Gain همیشه درخت یکسانی می‌سازند؟

خیر. این دو معیار ممکن است تقسیم‌های متفاوتی انتخاب کنند؛ اما در آزمایش من با عمق ۴، دقت اعتبارسنجی هر دو تقریباً ۰،۷۳۱ بود و ساختار کلی درخت‌هایشان شباهت زیادی داشت.

4. چرا به مجموعه Validation و Test هر دو نیاز داریم؟

مجموعه Validation برای تنظیم پارامترها و انتخاب مدل استفاده می‌شود، اما مجموعه Test برای ارزیابی نهایی مدل روی داده‌های مستقل است. اگر از Test برای انتخاب مدل استفاده کنیم، ارزیابی نهایی دیگر کاملاً مستقل نخواهد بود و مدل ممکن است داده های تست رو به خاطر بسپرد.

5 . چرا K-Fold CV بهتر از یک تقسیم‌بندی اعتبارسنجی است و هزینه آن چیست؟

در K-Fold ، مدل روی چند تقسیم‌بندی مختلف ارزیابی می‌شود؛ بنابراین، نتیجه کمتر به یک تقسیم‌بندی خاص وابسته است و میانگین و انحراف معیار دقت نیز قابل محاسبه‌اند. هزینه آن این است که مدل چندین بار آموزش داده می‌شود و زمان محاسباتی بیشتری نیاز دارد. همانطور که در گرید سرچ مشاهده شد زمان اجرا به حدود ۳ ساعت طول کشید.

6. چه رابطه‌ای بین دقت و پیچیدگی مشاهده شد و کدام مدل را انتخاب می‌کنید؟

درخت بسیار بزرگ با ۲۵۵۰۳ گره دقت آموزش تقریباً ٪۱۰۰ داشت، اما دقت اعتبارسنجی آن فقط ۰٫۶۵۶ بود. در مقابل، مدل نهایی با عمق ۴، تعداد ۳۱ گره و ۱۶ برگ، دقت اعتبارسنجی ۰٫۷۲۸۳ و دقت آزمون ۰٫۷۲۶۵ داشت. من مدل نهایی را انتخاب می‌کنم، زیرا با پیچیدگی بسیار کمتر، عملکرد مناسبی دارد و نتایج اعتبارسنجی و تست آن نیز به یکدیگر نزدیک‌اند.

جدول خلاصه تمامی مدل های آزمایش ها

Model	Criterion	Pruning	Train Acc.	Val Acc. (or CV mean)	Depth	Nodes	Leaves	Test Acc.
Baseline (majority)	–	–			–	–	–	–
Unrestricted	Information Gain	None	0.9988	0.6563	38.0	25503	12752	–
Pre-Pruned	Information Gain	Pre	0.7596	0.7344	19.0	1441	721	–
Post-Pruned	Information Gain	Post	0.8727	0.7501	35.0	10281	5141	–
Gini (depth 4)	Gini	Depth	0.7315	0.7311	4	31	16	–
Final (Grid Search + CV)	gini	Grid + CV	0.7315	0.7283	4	31	16	0.7265

